



Competitive Security Assessment

Mantle V2

Mar 4th, 2024



Summary	4
Overview	5
Audit Scope	6
Code Assessment Findings	0
MNT-1 relayMessage() should directly transfer MNT or ETH to target, otherwise they will be lost	12
MNT-2 Potential reentrancy attack in <code>relayMessage()</code>	17
MNT-3 Pontential lock of ERC721 assets in <code>L1ERC721Bridge</code>	24
MNT-4 NFTs can be stuck in the protocol many NFTs have <code>pause</code> functionality	31
MNT-5 Malicious actor can steal deposited MNT/BMV_ETH token	34
MNT-6 L1 -> L2 messages with large calldata that fail will not be replayable	45
MNT-7 Incorrect revert check in function <code>finalizeWithdrawalTransaction()</code>	46
MNT-8 Funds transferred to incorrect user before snapshot	55
MNT-9 <code>core/state_transition.go::TransitionDb</code> will runtime panic and crash node by calling create contract with non-zero eth value	57
MNT-10 Wrong estimation of the <code>RELAY_RESERVED_GAS</code> might cause a DoS	59
MNT-11 Vulnerability in getPriceFromUniswap function	62
MNT-12 Use of <code>safeTransferFrom</code> instead of <code>transferFrom</code> in L1ERC721Bridge.sol	63
MNT-13 The price used when calculating <code>tokenRatio</code> will always come from <code>ceX</code>	64
MNT-14 The function <code>getTokenPricesFromUniswap</code> may return incorrect prices	68
MNT-15 The <code>txpool.go</code> does not properly remove invalid transactions by including expired sponsor payment to the sponsor cost sum	71
MNT-16 Some of the invalid transactions will not be deleted from the future queue	73
MNT-17 SELFDESTRUCT will be deprecated	77
MNT-18 Resource params will be reset when reinitializing <code>ResourceMetering</code> contract	78
MNT-19 Relay message on L1 will fail due to incorrect gas estimation	80
MNT-20 Reinitialization causes metering parameter to be reset	81
MNT-21 Overestimation could lead to permanent locking of funds on L1	83
MNT-22 Missing paranthesis causes fund loss	84
MNT-23 Lack of distribution of <code>BVM_ETH</code> for the call of <code>BVM_ETH.approve()</code>	86
MNT-24 Lack of check <code>st.msg.To</code> in function <code>TransitionDb()</code>	89
MNT-25 Insufficient Validation of Sponsor Balance in Meta Transactions	91
MNT-26 Incorrect return value in function <code>getTokenPricesFromUniswap()</code>	95
MNT-27 Incorrect calculation for the cost of the replaced transaction in <code>validateTx</code>	97
MNT-28 Incorrect Calculation of Median in Non-Zero Element Filtering Logic	100
MNT-29 If the token's decimal is not 18, the function <code>getTokenPriceFromUniswap</code> will return the wrong price	102

Exploitation of Fixed <code>fromAmount</code> in Price Queries on Low Liquidity Pools	104
MNT-31 EigenDA/MantleDA is not checked validated for liveness	105
MNT-32 ERC 20 approve is assigned twice and validated once.	107
MNT-33 EOAs depositing ETH through L1CrossDomainMessenger will lose their funds.	108
MNT-34 Due to incorrectly calculation of the user balance, the <code>txpool.go</code> does not properly remove invalid transactions	110
MNT-35 Could not estimate the exact gas cost in <code>finalizeWithdrawalTransaction</code> when <code>mntValue</code> is 0	112
MNT-36 Calling the <code>bridgeMNT</code> function in contract <code>L2StandardBridge</code> may result in a loss of funds	114
MNT-37 Bridge Insolvency Due to Rebasing, Deflationary or Fee-on-Transfer Tokens	116
MNT-38 deposited funds by the contract can be lost	119
MNT-39 <code>BVM_ETH</code> can be minted to <code>address(0)</code>	121
MNT-40 Use Two-Step Transfer Pattern for Access Controls	123
MNT-41 Unsafe type conversion	125
MNT-42 The amount of gas to reserve for the caller after a sub call in <code>relayMessage</code> is smaller than expected	127
MNT-43 Overdraft check is incorrect	131
MNT-44 No storage gap for upgradeable contract might lead to storage slot collision	133
MNT-45 Logic Vulnerability in <code>CalculateSponsorPercentAmount</code> Function Allowing Zero or Undefined Transaction Costs	134
MNT-46 Lack of nil check for <code>StateTransition.msg.To</code>	138
MNT-47 L2StandardBridge.bridgeMNT should add onlyEOA modifier	139
MNT-48 Known issues with compiler versions used to compile the contracts (sol 0.8.15 & 0.6.0)	141
MNT-49 Incorrect sponsor balance check	143
MNT-50 Incorrect return check for the available balance	145
MNT-51 Import Contract not exist	147
MNT-52 Gaps should be reserved for FeeVault	148
MNT-53 Flawed algorithm in tokenRatio function	150
MNT-54 Errors are not returned in the function <code>wrapUpdateTokenRatio</code>	153
Disclaimer	156

Summary

This report is prepared for the project to identify vulnerabilities and issues in the smart contract source code. A group of NDA covered experienced security experts have participated in the Secure3's Audit Contest to find vulnerabilities and optimizations. Secure3 team has participated in the contest process as well to provide extra auditing coverage and scrutiny of the finding submissions.

The comprehensive examination and auditing scope includes:

- Cross checking contract implementation against functionalities described in the documents and white paper disclosed by the project owner.
- Contract Privilege Role Review to provide more clarity on smart contract roles and privilege.
- Using static analysis tools to analyze smart contracts against common known vulnerabilities patterns.
- Verify the code base is compliant with the most up-to-date industry standards and security best practices.
- Comprehensive line-by-line manual code review of the entire codebase by industry experts.

The security assessment resulted in findings that are categorized in four severity levels: Critical, Medium, Low, Informational. For each of the findings, the report has included recommendations of fix or mitigation for security and best practices.

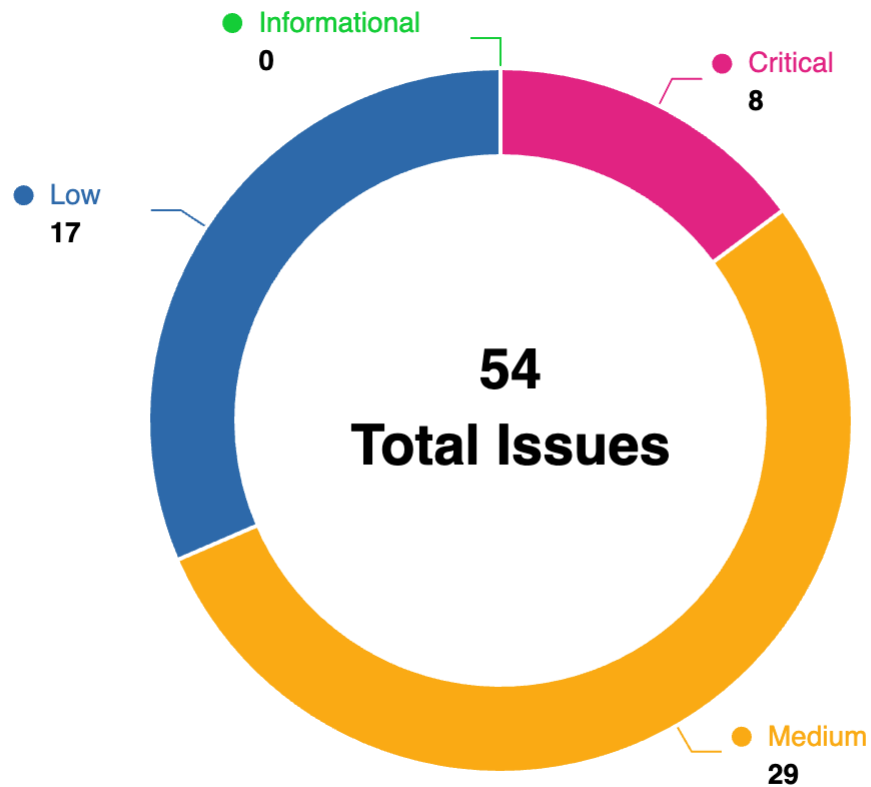
Overview

Project Name	Mantle
Language	Solidity and Go
Codebase	• https://github.com/mantlenetworkio/mantle-v2/ • audit commit - 00ce7b04bef0b4866200b9fcf98633d475290792 • final commit - 7040d029eefc7a2d5a33e03bc15d6815e4a25fd6 • https://github.com/mantlenetworkio/op-geth • audit commit - be91998dbc17f392fb4c5960fcf5b497227cc31a • final commit - 64996df634fbd58d9eea82cd4cf7bf3a782c2e03
Audit Methodology	• Audit Contest • Business Logic and Code Review • Privileged Roles Review • Static Analysis

Audit Scope

File	SHA256 Hash
mantle-v2	https://github.com/mantlenetworkio/mantle-v2/pull/72
op-geth	https://github.com/mantlenetworkio/op-geth/pull/20
mantle-v2	packages/contracts-bedrock/contracts/L1
mantle-v2	packages/contracts-bedrock/contracts/L2

Code Assessment Findings



ID	Name	Category	Severity	Client Response	Contributor
MNT-1	relayMessage() should directly transfer MNT or ETH to target, otherwise they will be lost	Logical	Critical	Fixed	thereksfour
MNT-2	Potential reentrancy attack in <code>relayMessage()</code>	Logical	Critical	Fixed	Hacker007
MNT-3	Pontential lock of ERC721 assets in <code>L1ERC721Bridge</code>	Logical	Critical	Fixed	biakia, Yaoda o
MNT-4	NFTs can be stuck in the protocol many NFTs have <code>pause</code> functionality	Logical	Critical	Fixed	rajatbeladiya
MNT-5	Malicious actor can steal deposited MNT/BMV_ETH token	Logical	Critical	Fixed	SerSomeone, lemonmon, csanuragain, newway55

MNT-6	L1 -> L2 messages with large calldata that fail will not be replayable	Logical	Critical	Acknowledged	SerSomeone
MNT-7	Incorrect revert check in function <code>finalizeWithdrawalTransaction()</code>	Logical	Critical	Fixed	lemonmon, newway55, rajatbeladiya, Yaodao, biakia
MNT-8	Funds transferred to incorrect user before snapshot	Logical	Critical	Fixed	csanuragjain
MNT-9	<code>core/state_transition.go::TransitionDb</code> will runtime panic and crash node by calling create contract with non-zero eth value	Language Specific	Medium	Fixed	lemonmon
MNT-10	Wrong estimation of the <code>RELAY_RESERVED_GAS</code> might cause a DoS	DOS	Medium	Acknowledged	newway55
MNT-11	Vulnerability in <code>getPriceFromUniswap</code> function	Logical	Medium	Acknowledged	zircon
MNT-12	Use of <code>safeTransferFrom</code> instead of <code>transferFrom</code> in L1 ERC721Bridge.sol	Logical	Medium	Fixed	rajatbeladiya
MNT-13	The price used when calculating <code>tokenRatio</code> will always come from <code>cex</code>	Logical	Medium	Acknowledged	biakia
MNT-14	The function <code>getTokenPricesFromUniswap</code> may return incorrect prices	Logical	Medium	Acknowledged	biakia
MNT-15	The <code>txpool.go</code> does not properly remove invalid transactions by including expired sponsor payment to the sponsor cost sum	Logical	Medium	Fixed	lemonmon
MNT-16	Some of the invalid transactions will not be deleted from the future queue	Logical	Medium	Acknowledged	biakia

MNT-17	SELFDESTRUCT will be deprecated	Logical	Medium	Acknowledged	rajatbeladiya
MNT-18	Resource params will be reset when reinitializing ResourceMetering contract	Logical	Medium	Fixed	Hacker007
MNT-19	Relay message on L1 will fail due to incorrect gas estimation	Logical	Medium	Acknowledged	Oxffchain, lemonmon
MNT-20	Reinitialization causes metering parameter to be reset	Logical	Medium	Fixed	lemonmon
MNT-21	Overestimation could lead to permanent locking of funds on L1	Logical	Medium	Acknowledged	Oxffchain
MNT-22	Missing paranthesis causes fund loss	Logical	Medium	Fixed	csanuragjain
MNT-23	Lack of distribution of BVM_ETH for the call of BVM_ETH.prove()	Logical	Medium	Acknowledged	Yaodao
MNT-24	Lack of check st.msg.To in function TransitionDb()	Logical	Medium	Fixed	biakia
MNT-25	Insufficient Validation of Sponsor Balance in Meta Transactions	Logical	Medium	Fixed	BradMoonUE STC
MNT-26	Incorrect return value in function getTokenPricesFromUniswap()	Logical	Medium	Acknowledged	Yaodao
MNT-27	Incorrect calculation for the cost of the replaced transaction in validateTx	Logical	Medium	Acknowledged	biakia
MNT-28	Incorrect Calculation of Median in Non-Zero Element Filtering Logic	Logical	Medium	Fixed	BradMoonUE STC
MNT-29	If the token's decimal is not 18, the function getTokenPriceFromUniswap will return the wrong price	Logical	Medium	Acknowledged	biakia

MNT-30	Exploitation of Fixed <code>fromAmount</code> in Price Queries on Low Liquidity Pools	Logical	Medium	Acknowledged	BradMoonUE STC
MNT-31	EigenDA/MantleDA is not checked validated for liveness	Logical	Medium	Acknowledged	Oxffchain
MNT-32	ERC 20 approve is assigned twice and validated once.	Logical	Medium	Fixed	Oxffchain
MNT-33	EOAs depositing ETH through L1CrossDomainMessenger will lose their funds.	Logical	Medium	Fixed	SerSomeone
MNT-34	Due to incorrect calculation of the user balance, the <code>txpool.go</code> does not properly remove invalid transactions	Logical	Medium	Fixed	lemonmon
MNT-35	Could not estimate the exact gas cost in <code>finalizeWithdrawalTransaction</code> when <code>mntValue</code> is 0	DOS	Medium	Fixed	lemonmon, Hacker007
MNT-36	Calling the <code>bridgeMNT</code> function in contract <code>L2StandardBridge</code> may result in a loss of funds	Logical	Medium	Fixed	biakia
MNT-37	Bridge Insolvency Due to Rebasing, Deflationary or Fee-on-Transfer Tokens	Logical	Medium	Acknowledged	rajatbeladiya
MNT-38	deposited funds by the contract can be lost	Logical	Low	Acknowledged	rajatbeladiya
MNT-39	<code>BVM_ETH</code> can be minted to <code>address(0)</code>	Logical	Low	Acknowledged	biakia
MNT-40	Use Two-Step Transfer Pattern for Access Controls	Logical	Low	Acknowledged	thereksfour
MNT-41	Unsafe type conversion	Code Style	Low	Acknowledged	biakia

MNT-42	The amount of gas to reserve for the caller after a sub call in <code>relayMessage</code> is smaller than expected	Logical	Low	Fixed	biakia
MNT-43	Overdraft check is incorrect	Logical	Low	Acknowledged	csanuragjain
MNT-44	No storage gap for upgradeable contract might lead to storage slot collision	Code Style	Low	Acknowledged	rajatbeladiya
MNT-45	Logic Vulnerability in <code>CalculateSponsorPercentAmount</code> Function Allowing Zero or Undefined Transaction Costs	Logical	Low	Fixed	BradMoonUE STC
MNT-46	Lack of nil check for <code>StateTransition.msg.To</code>	Logical	Low	Fixed	Yaodao
MNT-47	<code>L2StandardBridge.bridgeMNT</code> should add onlyEOA modifier	Logical	Low	Fixed	thereksfour
MNT-48	Known issues with compiler versions used to compile the contracts (sol 0.8.15 & 0.6.0)	Logical	Low	Acknowledged	rajatbeladiya
MNT-49	Incorrect sponsor balance check	Logical	Low	Fixed	biakia
MNT-50	Incorrect return check for the available balance	Logical	Low	Fixed	Yaodao
MNT-51	Import Contract not exist	Logical	Low	Declined	BradMoonUE STC
MNT-52	Gaps should be reserved for FeeVault	Logical	Low	Acknowledged	thereksfour
MNT-53	Flawed algorithm in tokenRatio function	Logical	Low	Acknowledged	zircon
MNT-54	Errors are not returned in the function <code>wrapUpdateTokenRatio</code>	Code Style	Low	Fixed	biakia

MNT-1:relayMessage() should directly transfer MNT or ETH to target, otherwise they will be lost

Category	Severity	Client Response	Contributor
Logical	Critical	Fixed	thereksfour

Code Reference

- code/mantle-v2/packages/contracts-bedrock/contracts/L1/L1CrossDomainMessenger.sol#L213-L221

```
....  
213: if (_mntValue!=0){  
214:     mntSuccess = IERC20(L1_MNT_ADDRESS).approve(_target, _mntValue);  
215: }  
216: xDomainMsgSender = _sender;  
217: bool success = SafeCall.call(_target, gasleft() - RELAY_RESERVED_GAS, _ethValue, _message);  
218: xDomainMsgSender = Constants.DEFAULT_L2_SENDER;  
219: if (_mntValue!=0){  
220:     mntSuccess = IERC20(L1_MNT_ADDRESS).approve(_target, 0);  
221: }
```

Description

thereksfour: After the user calls `L1CrossDomainMessenger.sendMessage()` on L1, OtherMessenger on L2 will call `L2CrossDomainMessenger.relayMessage()` to relay the message on L2.

```

function sendMessage(
    uint256 _mntAmount,
    address _target,
    bytes calldata _message,
    uint32 _minGasLimit
) external payable override {
    if (_mntAmount != 0) {
        IERC20(L1_MNT_ADDRESS).safeTransferFrom(msg.sender, address(this), _mntAmount);
        bool success = IERC20(L1_MNT_ADDRESS).approve(address(PORTAL), _mntAmount);
        require(success, "the approve for L1 mnt to OptimismPortal failed");
    }

    // Triggers a message to the other messenger. Note that the amount of gas provided to the
    // message is the amount of gas requested by the user PLUS the base gas value. We want to guarantee
    // the property that the call to the target contract will always have at least the minimum gas
    // limit specified by the user.
    _sendMessage(
        _mntAmount,
        OTHER_MESSENGER,
        baseGas(_message, _minGasLimit),
        abi.encodeWithSelector(
            L2CrossDomainMessenger.relayMessage.selector,
            messageNonce(),
            msg.sender,
            _target,
            _mntAmount,
            msg.value,
            _minGasLimit,
            _message
        )
    );
}

```

The problem here is that when there is ETH in the message, it will be represented as BVM_ETH on L2, and only the target will be approved to use BVM_ETH in L2.relayMessage, instead of sending BVM_ETH to the user directly.

```

bool ethSuccess = true;
if (_ethValue != 0) {
    ethSuccess = IERC20(Predelays.BVM_ETH).approve(_target, _ethValue);
}
xDomainMsgSender = _sender;
bool success = SafeCall.call(_target, gasleft() - RELAY_RESERVED_GAS, _mntValue, _message);

xDomainMsgSender = Constants.DEFAULT_L2_SENDER;
if (_ethValue != 0) {
    ethSuccess = IERC20(Predelays.BVM_ETH).approve(_target, 0);
}

```

```

bool mntSuccess =true;
    if (_mntValue!=0){
        mntSuccess = IERC20(L1_MNT_ADDRESS).approve(_target, _mntValue);
    }
    xDomainMsgSender = _sender;
    bool success = SafeCall.call(_target, gasleft() - RELAY_RESERVED_GAS, _ethValue, _message);
    xDomainMsgSender = Constants.DEFAULT_L2_SENDER;
    if (_mntValue!=0){
        mntSuccess = IERC20(L1_MNT_ADDRESS).approve(_target, 0);
    }

```

While L1/L2CrossDomainMessenger work fine when combined with L1/L2StandardBridge because L1/L2StandardBridge will transfer tokens from L1/L2CrossDomainMessenger. However, when the user uses L1/L2CrossDomainMessenger directly, the user is compromised.

```

function finalizeBridgeMNT(
    address _from,
    address _to,
    uint256 _amount,
    bytes calldata _extraData
) public payable override onlyOtherBridge {
    require(_to != address(this), "StandardBridge: cannot send to self");
    require(_to != address(MESSENGER), "StandardBridge: cannot send to messenger");

    IERC20(L1_MNT_ADDRESS).safeTransferFrom(address(MESSENGER), _to, _amount);
    ...
function finalizeBridgeETH(
    address _from,
    address _to,
    uint256 _amount,
    bytes calldata _extraData
) public payable override onlyOtherBridge {
    require(_to != address(this), "StandardBridge: cannot send to self");
    require(_to != address(MESSENGER), "StandardBridge: cannot send to messenger");
    // Emit the correct events. By default this will be _amount, but child
    // contracts may override this function in order to emit legacy events as well.

    //move the BVM_ETH mint to op-geth.
    IERC20(Predelays.BVM_ETH).safeTransferFrom(Predelays.L2_CROSS_DOMAIN_MESSENGER, _to, _amount);
}

```

Recommendation

thereksfour: Change `L1CrossDomainMessenger.relayMessage()` and `L2CrossDomainMessenger.relayMessage()` as follows

```

bool mntSuccess = true;
    if (_mntValue!=0){
-        mntSuccess = IERC20(L1_MNT_ADDRESS).approve(_target, _mntValue);+        mntSuccess
= IERC20(L1_MNT_ADDRESS).transfer(_target, _mntValue);
    }
    xDomainMsgSender = _sender;
    bool success = SafeCall.call(_target, gasleft() - RELAY_RESERVED_GAS, _ethValue, _messag
e);

    xDomainMsgSender = Constants.DEFAULT_L2_SENDER;
-    if (_mntValue!=0){-        mntSuccess = IERC20(L1_MNT_ADDRESS).approve(_target, 0);-
}

```

```

bool ethSuccess = true;
if (_ethValue != 0) {
-    ethSuccess = IERC20(Predeploys.BVM_ETH).approve(_target, _ethValue);
+    ethSuccess = IERC20(Predeploys.BVM_ETH).transfer(_target, _ethValue);
}
xDomainMsgSender = _sender;
bool success = SafeCall.call(_target, gasleft() - RELAY_RESERVED_GAS, _mntValue, _messag
e);

xDomainMsgSender = Constants.DEFAULT_L2_SENDER;
-    if (_ethValue != 0) {
-        ethSuccess = IERC20(Predeploys.BVM_ETH).approve(_target, 0);
-    }

```

And change L1StandardBridge.finalizeBridgeMNT() and L2StandardBridge.finalizeBridgeETH() as follows

```

function finalizeBridgeMNT(
    address _from,
    address _to,
    uint256 _amount,
    bytes calldata _extraData
) public payable override onlyOtherBridge {
    require(_to != address(this), "StandardBridge: cannot send to self");
    require(_to != address(MESSENGER), "StandardBridge: cannot send to messenger");

-    IERC20(L1_MNT_ADDRESS).safeTransferFrom(address(MESSENGER), _to, _amount);
+    IERC20(L1_MNT_ADDRESS).safeTransfer(_to, _amount);

    // Emit the correct events. By default this will be ERC20BridgeFinalized, but child
    // contracts may override this function in order to emit legacy events as well.
    _emitMNTBridgeFinalized(_from, _to, _amount, _extraData);
}

```

```
function finalizeBridgeETH(
    address _from,
    address _to,
    uint256 _amount,
    bytes calldata _extraData
) public payable override onlyOtherBridge {
    require(_to != address(this), "StandardBridge: cannot send to self");
    require(_to != address(MESSENGER), "StandardBridge: cannot send to messenger");
    // Emit the correct events. By default this will be _amount, but child
    // contracts may override this function in order to emit legacy events as well.

    //move the BVM_ETH mint to op-geth.
    - IERC20(Predeploys.BVM_ETH).safeTransferFrom(Predeploys.L2_CROSS_DOMAIN_MESSENGER, _to, _am
    ount);+ IERC20(Predeploys.BVM_ETH).safeTransfer( _to, _amount);
    _emitETHBridgeFinalized(_from, _to, _amount, _extraData);
}
```

Client Response

thereksfour: Fixed. <https://github.com/mantlenetworkio/mantle-v2/pull/110>

MNT-2:Potential reentrancy attack in relayMessage ()

Category	Severity	Client Response	Contributor
Logical	Critical	Fixed	Hacker007

Code Reference

- code/mantle-v2/packages/contracts-bedrock/contracts/universal/CrossDomainMessenger.sol#L319-L431
- code/mantle-v2/packages/contracts-bedrock/contracts/universal/CrossDomainMessenger.sol#L495-L497

```
....  
  
319
```

```

: function relayMessage(
320:     uint256 _nonce,
321:     address _sender,
322:     address _target,
323:     uint256 _mntValue,
324:     uint256 _value,
325:     uint256 _minGasLimit,
326:     bytes calldata _message
327: ) external payable virtual {
328:     (, uint16 version) = Encoding.decodeVersionedNonce(_nonce);
329:     require(
330:         version < 2,
331:         "CrossDomainMessenger: only version 0 or 1 messages are supported at this time"332:
    );
333:
334:     // If the message is version 0, then it's a migrated legacy withdrawal. We therefore ne
ed335:     // to check that the legacy version of the message has not already been relayed.336:
if (version == 0) {
337:     bytes32 oldHash = Hashing.hashCrossDomainMessageV0(_target, _sender, _message, _non
ce);
338:     require(
339:         successfulMessages[oldHash] == false,
340:         "CrossDomainMessenger: legacy withdrawal already relayed"341:
    );
342: }
343:
344:     // We use the v1 message hash as the unique identifier for the message because it commi
ts345:     // to the value and minimum gas limit of the message.346:     bytes32 versionedHa
sh = Hashing.hashCrossDomainMessageV1(
347:         _nonce,
348:         _sender,
349:         _target,
350:         _mntValue,
351:         _value,
352:         _minGasLimit,
353:         _message
354:     );
355:
356:     if (_isOtherMessenger()) {
357:         // These properties should always hold when the message is first submitted (as358:
// opposed to being replayed).359:         assert(msg.value == _value);
360:         assert(!failedMessages[versionedHash]);
361:     } else {
362:         require(
363:             msg.value == 0,
364:             "CrossDomainMessenger: value must be zero unless message is from a system addre
ss"365:
        );
366:
367:         require(
368:             failedMessages[versionedHash],
369:             "CrossDomainMessenger: message cannot be replayed"370:
        );
371:     }
372:
373:     require(
374:         _isUnsafeTarget(_target) == false,
375:         "CrossDomainMessenger: cannot send message to blocked system address"376:
    );
377:
378:     require(
379:         successfulMessages[versionedHash] == false,
380:         "CrossDomainMessenger: message has already been relayed"381:
    );
382:

```

We are asserting that we have enough gas to:

```

386: // 1. Call the target contract (_minGasLimit + RELAY_CALL_OVERHEAD + RELAY_GAS_CHECK_BUFFER)
387: // 1.a. The RELAY_CALL_OVERHEAD is included in `hasMinGas`.
388: // 2. Finish the execution after the external call (RELAY_RESERVED_GAS).
389: // 390: // If `xDomainMsgSender` is not the default L2 sender, this function
391: // is being re-entered. This marks the message as failed to allow it to be replayed.
392: if (
393:     !SafeCall.hasMinGas(_minGasLimit, RELAY_RESERVED_GAS + RELAY_GAS_CHECK_BUFFER) ||
394:     xDomainMsgSender != Constants.DEFAULT_L2_SENDER
395: ) {
396:     failedMessages[versionedHash] = true;
397:     emit FailedRelayedMessage(versionedHash);
398:
399:     // Revert in this case if the transaction was triggered by the estimation address.
400:     // should only be possible during gas estimation or we have bigger problems. Reverting
401:     // here will make the behavior of gas estimation change such that the gas limit
402:     // computed will be the amount required to relay the message, even if that amount
403:     // is greater than the minimum gas limit specified by the user.
404:     if (tx.origin == Constants.ESTIMATION_ADDRESS) {
405:         revert("CrossDomainMessenger: failed to relay message");
406:     }
407:
408:     return;
409: }
410:
411: xDomainMsgSender = _sender;
412: bool success = SafeCall.call(_target, gasleft() - RELAY_RESERVED_GAS, _value, _message);
413: xDomainMsgSender = Constants.DEFAULT_L2_SENDER;
414:
415: if (success) {
416:     successfulMessages[versionedHash] = true;
417:     emit RelayedMessage(versionedHash);
418: } else {
419:     failedMessages[versionedHash] = true;
420:     emit FailedRelayedMessage(versionedHash);
421:
422:     // Revert in this case if the transaction was triggered by the estimation address.
423:     // should only be possible during gas estimation or we have bigger problems. Reverting
424:     // here will make the behavior of gas estimation change such that the gas limit
425:     // computed will be the amount required to relay the message, even if that amount
426:     // is greater than the minimum gas limit specified by the user.
427:     if (tx.origin == Constants.ESTIMATION_ADDRESS) {
428:         revert("CrossDomainMessenger: failed to relay message");
429:     }
430: }
431:
....

495: function __CrossDomainMessenger_init() internal onlyInitializing {
496:     xDomainMsgSender = Constants.DEFAULT_L2_SENDER;
497: }

```

Description

Hacker007: There is a potential reentrancy issue when upgrading the contracts deployed on L1. Consider this case:

1. The mantle team launches an upgrade for contracts deployed on L1.

2. The attack spots the signed upgrade transaction when monitoring the TX pool and front-runs it by a withdrawal transaction, including the signed upgrade transaction.
3. The attack deploys an attack contract like below.

```
contract
```

```

AttackContract{
    bool public donotRevert;
    L1CrossDomainMessenger public l1CrossDomainMessenger;
    uint256 constant ethValue = 50;
    uint256 constant mntValue = 50;
    bytes constant selector = abi.encodeWithSelector(this.reinitAndReenter.selector);
    address sender;
    bytes32 hash;
    address target;

    constructor(L1CrossDomainMessenger _l1CrossDomainMessenger ) {
        l1CrossDomainMessenger = _l1CrossDomainMessenger;
        target = address(this);
        sender = Predeploys.L2_CROSS_DOMAIN_MESSENGER;
        hash = Hashing.hashCrossDomainMessage(
            Encoding.encodeVersionedNonce({ _nonce: 0, _version: 1 }), sender, target, mntValue, e
thValue, 0, selector
        );
    }

    function enableRevert() public {
        donotRevert = false;
    }

    function reinitAndReenter() public {
        if (!donotRevert) {
            revert();
        } else {
            // call the initializer function with signed transaction, simplified here
            l1CrossDomainMessenger.initialize();

            // attempt to re-replay the withdrawal
            l1CrossDomainMessenger.relayMessage(
                Encoding.encodeVersionedNonce({ _nonce: 0, _version: 1 }), // nonce
                sender,
                target,
                mntValue,
                ethValue,
                0,
                selector
            );
        }
    }
}

```

4. The attack relays the withdrawal transaction message which will call the function ``AttackContract#reinitAndReenter()``.

```

l1CrossDomainMessenger.relayMessage(
    Encoding.encodeVersionedNonce({ _nonce: 0, _version: 1 }), // nonce
    sender,
    target,
    mntValue,
    ethValue,
    0,
    selector
);

```

since `donotRevert` is false, this message will fail and `failedMessages[versionedHash]` is set to false. 5. The attack sets `donotRevert` to true and relays the withdrawal transaction message again, this time the function `reinitAndReenter()` will call their contract which would run the upgrade transaction first and then re-enter `relayMessage()` with the same withdrawal message. 6. The reentrancy will succeed since `xDomainMsgSender` has been reset by the upgrade transaction. 7. Both the first `relayMessage()` and re-enter `relayMessage()` will succeed, which will let the user get more MTN tokens and native tokens.

References:

1. <https://github.com/ethereum-optimism/optimism/pull/8864>

Recommendation

Hacker007: 1. Only set `xDomainMsgSender` to `Constants.DEFAULT_L2_SENDER` if it is a zero value.

`packages/contracts-bedrock/contracts/universal/CrossDomainMessenger.sol`

```

function __CrossDomainMessenger_init(CrossDomainMessenger _otherMessenger) internal onlyInitializing {
    if (xDomainMsgSender == address(0)) {
        xDomainMsgSender = Constants.DEFAULT_L2_SENDER;
    }
}

```

`packages/contracts-bedrock/contracts/L1/OptimismPortal.sol`

```

function initialize(bool _paused) public initializer {
    if (l2Sender == address(0)) {
        l2Sender = Constants.DEFAULT_L2_SENDER;
    }
    paused = _paused;
    __ResourceMetering_init();
}

```

2. Ensure `successfulMessages[versionedHash]` is still false after the external call.

```
//...
```

```
xDomainMsgSender = _sender;
    bool success = SafeCall.call(_target, gasleft() - RELAY_RESERVED_GAS, _ethValue, _message);
xDomainMsgSender = Constants.DEFAULT_L2_SENDER;
if (_mntValue!=0){
    IERC20(L1_MNT_ADDRESS).approve(_target, 0);
}
if (success) {
    assert(successfulMessages[versionedHash] ==false);
    successfulMessages[versionedHash] =true;
    emit RelayedMessage(versionedHash);
//...
```

Client Response

Hacker007: Fixed. <https://github.com/mantlenetworkio/mantle-v2/pull/109>

MNT-3: Potential lock of ERC721 assets in L1ERC721Bridge

Category	Severity	Client Response	Contributor
Logical	Critical	Fixed	biakia, Yaodao

Code Reference

- `code/mantle-v2/packages/contracts-bedrock/contracts/L1/L1ERC721Bridge.sol#L46-L72`
- `code/mantle-v2/packages/contracts-bedrock/contracts/L1/L1ERC721Bridge.sol#L46-72`
- `code/mantle-v2/packages/contracts-bedrock/contracts/L1/L1ERC721Bridge.sol#L77-L106`
- `code/mantle-v2/packages/contracts-bedrock/contracts/L1/L1ERC721Bridge.sol#L77-106`

....

46

```

: function finalizeBridgeERC721(
47:     address _localToken,
48:     address _remoteToken,
49:     address _from,
50:     address _to,
51:     uint256 _tokenId,
52:     bytes calldata _extraData
53: ) external onlyOtherBridge {
54:     require(_localToken !=address(this), "L1ERC721Bridge: local token cannot be self");
55:
56:     // Checks that the L1/L2 NFT pair has a token ID that is escrowed in the L1 Bridge.57:
require(
58:         deposits[_localToken][_remoteToken][_tokenId] ==true,
59:         "L1ERC721Bridge: Token ID is not escrowed in the L1 Bridge"60:         );
61:
62:     // Mark that the token ID for this L1/L2 token pair is no longer escrowed in the L163:
// Bridge.64:         deposits[_localToken][_remoteToken][_tokenId] =false;
65:
66:     // When a withdrawal is finalized on L1, the L1 Bridge transfers the NFT to the67:
// withdrawer.68:         IERC721(_localToken).safeTransferFrom(address(this), _to, _tokenId);
69:
70:     // slither-disable-next-line reentrancy-events71:         emit ERC721BridgeFinalized(_lo
calToken, _remoteToken, _from, _to, _tokenId, _extraData);
72: }

....

46: function finalizeBridgeERC721(
47:     address _localToken,
48:     address _remoteToken,
49:     address _from,
50:     address _to,
51:     uint256 _tokenId,
52:     bytes calldata _extraData
53: ) external onlyOtherBridge {
54:     require(_localToken !=address(this), "L1ERC721Bridge: local token cannot be self");
55:
56:     // Checks that the L1/L2 NFT pair has a token ID that is escrowed in the L1 Bridge.57:
require(
58:         deposits[_localToken][_remoteToken][_tokenId] ==true,
59:         "L1ERC721Bridge: Token ID is not escrowed in the L1 Bridge"60:         );
61:
62:     // Mark that the token ID for this L1/L2 token pair is no longer escrowed in the L163:
// Bridge.64:         deposits[_localToken][_remoteToken][_tokenId] =false;
65:
66:     // When a withdrawal is finalized on L1, the L1 Bridge transfers the NFT to the67:
// withdrawer.68:         IERC721(_localToken).safeTransferFrom(address(this), _to, _tokenId);
69:
70:     // slither-disable-next-line reentrancy-events71:         emit ERC721BridgeFinalized(_lo
calToken, _remoteToken, _from, _to, _tokenId, _extraData);
72: }

....

77: function _initiateBridgeERC721(
78:     address _localToken,
79:     address _remoteToken,
80:     address _from,
81:     address _to,
82:     uint256 _tokenId,
83:     uint32 _minGasLimit,

```

```

86:         require(_remoteToken !=address(0), "L1ERC721Bridge: remote token cannot be address(0)");
87:
88:         // Construct calldata for _l2Token.finalizeBridgeERC721(_to, _tokenId)89:             bytesm
emory message =abi.encodeWithSelector(
90:             L2ERC721Bridge.finalizeBridgeERC721.selector,
91:             _remoteToken,
92:             _localToken,
93:             _from,
94:             _to,
95:             _tokenId,
96:             _extraData
97:         );
98:
99:         // Lock token into bridge100:             deposits[_localToken][_remoteToken][_tokenId] =tru
e;
101:         IERC721(_localToken).transferFrom(_from, address(this), _tokenId);
102:
103:         // Send calldata into L2104:             MESSENGER.sendMessage(0, OTHER_BRIDGE, message, _m
inGasLimit);
105:         emit ERC721BridgeInitiated(_localToken, _remoteToken, _from, _to, _tokenId, _extraDat
a);
106:     }

....

77: function _initiateBridgeERC721(
78:     address _localToken,
79:     address _remoteToken,
80:     address _from,
81:     address _to,
82:     uint256 _tokenId,
83:     uint32 _minGasLimit,
84:     bytes calldata _extraData
85: ) internal override {
86:     require(_remoteToken !=address(0), "L1ERC721Bridge: remote token cannot be address(0)");
87:
88:     // Construct calldata for _l2Token.finalizeBridgeERC721(_to, _tokenId)89:             bytesm
emory message =abi.encodeWithSelector(
90:             L2ERC721Bridge.finalizeBridgeERC721.selector,
91:             _remoteToken,
92:             _localToken,
93:             _from,
94:             _to,
95:             _tokenId,
96:             _extraData
97:         );
98:
99:         // Lock token into bridge100:             deposits[_localToken][_remoteToken][_tokenId] =tru
e;
101:         IERC721(_localToken).transferFrom(_from, address(this), _tokenId);
102:
103:         // Send calldata into L2104:             MESSENGER.sendMessage(0, OTHER_BRIDGE, message, _m
inGasLimit);
105:         emit ERC721BridgeInitiated(_localToken, _remoteToken, _from, _to, _tokenId, _extraDat
a);
106:     }

```

- [code/mantle-v2/packages/contracts-bedrock/contracts/L2/L2ERC721Bridge.sol#L46-74](#)

```

: function finalizeBridgeERC721(
47:     address _localToken,
48:     address _remoteToken,
49:     address _from,
50:     address _to,
51:     uint256 _tokenId,
52:     bytes calldata _extraData
53: ) external onlyOtherBridge {
54:     require(_localToken !=address(this), "L2ERC721Bridge: local token cannot be self");
55:
56:     // Note that supportsInterface makes a callback to the _localToken address which is user
57:     // provided.58:         require(
59:         ERC165Checker.supportsInterface(_localToken, type(IOptimismMintableERC721).interface
Id),
60:         "L2ERC721Bridge: local token interface is not compliant"61:         );
62:
63:     require(
64:         _remoteToken == IOptimismMintableERC721(_localToken).remoteToken(),
65:         "L2ERC721Bridge: wrong remote token for Optimism Mintable ERC721 local token"66:
);
67:
68:     // When a deposit is finalized, we give the NFT with the same tokenId to the account69:
// on L2. Note that safeMint makes a callback to the _to address which is user provided.70:
IOptimismMintableERC721(_localToken).safeMint(_to, _tokenId);
71:
72:     // slither-disable-next-line reentrancy-events73:         emit ERC721BridgeFinalized(_lo
calToken, _remoteToken, _from, _to, _tokenId, _extraData);
74: }

```

Description

biakia: In contract `L1ERC721Bridge``, the function `_initiateBridgeERC721`` will be called when a user wants to bridge their NFT from ethereum to the mantle network. It will lock the user's NFT in the contract `L1ERC721Bridge``:

```

// Lock token into bridge
deposits[_localToken][_remoteToken][_tokenId] = true;
IERC721(_localToken).transferFrom(_from, address(this), _tokenId);

```

When the user wants to bridge their NFT back to the ethereum, the function `finalizeBridgeERC721`` will be called finally. It will send the NFT back to the user:

```

// Mark that the token ID for this L1/L2 token pair is no longer escrowed in the L1
// Bridge.
deposits[_localToken][_remoteToken][_tokenId] = false;

// When a withdrawal is finalized on L1, the L1 Bridge transfers the NFT to the
// withdrawer.
IERC721(_localToken).safeTransferFrom(address(this), _to, _tokenId);

```

Here we can see that the function `safeTransferFrom`` will be called. The issue here is that some kind of NTF collection does not implement the function `safeTransferFrom``, for example, the NFT collection `CryptoKitties``. You can see the code here: <https://etherscan.io/token/0x06012c8cf97bead5deae237070f9587f8e7a266d#code>

```

/**
*Submitted for verification at Etherscan.io on 2017-11-28

```

```

solidity ^0.4.11;

/**
 * @title Ownable
 * @dev The Ownable contract has an owner address, and provides basic authorization control
 * functions, this simplifies the implementation of "user permissions".
 */contract Ownable {
    addresspublic owner;

    /**
     * @dev The Ownable constructor sets the original `owner` of the contract to the sender
     * account.
     */function Ownable() {
        owner =msg.sender;
    }

    /**
     * @dev Throws if called by any account other than the owner.
     */modifier onlyOwner() {
        require(msg.sender== owner);
        _;
    }

    /**
     * @dev Allows the current owner to transfer control of the contract to a newOwner.
     * @param newOwner The address to transfer ownership to.
     */function transferOwnership(address newOwner) onlyOwner {
        if (newOwner !=address(0)) {
            owner = newOwner;
        }
    }
}

/// @title Interface for contracts conforming to ERC-721: Non-Fungible Tokens/// @author Dieter Sh
irley <dete@axiomzen.co> (https://github.com/dete)contract ERC721 {
    // Required methodsfunction totalSupply() public view returns (uint256 total);
    function balanceOf(address _owner) public view returns (uint256 balance);
    function ownerOf(uint256 _tokenId) external view returns (address owner);
    function approve(address _to, uint256 _tokenId) external;

```

```
function transferFrom(address _from, address _to, uint256 _tokenId) external;

// Event
event Transfer(address from, address to, uint256 tokenId);
event Approval(address owner, address approved, uint256 tokenId);

// Optional
// function name() public view returns (string name);
// function symbol() public view returns (string symbol);
// function tokensOfOwner(address _owner) external view returns (uint256[] tokenIds);
// function tokenMetadata(uint256 _tokenId, string _preferredTransport) public view returns (string infoUrl);
// ERC-165 Compatibility (https://github.com/ethereum/EIPs/issues/165)
function supportsInterface(bytes4 _interfaceID) external view returns (bool);
}

...
....
```

Since the `L1ERC721Bridge` is a common ERC721 bridge for all NFT collection in ethereum, it is possible that the user bridges their `CryptoKitties` to the mantle network. After bridging NFT `CryptoKitties` to the mantle, it will be locked in the `L1ERC721Bridge`, and when the user wants to bridge back their `CryptoKitties`, he will fail to do so due to the failure of calling the function `safeTransferFrom`. There is no function to withdraw NFT from the `L1ERC721Bridge`, as a result, the user's `CryptoKitties` will be locked in the contract forever.

Yaodao: The function `_initiateBridgeERC721()` is used to receive the ERC721 assets from the user on the Ethereum network, lock the ERC721 assets in the contract and record the deposit information in the variable `deposits`. The user will get the corresponding ERC721 assets minted by the function `finalizeBridgeERC721` in contract `L2ERC721Bridge`.

```
require(
    _remoteToken == IOptimismMintableERC721(_localToken).remoteToken(),
    "L2ERC721Bridge: wrong remote token for Optimism Mintable ERC721 local token"
);

// When a deposit is finalized, we give the NFT with the same tokenId to the account
// on L2. Note that safeMint makes a callback to the _to address which is user provided.
IOptimismMintableERC721(_localToken).safeMint(_to, _tokenId);
```

For the users to get back their ERC721 asset from Mantle to Ethereum, the function `finalizeBridgeERC721()` is used to complete an ERC721 bridge from the other domain and send the ERC721 token to the recipient on this domain.

However, the function `finalizeBridgeERC721()` uses the function `IERC721(_localToken).safeTransferFrom()` to transfer the ERC721 asset locked in the contract, which is not implemented in some ERC721 contracts.

```
// Mark that the token ID for this L1/L2 token pair is no longer escrowed in the L1
// Bridge.
deposits[_localToken][_remoteToken][_tokenId] = false;

// When a withdrawal is finalized on L1, the L1 Bridge transfers the NFT to the
// withdrawer.
IERC721(_localToken).safeTransferFrom(address(this), _to, _tokenId);
```

``IERC721(_localToken).safeTransferFrom()`` will fail and the ERC721 asset will be locked in the contract forever.

Recommendation

biakia: Consider using ``transferFrom`` instead if the NFT collection does not support ``safeTransferFrom``.

Yaodao: Recommend adding the whitelist to manage the ERC721 assets to avoid this.

Client Response

biakia: Fixed. Fixed in <https://github.com/mantlenetworkio/mantle-v2/pull/103>

Yaodao: Fixed. Fixed in <https://github.com/mantlenetworkio/mantle-v2/pull/103>

mantle-v1 does not support bridging NFT. mantle-v2 uses `safeTransferFrom` and does not support older version NFT bridging by design.

MNT-4:NFTs can be stuck in the protocol many NFTs have pause functionality

Category	Severity	Client Response	Contributor
Logical	Critical	Fixed	rajatbeladiya

Code Reference

- code/mantle-v2/packages/contracts-bedrock/contracts/L1/OptimismPortal.sol#L331-L434

```
....  
  
331
```

```

: /**
332:  * @notice Finalizes a withdrawal transaction.
333:  *
334:  * @param _tx Withdrawal transaction to finalize.
335:  */336:  function finalizeWithdrawalTransaction(Types.WithdrawalTransaction memory _tx)
337:  external
338:  whenNotPaused
339:  {
340:      // Make sure that the l2Sender has not yet been set. The l2Sender is set to a value oth
er341:      // than the default value when a withdrawal transaction is being finalized. This chec
k is342:      // a defacto reentrancy guard.343:      require(
344:          l2Sender == Constants.DEFAULT_L2_SENDER,
345:          "OptimismPortal: can only trigger one withdrawal per transaction"346:      );
347:
348:      // Grab the proven withdrawal from the `provenWithdrawals` map.349:      bytes32 wit
hdrawalHash = Hashing.hashWithdrawal(_tx);
350:      ProvenWithdrawal memory provenWithdrawal = provenWithdrawals[withdrawalHash];
351:
352:      // A withdrawal can only be finalized if it has been proven. We know that a withdrawal
has353:      // been proven at least once when its timestamp is non-zero. Unproven withdrawals wi
ll have354:      // a timestamp of zero.355:      require(
356:          provenWithdrawal.timestamp !=0,
357:          "OptimismPortal: withdrawal has not been proven yet"358:      );
359:
360:      // As a sanity check, we make sure that the proven withdrawal's timestamp is greater th
an361:      // starting timestamp inside the L2OutputOracle. Not strictly necessary but extra lay
er of362:      // safety against weird bugs in the proving step.363:      require(
364:          provenWithdrawal.timestamp >= L2_ORACLE.startingTimestamp(),
365:          "OptimismPortal: withdrawal timestamp less than L2 Oracle starting timestamp"366:
      );
367:
368:      // A proven withdrawal must wait at least the finalization period before it can be369:
// finalized. This waiting period can elapse in parallel with the waiting period for the370:
// output the withdrawal was proven against. In effect, this means that the minimum371:      // w
ithdrawal time is proposal submission time + finalization period.372:      require(
373:          _isFinalizationPeriodElapsed(provenWithdrawal.timestamp),
374:          "OptimismPortal: proven withdrawal finalization period has not elapsed"375:
      );
376:
377:      // Grab the OutputProposal from the L2OutputOracle, will revert if the output that378:
// corresponds to the given index has not been proposed yet.379:      Types.OutputProposal memory
proposal = L2_ORACLE.getL2Output(
380:          provenWithdrawal.l2OutputIndex
381:      );
382:
383:      // Check that the output root that was used to prove the withdrawal is the same as the3
84:      // current output root for the given output index. An output root may change if it is38
5:      // deleted by the challenger address and then re-proposed.386:      require(
387:          proposal.outputRoot == provenWithdrawal.outputRoot,
388:          "OptimismPortal: output root proven is not the same as current output root"389:
      );
390:
391:      // Check that the output proposal has also been finalized.392:      require(
393:          _isFinalizationPeriodElapsed(proposal.timestamp),
394:          "OptimismPortal: output proposal finalization period has not elapsed"395:
      );
396:
397:      // Check that this withdrawal has not already been finalized, this is replay protectio
n.398:      require(
399:          finalizedWithdrawals[withdrawalHash] ==false,
400:          "OptimismPortal: withdrawal has already been finalized"401:      );
402:

```

```

rawals[withdrawalHash] =true;
405:
406:         // Set the l2Sender so contracts know who triggered this withdrawal on L2.407:
l2Sender = _tx.sender;
408:
409:         // Trigger the call to the target contract. We use a custom low level method410:
// SafeCall.callWithMinGas to ensure two key properties411:         // 1. Target contracts cannot
force this call to run out of gas by returning a very large412:         // amount of data (and
this is OK because we don't care about the returndata here).413:         // 2. The amount of gas p
rovided to the execution context of the target is at least the414:         // gas limit specifi
ed by the user. If there is not enough gas in the current context415:         // to accomplish
this, `callWithMinGas` will revert.416:         bool l1mntSuccess =false;
417:         if (_tx.mntValue>0){
418:             l1mntSuccess = IERC20(L1_MNT_ADDRESS).transfer(_tx.target, _tx.mntValue);
419:         }
420:         bool success = SafeCall.callWithMinGas(_tx.target, _tx.gasLimit, _tx.ethValue, _tx.dat
a);
421:         // Reset the l2Sender back to the default value.422:         l2Sender = Constants.DEFAU
LT_L2_SENDER;
423:
424:         // All withdrawals are immediately finalized. Replayability can425:         // be achie
ved through contracts built on top of this contract426:         emit WithdrawalFinalized(withdrawalH
ash, success && l1mntSuccess);
427:
428:         // Reverting here is useful for determining the exact gas cost to successfully execute
the429:         // sub call to the target contract if the minimum gas limit specified by the user wo
uld not430:         // be sufficient to execute the sub call.431:         if (success && l1mntSucces
s ==false&&tx.origin== Constants.ESTIMATION_ADDRESS) {
432:             revert("OptimismPortal: withdrawal failed");
433:         }
434:     }

```

Description

rajatbeladiya: here Mantle V2 enables users to transfer NFTs from L2 to L1 by bridging them to the recipient's account.

However, many NFTs such as `CryptoKitties` and `CryptoFighters` that have `pause` functionality. Attacker can use this functionality to prevent NFTs from bridging and leading to stuck the NFTs in the protocol.

Scenario:

1. Alice uses CryptoKitties NFTs to bridge from L2 to L1
2. Alice initiates process (verified transaction, waiting for the challenge period to conclude)
3. CryptoKitties paused their contracts
4. Malicious attacker can exploit `finalizeWithdrawalTransaction()` function using alice's transaction information and leads to cause the Alice's function call to be unsuccessful (it will mark transaction as completed because error is not handled properly for failed calls) .
5. Alice's NFT will stuck in the protocol

```

require(
    finalizedWithdrawals[withdrawalHash] == false,
    "OptimismPortal: withdrawal has already been finalized"
);

```

MNT-5: Malicious actor can steal deposited MNT/BMV_ETH token

Category	Severity	Client Response	Contributor
Logical	Critical	Fixed	SerSomeone, lemonmon, csanuragjain, newway55

Code Reference

- code/mantle-v2/packages/contracts-bedrock/contracts/L1/L1CrossDomainMessenger.sol#L213
- code/mantle-v2/packages/contracts-bedrock/contracts/L1/L1CrossDomainMessenger.sol#L213
- code/mantle-v2/packages/contracts-bedrock/contracts/L1/L1CrossDomainMessenger.sol#L217
- code/mantle-v2/packages/contracts-bedrock/contracts/L1/L1CrossDomainMessenger.sol#L217

```
....  
213: if (_mntValue!=0){  
....  
213: if (_mntValue!=0){  
....  
217: bool success = SafeCall.call(_target, gasleft() - RELAY_RESERVED_GAS, _ethValue, _message);  
....  
217: bool success = SafeCall.call(_target, gasleft() - RELAY_RESERVED_GAS, _ethValue, _message);
```

- code/mantle-v2/packages/contracts-bedrock/contracts/L1/OptimismPortal.sol#L253
- code/mantle-v2/packages/contracts-bedrock/contracts/L1/OptimismPortal.sol#L265
- code/mantle-v2/packages/contracts-bedrock/contracts/L1/OptimismPortal.sol#L418
- code/mantle-v2/packages/contracts-bedrock/contracts/L1/OptimismPortal.sol#L420
- code/mantle-v2/packages/contracts-bedrock/contracts/L1/OptimismPortal.sol#L420

```
....  
253
```

```

: function proveWithdrawalTransaction(
    ....
265: );
    ....
418: l1mntSuccess = IERC20(L1_MNT_ADDRESS).transfer(_tx.target, _tx.mntValue);
    ....
420: bool success = SafeCall.callWithMinGas(_tx.target, _tx.gasLimit, _tx.ethValue, _tx.data);
    ....
420: bool success = SafeCall.callWithMinGas(_tx.target, _tx.gasLimit, _tx.ethValue, _tx.data);

```

- [code/mantle-v2/packages/contracts-bedrock/contracts/L2/L2CrossDomainMessenger.sol#L255](#)

```

....
255: return _target == address(this) || _target == address(Predeploys.L2_TO_L1_MESSAGE_PASSER);

```

Description

SerSomeone: All MNT that users are depositing from L1 to L2 it gets locked in `OptimismPortal`` through `depositTransaction``

```

function depositTransaction(
    uint256 _mntValue,
    address _to,
    uint256 _mntTxValue,
    uint64 _gasLimit,
    bool _isCreation,
    bytes memory _data
) public payable metered(_gasLimit) {
    -----if (_mntValue !=0) {
        IERC20(L1_MNT_ADDRESS).safeTransferFrom(msg.sender, address(this), _mntValue);
    }
    -----bytes memory opaqueData =abi.encodePacked(
        _mntValue,
        _mntTxValue,
        msg.value,
        _gasLimit,
        _isCreation,
        _data
    );
    -----emit TransactionDeposited(from, _to, DEPOSIT_VERSION, opaqueData);
}

```

A malicious actor can create a withdraw transaction from L2 and set:

1. ``_target`` to ``L1_MNT_ADDRESS``
2. ``_data`` to ``abi.encodeWithSignature("approve(address,uint256)", <HACKER ADDRESS>, type(uint256).max)``

```
function initiateWithdrawal(
    uint256 _ethValue,
    address _target,
    uint256 _gasLimit,
    bytes memory _data
) public payable {
    -----
    sentMessages[withdrawalHash] = true;

    emit MessagePassed(
        messageNonce(),
        msg.sender,
        _target,
        msg.value,
        _ethValue,
        _gasLimit,
        _data,
        withdrawalHash
    );
    -----
}
```

The hacker can then prove and finalize the withdrawal which will make `OptimismPortal` call `MNT`'s `approve` function to give the hacker full allowance to spend `MNT` on behalf of `OptimismPortal`

```

function proveWithdrawalTransaction(
    Types.WithdrawalTransaction memory _tx,
    uint256 _l2OutputIndex,
    Types.OutputRootProof calldata _outputRootProof,
    bytes[] calldata _withdrawalProof
) external whenNotPaused{
    -----require(
        _tx.target != address(this),
        "OptimismPortal: you cannot send messages to the portal contract"
    );
    -----

    provenWithdrawals[withdrawalHash] = ProvenWithdrawal({
        outputRoot: outputRoot,
        timestamp: uint128(block.timestamp),
        l2OutputIndex: uint128(_l2OutputIndex)
    });
    -----
}

function finalizeWithdrawalTransaction(Types.WithdrawalTransaction memory _tx)
    external
    whenNotPaused
{
    -----bool success = SafeCall.callWithMinGas(_tx.target, _tx.gasLimit, _tx.ethValue, _tx.data);
    -----
}

```

Once executed - the hacker will have infinite allowance and drain all user deposits by calling `MNT.transferFrom(0ptimimPortal, <ALL BALANCE>)`. All user deposits will be gone and the protocol will be completely insolvent.

SerSomeone: The `L1CrossDomainMessenger` holds `MNT` when messages fail to execute and before the funds are "pulled" by the target.

A malicious actor can send a message from L2 to call `approve` on the L1 `MNT` token with `type(uint256).max`.

This call will originate from `L1CrossDomainMessenger`'s `relayMessage` function and will give the hacker full approval to spend `L1CrossDomainMessenger`'s `MNT` tokens.

A message can look like this:

```

L2Messenger.sendMessage(0, <MNT_ADDR>, abi.encodeWithSignature("approve(address,uint256)", <HACKER ADDRESS>, type(uint256).max), 200_000);

```

This will arrive at `L1CrossDomainMessenger` and call `MNT` approve function:

```
function relayMessage(
    uint256 _nonce,
    address _sender,
    address _target,
    uint256 _mntValue,
    uint256 _ethValue,
    uint256 _minGasLimit,
    bytes calldata _message
) external payable override {
    -----require(
        _isUnsafeTarget(_target) == false,
        "CrossDomainMessenger: cannot send message to blocked system address"
    );
    -----bool success = SafeCall.call(_target, gasleft() - RELAY_RESERVED_GAS, _ethValue, _message);
    -----
}
```

This is possible because `\_isUnsafeTarget` does not prevent calls to `MNT`.

```
function _isUnsafeTarget(address _target) internal view override returns (bool) {
    return _target == address(this) || _target == address(PORTAL);
}
```

The hacker can steal funds from failed message - the hacker will receive the funds and also make the message not replayable.

SerSomeone: The `L2CrossDomainMessenger` can hold `BMV\_ETH` funds that are pending to be relayed. This is because failed transactions can be replayed with more gas. An issue rises where a malicious actor can see that `L2CrossDomainMessenger` has a positive balance and then send a message from `L1CrossDomainMessenger` to transfer the funds. The call to `L1CrossDomainMessenger` will look like this:

```
L1Messenger.sendMessage(0, Predeploys.BVM_ETH, abi.encodeWithSignature("transfer(address,uint256)", address(this), <BALANCE>), 200_000);
```

Essentially the above message will make the `relayMessage` function on the `L2CrossDomainMessenger` call `BVM\_ETH`'s `transfer` function to transfer the remaining balance to the hackers account. Additionally, instead of direct `transfer` the malicious actor can call `approve` with the maximum approval to his L2 address - the hacker would be able to `transferFrom` directly whenever there is a positive balance.

```
function relayMessage(
    uint256 _nonce,
    address _sender,
    address _target,
    uint256 _mntValue,
    uint256 _ethValue,
    uint256 _minGasLimit,
    bytes calldata _message
) external payable override {
    -----require(
        _isUnsafeTarget(_target) == false,
        "CrossDomainMessenger: cannot send message to blocked system address"
    );
    -----bool success = SafeCall.call(_target, gasleft() - RELAY_RESERVED_GAS, _mntValue, _message);
    -----
}
```

lemonmon: When users are bridging MNT token from L1 to L2, the MNT token will be held by `OptimismPortal`. When MNT token is bridged back from L2 to L1, the `OptimismPortal` will call on the L1 mnt token to transfer the amount to the user back.

https://github.com/Secure3Audit/code_Mantle_V2/blob/97bb2aad421c306241ca1681dd3c68b6dfc3355b/code/mantle-v2/packages/contracts-bedrock/contracts/L1/OptimismPortal.sol#L418

However, there is no protection against an attacker to specify the L1 mnt token as the target from L2. It means that the attacker can transfer out the mnt held by the `OptimismPortal` freely by calling directly the L1 mnt token. The attack scenario would be like

1. the attacker is using `L2ToL1Passer` and specifying `L1_MNT_TOKEN` as target. and the `_data` as the call data to `transfer(attacker_address, amount)`.
2. the withdrawal transaction will be eventually included in the Oracle
3. the attacker proves the transaction and finalizes it, and gets the mnt token.

csanuragjain: L1 CrossDomainMessenger can hold MNT balance from failed transactions which are yet to be replayed. Similarly ETH balance could be present on L2 CrossDomainMessenger. Both can be stolen by using target as `L1_MNT_ADDRESS` (on L1) and `Predeploys.BVM_ETH` (on L2). Since CrossDomainMessenger on both sides does not have this in unsafe target so balance can be stolen

Note

1. This also needs to be fixed in OptimismPortal where target cannot be L1_MNT_ADDRESS while proving or finalizing transaction
2. POC discuss exploit on L1CrossDomainMessenger but the same trick can be used to exploit on L2 side

POC

1. On L2 call sendMessage on L2CrossDomainMessenger

```
_sendMessage(
    _ethAmount,
    OTHER_MESSENGER,
    baseGas(_message, _minGasLimit),
    abi.encodeWithSelector(
        L1CrossDomainMessenger.relayMessage.selector,
        messageNonce(),
        msg.sender,
        _target,
        msg.value,
        _ethAmount,
        _minGasLimit,
        _message
    )
)
```

2. Lets say below params were passed:

```
_ethAmount=0
msg.sender=Attacker
_target= L1_MNT_ADDRESS
msg.value=0
_message = transfer(Attacker, amount)
```

3. This makes the send message as :

```
_sendMessage(
    0,
    OTHER_MESSENGER,
    baseGas(_message, _minGasLimit),
    abi.encodeWithSelector(
        L1CrossDomainMessenger.relayMessage.selector,
        messageNonce(),
        Attacker,
        L1_MNT_ADDRESS,
        0,
        0,
        _minGasLimit,
        transfer(Attacker, amount)
    )
)
```

4. Now _sendMessage wraps it further as below:

```
payable(Predeploys.L2_TO_L1_MESSAGE_PASSER)).initiateWithdrawal{
    value: 0
}(0, OTHER_MESSENGER, _gasLimit,

    abi.encodeWithSelector(
        L1CrossDomainMessenger.relayMessage.selector,
        messageNonce(),
        Attacker,
        L1_MNT_ADDRESS,
        0,
        0,
        _minGasLimit,
        transfer(Attacker, amount)
    )

);
```

5. Now initiateWithdrawal is called like below:

```
_target= OTHER_MESSENGER so check passes
_data = abi.encodeWithSelector(L1CrossDomainMessenger.relayMessage.selector,messageNonce(),Attacker,L1_MNT_ADDRESS,0,0,_minGasLimit,transfer(Attacker, amount))
```

6. Now this message is sent to L1 where Optimism portal calls the allowed _target which is L1CrossDomainMessenger with this _data

7. So `L1CrossDomainMessenger.relayMessage.selector`` is called with below params

```
function relayMessage(
    uint256 _nonce, // messageNonce()
    address _sender, // Attacker
    address _target, //L1_MNT_ADDRESS
    uint256 _mntValue, //0
    uint256 _ethValue, //0
    uint256 _minGasLimit,
    bytes calldata _message // transfer(Attacker, amount)
)
```

8. Since unsafe target does not checks L1_MNT_ADDRESS so it is accepted

```
function _isUnsafeTarget(address _target) internal view override returns (bool) {
    return _target == address(this) || _target == address(PORTAL);
}
```

9. So finally this is executed

```
success = SafeCall.call(L1_MNT_ADDRESS, gasleft() - RELAY_RESERVED_GAS, 0, transfer(Attacker, amount));
```

10. This means mnt balance at CrossDomainMessenger will be transferred to Attacker

newway55: By design, the portal contract on L1 can receive ETH or MNT [here](#). When **proving** a withdrawal transaction on L1, the Portal will **check** whether the target is the contract itself. When calling `finalizeWithdrawalTransaction`, the portal will perform a call to a target address, which can be the L1_MNT address and the transaction data can be a call to the `transfer` function of the L1_MNT contract. This can lead to draining the L1_MNT balance from the PORTAL.

newway55: By Logic in the following [code](#) inside `L1CrossDomainMessenger` contract, this contract holds at least an amount equal or greater than `_mntValue` MNT tokens. The contract approves the `_target` to make use of them and then sets the approval to 0 after the call. The target can be the `L1_MNT_ADDRESS` and the transaction data can be a call to the `transfer` function of the L1_MNT contract. This can lead to draining the L1_MNT balance from the `L1CrossDomainMessenger`.

Recommendation

SerSomeone: In `OptimismPortal`'s `proveWithdrawalTransaction` consider adding a check that `_tx.target != L1_MNT_ADDRESS`

SerSomeone: Add `MNT` to `_isUnsafeTarget` in `L1CrossDomainSender`

SerSomeone: Consider adding `BVM_ETH` to `_isUnsafeTarget`

lemonmon: Forbid the `OptimismPortal` to call on the l1 mnt token directly.

csanuragjain: Add `L1_MNT_ADDRESS` to unsafe address in L1 CrossDomainMessenger and `Predeploys.BVM_ETH` on L2 CrossDomainMessenger

newway55: - add a check to see if the target is the L1_MNT contract and revert if it is.

```
function proveWithdrawalTransaction()...

    require(
        _tx.target != address(this),
        "OptimismPortal: you cannot send messages to the portal contract"
    );

    require(
        _tx.target != L1_MNT_ADDRESS,
        "OptimismPortal: you cannot send messages to the portal contract"
    );
```

newway55: add `L1_MNT_ADDRESS` to the function `_isUnsafeTarget` and revert if it is.

```
function _isUnsafeTarget(address _target) internal view override returns (bool) {
    return _target == address(this) || _target == address(PORTAL) || _target == L1_MNT_ADDRESS;
}
```

Client Response

SerSomeone: Fixed. <https://github.com/mantlenetworkio/mantle-v2/pull/98>

lemonmon: Fixed. <https://github.com/mantlenetworkio/mantle-v2/pull/98>

Fixed. <https://github.com/mantlenetworkio/mantle-v2/pull/98>

newway55: Fixed. <https://github.com/mantlenetworkio/mantle-v2/pull/98>

MNT-6:L1 -> L2 messages with large calldata that fail will not be replayable

Category	Severity	Client Response	Contributor
Logical	Critical	Acknowledged	SerSomeone

Code Reference

- code/mantle-v2/packages/contracts-bedrock/contracts/L1/L1CrossDomainMessenger.sol#L87

```
....  
  
87: baseGas(_message, _minGasLimit),
```

Description

SerSomeone: Due to insufficient gas calculations on calldata overhead in `CrossDomainMessenger`'s`baseGas`` function - When large L1 -> L2 messages that are sent using the `CrossDomainMessenger`` fail they cannot be replayed and any funds attached will be lost.

Recommendation

SerSomeone: Add to `baseGas`` the cost of hashing relative to the message length.

Client Response

SerSomeone: Acknowledged. will fix later

MNT-7:Incorrect revert check in function `finalizeWithdrawalTransaction()`

Category	Severity	Client Response	Contributor
Logical	Critical	Fixed	lemonmon, newway55, rajatbeladiya, Yaodao, biakia

Code Reference

- code/mantle-v2/packages/contracts-bedrock/contracts/L1/OptimismPortal.sol#L331-L434
- code/mantle-v2/packages/contracts-bedrock/contracts/L1/OptimismPortal.sol#L336
- code/mantle-v2/packages/contracts-bedrock/contracts/L1/OptimismPortal.sol#L409-433
- code/mantle-v2/packages/contracts-bedrock/contracts/L1/OptimismPortal.sol#L428-L433
- code/mantle-v2/packages/contracts-bedrock/contracts/L1/OptimismPortal.sol#L431-L433

....

331

```

: /**
332:  * @notice Finalizes a withdrawal transaction.
333:  *
334:  * @param _tx Withdrawal transaction to finalize.
335:  */336:  function finalizeWithdrawalTransaction(Types.WithdrawalTransaction memory _tx)
337:  external
338:  whenNotPaused
339:  {
340:      // Make sure that the l2Sender has not yet been set. The l2Sender is set to a value oth
er341:      // than the default value when a withdrawal transaction is being finalized. This chec
k is342:      // a defacto reentrancy guard.343:      require(
344:          l2Sender == Constants.DEFAULT_L2_SENDER,
345:          "OptimismPortal: can only trigger one withdrawal per transaction"346:      );
347:
348:      // Grab the proven withdrawal from the `provenWithdrawals` map.349:      bytes32 wit
hdrawalHash = Hashing.hashWithdrawal(_tx);
350:      ProvenWithdrawal memory provenWithdrawal = provenWithdrawals[withdrawalHash];
351:
352:      // A withdrawal can only be finalized if it has been proven. We know that a withdrawal
has353:      // been proven at least once when its timestamp is non-zero. Unproven withdrawals wi
ll have354:      // a timestamp of zero.355:      require(
356:          provenWithdrawal.timestamp !=0,
357:          "OptimismPortal: withdrawal has not been proven yet"358:      );
359:
360:      // As a sanity check, we make sure that the proven withdrawal's timestamp is greater th
an361:      // starting timestamp inside the L2OutputOracle. Not strictly necessary but extra lay
er of362:      // safety against weird bugs in the proving step.363:      require(
364:          provenWithdrawal.timestamp >= L2_ORACLE.startingTimestamp(),
365:          "OptimismPortal: withdrawal timestamp less than L2 Oracle starting timestamp"366:
      );
367:
368:      // A proven withdrawal must wait at least the finalization period before it can be369:
// finalized. This waiting period can elapse in parallel with the waiting period for the370:
// output the withdrawal was proven against. In effect, this means that the minimum371:      // w
ithdrawal time is proposal submission time + finalization period.372:      require(
373:          _isFinalizationPeriodElapsed(provenWithdrawal.timestamp),
374:          "OptimismPortal: proven withdrawal finalization period has not elapsed"375:
      );
376:
377:      // Grab the OutputProposal from the L2OutputOracle, will revert if the output that378:
// corresponds to the given index has not been proposed yet.379:      Types.OutputProposal memory
proposal = L2_ORACLE.getL2Output(
380:          provenWithdrawal.l2OutputIndex
381:      );
382:
383:      // Check that the output root that was used to prove the withdrawal is the same as the3
84:      // current output root for the given output index. An output root may change if it is38
5:      // deleted by the challenger address and then re-proposed.386:      require(
387:          proposal.outputRoot == provenWithdrawal.outputRoot,
388:          "OptimismPortal: output root proven is not the same as current output root"389:
      );
390:
391:      // Check that the output proposal has also been finalized.392:      require(
393:          _isFinalizationPeriodElapsed(proposal.timestamp),
394:          "OptimismPortal: output proposal finalization period has not elapsed"395:
      );
396:
397:      // Check that this withdrawal has not already been finalized, this is replay protectio
n.398:      require(
399:          finalizedWithdrawals[withdrawalHash] ==false,
400:          "OptimismPortal: withdrawal has already been finalized"401:      );
402:

```

```

rawals[withdrawalHash] =true;
405:
406:     // Set the l2Sender so contracts know who triggered this withdrawal on L2.407:
l2Sender = _tx.sender;
408:
409:     // Trigger the call to the target contract. We use a custom low level method410:
// SafeCall.callWithMinGas to ensure two key properties411:         // 1. Target contracts cannot
force this call to run out of gas by returning a very large412:         // amount of data (and
this is OK because we don't care about the returndata here).413:         // 2. The amount of gas p
rovided to the execution context of the target is at least the414:         // gas limit specifi
ed by the user. If there is not enough gas in the current context415:         // to accomplish
this, `callWithMinGas` will revert.416:         bool l1mntSuccess =false;
417:         if (_tx.mntValue>0){
418:             l1mntSuccess = IERC20(L1_MNT_ADDRESS).transfer(_tx.target, _tx.mntValue);
419:         }
420:         bool success = SafeCall.callWithMinGas(_tx.target, _tx.gasLimit, _tx.ethValue, _tx.dat
a);
421:         // Reset the l2Sender back to the default value.422:         l2Sender = Constants.DEFAU
LT_L2_SENDER;
423:
424:         // All withdrawals are immediately finalized. Replayability can425:         // be achie
ved through contracts built on top of this contract426:         emit WithdrawalFinalized(withdrawalH
ash, success && l1mntSuccess);
427:
428:         // Reverting here is useful for determining the exact gas cost to successfully execute
the429:         // sub call to the target contract if the minimum gas limit specified by the user wo
uld not430:         // be sufficient to execute the sub call.431:         if (success && l1mntSucces
s ==false&&tx.origin== Constants.ESTIMATION_ADDRESS) {
432:             revert("OptimismPortal: withdrawal failed");
433:         }
434:     }

....

336: function finalizeWithdrawalTransaction(Types.WithdrawalTransaction memory _tx)

....

409: // Trigger the call to the target contract. We use a custom low level method
410: // SafeCall.callWithMinGas to ensure two key properties
411: // 1. Target contracts cannot force this call to run out of gas by returning a very l
arge
412: // amount of data (and this is OK because we don't care about the returndata her
e).
413: // 2. The amount of gas provided to the execution context of the target is at least t
he
414: // gas limit specified by the user. If there is not enough gas in the current cont
ext
415: // to accomplish this, `callWithMinGas` will revert.
416: bool l1mntSuccess = false;
417: if (_tx.mntValue>0){
418:     l1mntSuccess = IERC20(L1_MNT_ADDRESS).transfer(_tx.target, _tx.mntValue);
419: }
420: bool success = SafeCall.callWithMinGas(_tx.target, _tx.gasLimit, _tx.ethValue, _tx.dat
a);
421: // Reset the l2Sender back to the default value.422:         l2Sender = Constants.DEFAU
LT_L2_SENDER;
423:
424:         // All withdrawals are immediately finalized. Replayability can425:         // be achie
ved through contracts built on top of this contract426:         emit WithdrawalFinalized(withdrawalH
ash, success && l1mntSuccess);

```

```

the
429:         // sub call to the target contract if the minimum gas limit specified by the user would
not430:         // be sufficient to execute the sub call.431:         if (success && l1mntSuccess ==
false&&tx.origin== Constants.ESTIMATION_ADDRESS) {
432:             revert("OptimismPortal: withdrawal failed");
433:         }

....

428: // Reverting here is useful for determining the exact gas cost to successfully execute the429:
// sub call to the target contract if the minimum gas limit specified by the user would not430:
// be sufficient to execute the sub call.431:         if (success && l1mntSuccess ==false&&tx.origin
== Constants.ESTIMATION_ADDRESS) {
432:             revert("OptimismPortal: withdrawal failed");
433:         }

....

431: if (success && l1mntSuccess ==false&&tx.origin== Constants.ESTIMATION_ADDRESS) {
432:     revert("OptimismPortal: withdrawal failed");
433: }

```

Description

lemonmon: To bridge between mainnet and mantle without using `CrossDomainMessenger`, it is very important to estimate the gas needed for the call. If the call to the target fails due to out of gas error, the user would not be able to replay the transaction again. In the case of L2 to L1 transaction, if the user specifies too less gas as minimum gas limit (than the real call), a malicious attacker can finalize the transaction via `OptimismPortal::finalizeWithdrawalTransaction` with the user specified minimum gas limit. If the user underestimated the gas limit, the call by the attacker will result in out of gas error, and the user's fund will be locked.

There is a mechanism to estimate the gas usage by calling on the OptimismPortal using `Constants.ESTIMATION_ADDRESS` as the from address. Using this mechanism the user can estimate how much gas is needed for a call to the OptimismPortal.

<https://docs.optimism.io/stack/protocol/deposit-flow>

If the `OptimismPortal::finalizeWithdrawalTransaction` is called by the `Constants.ESTIMATION_ADDRESS`, it will revert unless the call to the target succeed. It is to ensure to estimate the gas, enough for the call to the target and succeed.

In the case of Mantle, there will be MNT transfer and call to the target. To estimate the gas enough for these calls (when it is called by the `ESTIMATION_ADDRESS`, if either of the call fails, it should revert.

However, the conditions are not implemented correctly. There is a test code to test whether the call will revert for each cases below.

```

// Reverting here is useful for determining the exact gas cost to successfully execute the
// sub call to the target contract if the minimum gas limit specified by the user would no
t
// be sufficient to execute the sub call.
if (success && l1mntSuccess == false && tx.origin == Constants.ESTIMATION_ADDRESS) {
    revert("OptimismPortal: withdrawal failed");
}

```

```

// SPDX-License-Identifier: MIT
pragma

```

```

solidity 0.8.15;
import "forge-std/Test.sol";
contract MyTest is Test {
    address constant ESTIMATION_ADDRESS = address(1);
    function testConditionalLogging() public {
        address origin = address(1);
        // Loop through all combinations of success and l1mntSuccess
        for (uint8 i = 0; i < 4; i++) {
            bool success = i & 1 > 0; // Get 'success' from bit 0
            bool l1mntSuccess = i & 2 > 0; // Get 'l1mntSuccess' from bit 1
            if (success && l1mntSuccess == false && origin == ESTIMATION_ADDRESS) {
                emit log_named_string("revert", toString(success, l1mntSuccess));
            } else {
                emit log_named_string("doesn't revert", toString(success, l1mntSuccess));
            }
        }
    }
    function toString(bool success, bool l1mntSuccess) pure public returns (string memory) {
        return string(abi.encodePacked(
            "success: ", success ? "true" : "false",
            ", l1mntSuccess: ", l1mntSuccess ? "true" : "false"
        ));
    }
}

```

Logs:

```

doesn't revert: success: false, l1mntSuccess: false
revert          : success: true, l1mntSuccess: false
doesn't revert: success: false, l1mntSuccess: true
doesn't revert: success: true, l1mntSuccess: true

```

The function should not revert only when both results are true. But the condition does not behave as desired as shown above. Note that the `l1mntSuccess` will be always true, otherwise the entire call will revert. Which means even if the call to the target reverts due to out of gas, as long as the `l1mnt` transfer happens, it will not revert. As the result the user will assume that the call to the target had enough gas. If the user trusts this gas estimation and uses the gas as the minimum gas, the user's transaction will be subject to the griefing attack.

newway55: Different to the original optimism portal's `finalizeWithdrawalTransaction`, the mantle optimism portal introduces an extra transfer into the logic of `finalizeWithdrawalTransaction` which is a mantle token transfer on L1, the logic for the `ESTIMATION_ORACLE` is as follows :

```

if (success && l1mntSuccess == false && tx.origin == Constants.ESTIMATION_ADDRESS) {
    revert("OptimismPortal: withdrawal failed");
}

```

This is incorrect, because if `l1mntSuccess` and `success` are both false, the transaction will not revert and the `ESTIMATION` will fail. The transaction should revert if any of the two transactions fail.

rajatbeladiya: The `finalizeWithdrawalTransaction()` Method in the `OptimismPortal.sol` contract does not correctly handle failures in the MNT token transfer. The current implementation only reverts the transaction when `i`

```
(success && l1mntSuccess == false && tx.origin == Constants.ESTIMATION_ADDRESS) {
    revert("OptimismPortal: withdrawal failed");
}
```

This means that if the MNT token transfer fails (`l1mntSuccess = false`) and `success = false`, the contract will not revert and the function will continue executing. This could lead to inconsistent state even though the MNT token transfer failed.

Impact: This could lead to a situation where the MNT token transfer fails but the contract does not revert leading to loss of funds or incorrect balances.

in this scenario, the user would expect to receive the MNT tokens on the L1 chain as part of the withdrawal. However, due

Yaodao: The following codes in the function `finalizeWithdrawalTransaction()` are used to transfer the tokens.

```
// Trigger the call to the target contract. We use a custom low level method
// SafeCall.callWithMinGas to ensure two key properties
// 1. Target contracts cannot force this call to run out of gas by returning a very large
// amount of data (and this is OK because we don't care about the returndata here).
// 2. The amount of gas provided to the execution context of the target is at least the
// gas limit specified by the user. If there is not enough gas in the current context
// to accomplish this, `callWithMinGas` will revert.
bool l1mntSuccess = false;
if (_tx.mntValue > 0){
    l1mntSuccess = IERC20(L1_MNT_ADDRESS).transfer(_tx.target, _tx.mntValue);
}
bool success = SafeCall.callWithMinGas(_tx.target, _tx.gasLimit, _tx.ethValue, _tx.data);
// Reset the l2Sender back to the default value.
l2Sender = Constants.DEFAULT_L2_SENDER;

// All withdrawals are immediately finalized. Replayability can
// be achieved through contracts built on top of this contract
emit WithdrawalFinalized(withdrawalHash, success && l1mntSuccess);
```

After the transfer, the result of the transfer will be checked. Due to the call of `SafeCall.callWithMinGas()` only returning the result but never reverting when the call fails.

As a result, for the current check, it will revert only when `IERC20(L1_MNT_ADDRESS).transfer` fails and `SafeCall.callWithMinGas` succeeds but not revert when the call `SafeCall.callWithMinGas()` fails.

```
// Reverting here is useful for determining the exact gas cost to successfully execute the
// sub call to the target contract if the minimum gas limit specified by the user would not
// be sufficient to execute the sub call.
if (success && l1mntSuccess == false && tx.origin == Constants.ESTIMATION_ADDRESS) {
    revert("OptimismPortal: withdrawal failed");
}
```

biakia: In `OptimismPortal` contract, the function `finalizeWithdrawalTransaction` will trigger a sub call to the target contract by following code:

```
bool
```

```
success = SafeCall.callWithMinGas(_tx.target, _tx.gasLimit, _tx.ethValue, _tx.data);
```

It is possible that the provided gas is not sufficient to execute the sub call. As a result, before calling the function `finalizeWithdrawalTransaction`, the account `Constants.ESTIMATION_ADDRESS` will be used to estimate the gas consumption at first. When the gas is not sufficient, the call of the function `finalizeWithdrawalTransaction` should revert:

```
bool l1mntSuccess = false;
if (_tx.mntValue>0){
    l1mntSuccess = IERC20(L1_MNT_ADDRESS).transfer(_tx.target, _tx.mntValue);
}
bool success = SafeCall.callWithMinGas(_tx.target, _tx.gasLimit, _tx.ethValue, _tx.data);
// Reset the l2Sender back to the default value.
l2Sender = Constants.DEFAULT_L2_SENDER;

// All withdrawals are immediately finalized. Replayability can
// be achieved through contracts built on top of this contract
emit WithdrawalFinalized(withdrawalHash, success && l1mntSuccess);

// Reverting here is useful for determining the exact gas cost to successfully execute the
// sub call to the target contract if the minimum gas limit specified by the user would no
t
// be sufficient to execute the sub call.
if (success && l1mntSuccess == false && tx.origin == Constants.ESTIMATION_ADDRESS) {
    revert("OptimismPortal: withdrawal failed");
}
```

However, the condition for reverting is incorrect:

1. When the MNT transfer does not exists, the `l1mntSuccess` will be false. if the sub call succeeds, the `success` will be true, and the `success && l1mntSuccess == false && tx.origin == Constants.ESTIMATION_ADDRESS` will be true too. As a result, when the sub call succeed, the call for gas estimation will revert. The caller cannot estimate gas for this successful sub call.
2. When the MNT transfer exists and succeeds, the `l1mntSuccess` will be true, the `success && l1mntSuccess == false && tx.origin == Constants.ESTIMATION_ADDRESS` will always be false. As a result, the caller can estimate gas for a failed sub call.

Recommendation

lemonmon: The function should revert only when both calls succeed.

```
// Reverting here is useful for determining the exact gas cost to successfully execute the// sub c
all to the target contract if the minimum gas limit specified by the user would not// be sufficien
t to execute the sub call.-if (success && l1mntSuccess ==false&&tx.origin== Constants.ESTIMATION_A
DDRESS) {
+if ((!success ||!l1mntSuccess) &&tx.origin== Constants.ESTIMATION_ADDRESS) {
    revert("OptimismPortal: withdrawal failed");
}
```

newway55: - change the logic to revert if any of the two transactions fail.

```
if ((success == false || l1mntSuccess == false) && tx.origin == Constants.ESTIMATION_ADDRESS) {
    revert("OptimismPortal: withdrawal failed");
}
```

rajatbeladiya: handle the failure of the MNT token transfer independently

```
if (!l1mntSuccess) {
    revert("OptimismPortal: MNT token transfer failed");
}

if (success && tx.origin == Constants.ESTIMATION_ADDRESS) {
    revert("OptimismPortal: withdrawal failed");
}
```

Yaodao: Recommend updating the revert check as follows:

```
if ((success == false || l1mntSuccess == false) && tx.origin == Constants.ESTIMATION_ADDRE
SS) {
    revert("OptimismPortal: withdrawal failed");
}
```

biakia: Consider following fix:

```
bool l1mntSuccess = true;
if (_tx.mntValue>0){
    l1mntSuccess = IERC20(L1_MNT_ADDRESS).transfer(_tx.target, _tx.mntValue);
}
....
....
if ((success==false || l1mntSuccess == false) && tx.origin == Constants.ESTIMATION_ADDRESS) {
    revert("OptimismPortal: withdrawal failed");
}
```

Client Response

lemonmon: Fixed.Already fixed in <https://github.com/mantlenetworkio/mantle-v2/pull/95/files>

newway55: Fixed.Already fixed in <https://github.com/mantlenetworkio/mantle-v2/pull/95/files>

rajatbeladiya: Fixed.Already fixed in <https://github.com/mantlenetworkio/mantle-v2/pull/95/files>

Yaodao: Fixed.Already fixed in <https://github.com/mantlenetworkio/mantle-v2/pull/95/files>

Fixed. Already fixed in <https://github.com/mantlenetworkio/mantle-v2/pull/95/files>

MNT-8:Funds transferred to incorrect user before snapshot

Category	Severity	Client Response	Contributor
Logical	Critical	Fixed	csanuragjain

Code Reference

- [code/op-geth/core/state_transition.go#L441-L446](#)
- [code/op-geth/core/state_transition.go#L449](#)

```
....  
  
441: if ethValue := st.msg.ETHValue; ethValue != nil && ethValue.Cmp(big.NewInt(0)) != 0 {  
442:     st.addBVMETHBalance(ethValue)  
443:     st.addBVMETHTotalSupply(ethValue)  
444:     st.generateBVMETHMintEvent(*st.msg.To, ethValue)  
445: }  
446: snap := st.state.Snapshot()  
  
....  
  
449: // Failed deposits must still be included. Unless we cannot produce the block at all due to the  
    gas limit.
```

Description

csanuragjain: ## Description L2 side mints eth to recipient instead of sender before snapshot. This means sender loses funds if the L2 call fails since failing still cause recipient to get those ETH funds

Steps

1. Currently, on sending message from L1 to L2, below events occur:
2. MNT transferred is minted to sender address
3. ETH transferred is minted to recipient address
4. A snapshot is taken
5. Rest operation like MNT transfer and calling target calldata happens

```
if
```

```
mint := st.msg.Mint; mint != nil {
    st.state.AddBalance(st.msg.From, mint)
}
//add eth valueif ethValue := st.msg.ETHValue; ethValue != nil && ethValue.Cmp(big.NewInt(0))
!=0 {
    st.addBVMETHBalance(ethValue)
    st.addBVMETHTotalSupply(ethValue)
    st.generateBVMETHMintEvent(*st.msg.To, ethValue)
}
snap := st.state.Snapshot()
```

2. This becomes a problem if transaction fails (one reason maybe because MNT transferred is insufficient). This causes Go code to use last snapshot which is Step 3
3. This means on L2 side, MNT is minted to sender but ETH is minted to target without any target calldata calling
4. This might not be expected, as the target calling may have updated user balance which will not happen in this case and User would lose ETH

Recommendation

csanuragjain: Before snapshot, eth should be minted to sender side and post snapshot, it should be then transferred to recipient as done with MNT balance

Client Response

csanuragjain: Fixed. <https://github.com/mantlenetworkio/op-geth/pull/31> mint ETH in 2 steps

MNT-9: core/state_transition.go::TransitionDb will runtime panic and crash node by calling create contract with non-zero eth value

Category	Severity	Client Response	Contributor
Language Specific	Medium	Fixed	lemonmon

Code Reference

- code/op-geth/core/state_transition.go#L441-L445
- code/op-geth/core/state_transition.go#L668

```
....  
  
441: if ethValue := st.msg.ETHValue; ethValue != nil && ethValue.Cmp(big.NewInt(0)) != 0 {  
442:     st.addBVMETHBalance(ethValue)  
443:     st.addBVMETHTotalSupply(ethValue)  
444:     st.generateBVMETHMintEvent(*st.msg.To, ethValue)  
445: }  
  
....  
  
668: key := getBVMETHBalanceKey(*st.msg.To)
```

Description

lemonmon: For contract creation transactions, the `To` address will be `nil`. However, the `TransitionDb` does not have the nil check for the `To` address.

The `addBVMETHBalance` function will try to deference the `st.msg.To` and it will have runtime panic if the `st.msg.To` is nil.

If the node crushes, it means the node will be temporarily unavailable.

By just composing transaction of contract creation (no `To` address specified), with non-zero ethValue, it may lead to node crush.

```
// core/state_transition.go::TransitionDb  
    if ethValue := st.msg.ETHValue; ethValue != nil && ethValue.Cmp(big.NewInt(0)) != 0 {  
        st.addBVMETHBalance(ethValue)  
        st.addBVMETHTotalSupply(ethValue)  
        st.generateBVMETHMintEvent(*st.msg.To, ethValue)  
    }  
  
// ...  
  
func (st *StateTransition) addBVMETHBalance(ethValue *big.Int) {  
    key := getBVMETHBalanceKey(*st.msg.To)
```

Check for ``nil`` before dereferencing.

Client Response

lemonmon: Fixed. Already fixed in <https://github.com/mantlenetworkio/op-geth/pull/32/files>

MNT-10: Wrong estimation of the `RELAY_RESERVED_GAS` might cause a DoS

Category	Severity	Client Response	Contributor
DOS	Medium	Acknowledged	newway55

Code Reference

- code/mantle-v2/packages/contracts-bedrock/contracts/L1/L1CrossDomainMessenger.sol#L217-L237

```
....
217: bool success = SafeCall.call(_target, gasleft() - RELAY_RESERVED_GAS, _ethValue, _message);
218:     xDomainMsgSender = Constants.DEFAULT_L2_SENDER;
219:     if (_mntValue!=0){
220:         mntSuccess = IERC20(L1_MNT_ADDRESS).approve(_target, 0);
221:     }
222:     if (success && mntSuccess) {
223:         successfulMessages[versionedHash] = true;
224:         emit RelayedMessage(versionedHash);
225:     } else {
226:         failedMessages[versionedHash] = true;
227:         emit FailedRelayedMessage(versionedHash);
228:
229:         // Revert in this case if the transaction was triggered by the estimation address.
This
230:         // should only be possible during gas estimation or we have bigger problems. Revert
ing
231:         // here will make the behavior of gas estimation change such that the gas limit
232:         // computed will be the amount required to relay the message, even if that amount i
s
233:         // greater than the minimum gas limit specified by the user.
234:         if (tx.origin == Constants.ESTIMATION_ADDRESS) {
235:             revert("CrossDomainMessenger: failed to relay message");
236:         }
237:     }
```

Description

newway55: The contract `L1CrossDomainMessenger` uses the following constant from the `universal`/`CrossDomainMessenger.sol`` contract:

```
/**
 * @notice Gas reserved for finalizing the execution of `relayMessage` after the safe call.
 */
uint64 public constant RELAY_RESERVED_GAS = 40_000;
```

this constant is used [here](#), the constant has the same value as the original one which can be found [here](#), however extra logic is introduced after the call to `relayMessage` in the `L1CrossDomainMessenger` contract, which can lead to a DoS if the `RELAY_RESERVED_GAS` is not enough to finalize the execution of `relayMessage`.

ORIGINAL :

```

= Constants.DEFAULT_L2_SENDER;

    if (success) {
        // This check is identical to one above, but it ensures that the same message cannot b
        e relayed// twice, and adds a layer of protection against reentrancy.assert(successfulMessages[vers
        ionedHash] ==false);
        successfulMessages[versionedHash] =true;
        emit RelayedMessage(versionedHash);
    } else {
        failedMessages[versionedHash] =true;
        emit FailedRelayedMessage(versionedHash);

        // Revert in this case if the transaction was triggered by the estimation address. Thi
        s// should only be possible during gas estimation or we have bigger problems. Reverting// here wil
        l make the behavior of gas estimation change such that the gas limit// computed will be the amount
        required to relay the message, even if that amount is// greater than the minimum gas limit specifi
        ed by the user.if (tx.origin== Constants.ESTIMATION_ADDRESS) {
            revert("CrossDomainMessenger: failed to relay message");
        }
    }
}

```

MANTLE VERSION :

```

xDomainMsgSender = Constants.DEFAULT_L2_SENDER;
    if (_mntValue!=0){
        mntSuccess = IERC20(L1_MNT_ADDRESS).approve(_target, 0);
    }
    if (success && mntSuccess) {
        successfulMessages[versionedHash] = true;
        emit RelayedMessage(versionedHash);
    } else {
        failedMessages[versionedHash] = true;
        emit FailedRelayedMessage(versionedHash);

        // Revert in this case if the transaction was triggered by the estimation address. Thi
        s

        // should only be possible during gas estimation or we have bigger problems. Reverting
        // here will make the behavior of gas estimation change such that the gas limit
        // computed will be the amount required to relay the message, even if that amount is
        // greater than the minimum gas limit specified by the user.
        if (tx.origin == Constants.ESTIMATION_ADDRESS) {
            revert("CrossDomainMessenger: failed to relay message");
        }
    }
}

```

```
if (_mntValue!=0){  
    mntSuccess = IERC20(L1_MNT_ADDRESS).approve(_target, 0);  
}
```

This can cause the `RELAY\_RESERVED\_GAS` to be not enough to finalize the execution of `relayMessage` and can lead to a DoS.

Recommendation

newway55: Calculate the extra gas cost and add it to the RELAY_RESERVED_GAS constant.

Client Response

newway55: Acknowledged. will evaluate the risk later.

MNT-11: Vulnerability in getPriceFromUniswap function

Category	Severity	Client Response	Contributor
Logical	Medium	Acknowledged	zircon

Code Reference

- code/mantle-v2/gas-oracle/tokenratio/tokenratio_dex.go#L76-L92

```
....  
76: func (c *Client) getTokenPriceFromUniswap(fromToken, toToken common.Address, decimals *big.Float)  
    t) (float64, error) {  
77:     fee := big.NewInt(3000)  
78:     fromAmount := floatStringToBigInt("1.00", 18)  
79:     sqrtPriceLimitX96 := big.NewInt(0)  
80:  
81:     var out []interface{}  
82:     rawCaller := &bindings.Uniswapv3QuoterRaw{Contract: c.uniswapQuoterClient.uniswapV3Quoter}  
83:     err := rawCaller.Call(nil, &out, "quoteExactInputSingle", fromToken, toToken,  
84:         fee, fromAmount, sqrtPriceLimitX96)  
85:     if err != nil {  
86:         return 0, err  
87:     }  
88:  
89:     resultBigFloat := new(big.Float).SetInt(out[0].(*big.Int))  
90:     result, _ := new(big.Float).Quo(resultBigFloat, decimals).Float64()  
91:     return result, nil  
92: }
```

Description

zircon: The `getPriceFromUniswap` function fetches token prices from Uniswap using the `quoteExactInputSingle` method. However, it is crucial to note that this method retrieves instantaneous prices, not Time-Weighted Average Price (TWAP) prices. Instantaneous prices are prone to significant short-term fluctuations and may deviate substantially from the actual market price. This vulnerability exposes the system to potential inaccuracies and exploitation due to the lack of stability in price data.

Recommendation

zircon: It is recommended to replace the usage of `quoteExactInputSingle` with a method that retrieves TWAP prices. TWAP prices provide a more stable and reliable representation of the token's true market value over a specified time period, thus mitigating the risks associated with short-term price fluctuations. Implementing TWAP-based price retrieval methods enhances the accuracy and resilience of the system against potential price manipulation or inaccuracies.

Client Response

zircon: Acknowledged. Will fix next time.

MNT-12: Use of `safeTransferFrom` instead of `transferFrom` in `L1ERC721Bridge.sol`

Category	Severity	Client Response	Contributor
Logical	Medium	Fixed	rajatbeladiya

Code Reference

- code/mantle-v2/packages/contracts-bedrock/contracts/L1/L1ERC721Bridge.sol#L68
- code/mantle-v2/packages/contracts-bedrock/contracts/L1/L1ERC721Bridge.sol#L101

```
....  
68: IERC721(_localToken).safeTransferFrom(address(this), _to, _tokenId);  
  
....  
101: IERC721(_localToken).transferFrom(_from, address(this), _tokenId);
```

Description

rajatbeladiya: ```solidity IERC721(_localToken).safeTransferFrom(address(this), _to, _tokenId);

here `_finalizeBridgeERC721()` function uses `safeTransferFrom()` to transfer the ERC721, but `_initiateBridgeERC721()` function uses `transferFrom()` to transfer the ERC721.

```
```solidity  
ERC721(_localToken).transferFrom(_from, address(this), _tokenId);
```

Note that the `transferFrom` method of both `ERC721` and `ERC20` have the same byte4 signature, which is `0x23b872dd`.

This implies that if `_localToken` is an `ERC20` instead of `ERC721`, the transfer can still be made, and the ERC20 token can be mistakenly bridged from L1 to L2. However, the token cannot be bridged back.

Moreover, the `onERC721Received()` function of the recipient is only triggered in the `safeTransferFrom()` function and not in `transferFrom()`.

### Recommendation

rajatbeladiya: use `safeTransferFrom()` instead of `transferFrom()` when transferring ERC721s

### Client Response

rajatbeladiya: Fixed. <https://github.com/mantlenetworkio/mantle-v2/pull/103/files>

## MNT-13: The price used when calculating `tokenRatio` will always come from `cex`

Category	Severity	Client Response	Contributor
Logical	Medium	Acknowledged	biakia

### Code Reference

- code/mantle-v2/gas-oracle/tokenratio/tokenratio.go#L121-L160

```
....
121: func (c *Client) tokenRatio() (float64, error) {
122: // Todo query token prices concurrent
123: var mntPrices, ethPrices []float64
124: // get token price from oracle1(dex)
125: mntPrice1, ethPrice1 := c.getTokenPricesFromUniswap()
126: mntPrices = append(mntPrices, mntPrice1)
127: ethPrices = append(ethPrices, ethPrice1)
128: log.Info("query prices from oracle1", "mntPrice", mntPrice1, "ethPrice", ethPrice1)
129:
130: // get token price from oracle2(cex)
131: mntPrice2, ethPrice2 := c.getTokenPricesFromCex()
132: mntPrices = append(mntPrices, mntPrice2)
133: ethPrices = append(ethPrices, ethPrice2)
134: log.Info("query prices from oracle2", "mntPrice", mntPrice2, "ethPrice", ethPrice2)
135:
136: // get token price from oracle3(cex)
137: // Todo add a third oracle to query prices
138: mntPrice3, ethPrice3 := c.getTokenPricesFromCex()
139: mntPrices = append(mntPrices, mntPrice3)
140: ethPrices = append(ethPrices, ethPrice3)
141: log.Info("query prices from oracle3", "mntPrice", mntPrice3, "ethPrice", ethPrice3)
142:
143: // median price for eth & mnt
144: medianMNTPrice := getMedian(mntPrices)
145: medianETHPrice := getMedian(ethPrices)
146:
147: // determine mnt_price, eth_price
148: mntPrice := c.determineMNTPrice(medianMNTPrice)
149: ethPrice := c.determineETHPrice(medianETHPrice)
150: log.Info("prices after determine", "mntPrice", mntPrice, "ethPrice", ethPrice)
151:
152: // calculate ratio
153: ratio := c.determineTokenRatio(mntPrice, ethPrice)
154:
155: c.lastRatio = ratio
156: c.lastEthPrice = ethPrice
157: c.lastMntPrice = mntPrice
158:
159: return ratio, nil
160: }
```

### Description

In `tokenRatio.go`, the function `tokenRatio()` will be used to calculate the param `tokenRatio`. It will fetch three prices, one from dex and two from cex:

```
// Todo query token prices concurrent
var mntPrices, ethPrices []float64
// get token price from oracle1(dex)
mntPrice1, ethPrice1 := c.getTokenPricesFromUniswap()
mntPrices = append(mntPrices, mntPrice1)
ethPrices = append(ethPrices, ethPrice1)
log.Info("query prices from oracle1", "mntPrice", mntPrice1, "ethPrice", ethPrice1)

// get token price from oracle2(cex)
mntPrice2, ethPrice2 := c.getTokenPricesFromCex()
mntPrices = append(mntPrices, mntPrice2)
ethPrices = append(ethPrices, ethPrice2)
log.Info("query prices from oracle2", "mntPrice", mntPrice2, "ethPrice", ethPrice2)

// get token price from oracle3(cex)
// Todo add a third oracle to query prices
mntPrice3, ethPrice3 := c.getTokenPricesFromCex()
mntPrices = append(mntPrices, mntPrice3)
ethPrices = append(ethPrices, ethPrice3)
log.Info("query prices from oracle3", "mntPrice", mntPrice3, "ethPrice", ethPrice3)
```

The function `getTokenPricesFromCex()` will be called for twice, which means the second and the third price are from the same cex. Since the interval between these two calls is very short, most of the time the prices returned by these two calls will be the same. After fetching all prices, the function `getMedian` will be called to calculate a median price:

```
// median price for eth & mnt
medianMNTPrice := getMedian(mntPrices)
medianETHPrice := getMedian(ethPrices)
```

In function `getMedian`, the prices will be sorted first and then the median price will be returned:

```
func getMedian(nums []float64) float64 {
 nonZeros := make([]float64, 0)
 for _, num := range nums {
 if num != 0 {
 nonZeros = append(nonZeros, num)
 }
 }
 sort.Float64s(nonZeros)
 if len(nonZeros) == 0 {
 return 0
 }
 return nonZeros[len(nonZeros)/2]
}
```

`nonZeros` array will be an array which size is 3, so the `nonZeros[len(nonZeros)/2]` will be `nonZeros[3/2]`. At last, the second price will be returned. As mentioned above, the second price and the third price can likely to be the same. Consider the following two case:

1. The eth price from the dex is 2100 and the eth price from cex is 2101, after sorted, the `nonZeros` will be [2100,2101,2101]. In this case, the returned price will be 2101, which is the price from the cex.
2. The eth price from the dex is 2100 and the eth price from cex is 2099, after sorted, the `nonZeros` will be [2099,2099,2100]. In this case, the returned price will be 2099, which is the price from the cex too.

At last, only cex price will be returned.

What's more, if the cex is down, it will always return 0:

```
func (c *Client) queryV5(symbol string) (float64, error) {
 response, err := c.client.R().
 SetResult(&Response{}).
 SetQueryParams(map[string]string{
 "symbol": symbol,
 }).
 Get("v5/market/tickers?category=linear&")
 if err != nil {
 return 0, fmt.Errorf("cannot fetch token price result: %w", err)
 }
 result, ok := response.Result().(*Response)
 if !ok {
 return 0, fmt.Errorf("cannot parse result")
 }
 if result.RetCode != noHTTPError {
 return 0, fmt.Errorf("query error")
 }
 if len(result.PriceResult.List) == 0 {
 return 0, fmt.Errorf("empty price in result")
 }
 priceBigFloat, _ := big.NewFloat(0).SetString(result.PriceResult.List[0].IndexPrice)
 priceFloat64, _ := priceBigFloat.Float64()
 return priceFloat64, nil
}
```

So the median price will always be 0 and when determining `mnt\_price` and `eth\_price`, last price will be returned:

```
func
```

```
(c *Client) determineMNTPrice(price float64) float64 {
 if price > MNTPriceMax || price < MNTPriceMin {
 return c.lastMntPrice
 }

 return price
}

func (c *Client) determineETHPrice(price float64) float64 {
 if price > ETHPriceMax || price < ETHPriceMin {
 return c.lastEthPrice
 }

 return price
}
```

As a result, the calculation for `tokenRatio` will always be based on expired prices.

## Recommendation

**biakia:** Consider using a third-part price for the second price, for example, a price from pyth or chainlink.

## Client Response

**biakia:** Acknowledged. Will fix next time.

## MNT-14: The function `getTokenPricesFromUniswap` may return incorrect prices

Category	Severity	Client Response	Contributor
Logical	Medium	Acknowledged	biakia

### Code Reference

- code/mantle-v2/gas-oracle/tokenratio/tokenratio\_dex.go#L57-L73

```
....
57: func (c *Client) getTokenPricesFromUniswap() (float64, float64) {
58:
59: eth2mntPrice, err := c.getTokenPriceFromUniswap(c.uniswapQuoterClient.ethAddress,
60: c.uniswapQuoterClient.mntAddress, c.uniswapQuoterClient.mntDecimals)
61: if err != nil {
62: log.Warn("get token prices from dex", "query eth/mnt error", err)
63: return 0, 0
64: }
65: eth2usdtPrice, err := c.getTokenPriceFromUniswap(c.uniswapQuoterClient.ethAddress,
66: c.uniswapQuoterClient.usdtAddress, c.uniswapQuoterClient.usdtDecimals)
67: if err != nil {
68: log.Warn("get token prices from dex", "query eth/usdt error", err)
69: return 0, eth2mntPrice
70: }
71:
72: return eth2usdtPrice / eth2mntPrice, eth2usdtPrice
73: }
```

### Description

biakia: The function `getTokenPricesFromUniswap` will get `eth/mnt` price and `eth/usdt` price through uniswap:

```
func
```

```
(c *Client) getTokenPricesFromUniswap() (float64, float64) {

 eth2mntPrice, err := c.getTokenPriceFromUniswap(c.uniswapQuoterClient.ethAddress,
 c.uniswapQuoterClient.mntAddress, c.uniswapQuoterClient.mntDecimals)
 if err != nil {
 log.Warn("get token prices from dex", "query eth/mnt error", err)
 return 0, 0
 }
 eth2usdtPrice, err := c.getTokenPriceFromUniswap(c.uniswapQuoterClient.ethAddress,
 c.uniswapQuoterClient.usdtAddress, c.uniswapQuoterClient.usdtDecimals)
 if err != nil {
 log.Warn("get token prices from dex", "query eth/usdt error", err)
 return 0, eth2mntPrice
 }

 return eth2usdtPrice / eth2mntPrice, eth2usdtPrice
}
```

The first returned value is `eth2usdtPrice / eth2mntPrice`, which will be `mnt/usdt` price, the second returned value is `eth/usdt` price. However, when failing to query `eth/usdt`, it will return `(0, eth2mntPrice)`:

```
eth2usdtPrice, err := c.getTokenPriceFromUniswap(c.uniswapQuoterClient.ethAddress,
 c.uniswapQuoterClient.usdtAddress, c.uniswapQuoterClient.usdtDecimals)
if err != nil {
 log.Warn("get token prices from dex", "query eth/usdt error", err)
 return 0, eth2mntPrice
}
```

The second returned value is `eth/mnt` price instead of `eth/usdt` price here, which is incorrect. Incorrect prices can lead to unintended side effects.

## Recommendation

**biakia:** Consider returning `(0,0)` instead when failing to query prices from uniswap:

```
func
```

```
(c *Client) getTokenPricesFromUniswap() (float64, float64) {

 eth2mntPrice, err := c.getTokenPriceFromUniswap(c.uniswapQuoterClient.ethAddress,
 c.uniswapQuoterClient.mntAddress, c.uniswapQuoterClient.mntDecimals)
 if err != nil {
 log.Warn("get token prices from dex", "query eth/mnt error", err)
 return 0, 0
 }
 eth2usdtPrice, err := c.getTokenPriceFromUniswap(c.uniswapQuoterClient.ethAddress,
 c.uniswapQuoterClient.usdtAddress, c.uniswapQuoterClient.usdtDecimals)
 if err != nil {
 log.Warn("get token prices from dex", "query eth/usdt error", err)
 return 0, 0
 }

 return eth2usdtPrice / eth2mntPrice, eth2usdtPrice
}
```

## Client Response

**biakia:** Acknowledged. Will fix later

## MNT-15: The `txpool.go` does not properly remove invalid transactions by including expired sponsor payment to the sponsor cost sum

Category	Severity	Client Response	Contributor
Logical	Medium	Fixed	lemonmon

### Code Reference

- code/op-geth/core/txpool/txpool.go#L1491-L1493

```
....
1491: if metaTxParams.ExpireHeight < currHeight {
1492: invalidMetaTxs = append(invalidMetaTxs, tx)
1493: }
```

### Description

**lemonmon:** The `txpool.go` is checking for the transactions for their validity and remove invalid transactions and promote valid transactions. It is important to validate the transactions properly for the performance of the network. The txpool has a limited size, and keeping invalid transactions consumes valuable space that could otherwise be used by valid transactions. This leads to unnecessary congestion in the txpool, potentially causing delays in processing and including valid transactions in the blockchain. Over time, this can degrade the performance of the node, as it has to manage a larger set of transactions, including processing, validating, and storing them. Also it may lead to the misallocation of miner's effort.

The function `validateMetaTxList` in the `core/txpool/txpool.go` is validating the meta transaction, and returns what the sponsor would pay given the list of transactions.

[https://github.com/Secure3Audit/code\\_Mantle\\_V2/blob/97bb2aad421c306241ca1681dd3c68b6dfc3355b/code/op-geth/core/txpool/txpool.go#L1491-L1493](https://github.com/Secure3Audit/code_Mantle_V2/blob/97bb2aad421c306241ca1681dd3c68b6dfc3355b/code/op-geth/core/txpool/txpool.go#L1491-L1493)

It is used in `validateTx`, `promoteExecutables`, `demoteUnexecutables` to check whether the sponsor and the `from` address have enough balance, therefore validating the transaction and promote or demote them.

[https://github.com/Secure3Audit/code\\_Mantle\\_V2/blob/97bb2aad421c306241ca1681dd3c68b6dfc3355b/code/op-geth/core/txpool/txpool.go#L726](https://github.com/Secure3Audit/code_Mantle_V2/blob/97bb2aad421c306241ca1681dd3c68b6dfc3355b/code/op-geth/core/txpool/txpool.go#L726)

[https://github.com/Secure3Audit/code\\_Mantle\\_V2/blob/97bb2aad421c306241ca1681dd3c68b6dfc3355b/code/op-geth/core/txpool/txpool.go#L1529](https://github.com/Secure3Audit/code_Mantle_V2/blob/97bb2aad421c306241ca1681dd3c68b6dfc3355b/code/op-geth/core/txpool/txpool.go#L1529)

[https://github.com/Secure3Audit/code\\_Mantle\\_V2/blob/97bb2aad421c306241ca1681dd3c68b6dfc3355b/code/op-geth/core/txpool/txpool.go#L1742](https://github.com/Secure3Audit/code_Mantle_V2/blob/97bb2aad421c306241ca1681dd3c68b6dfc3355b/code/op-geth/core/txpool/txpool.go#L1742)

However, the `validateMetaTxList` is incorrectly adding up expired meta transactions into the sponsor sum:

```
// validateMetaTxList
```

```

if metaTxParams.ExpireHeight < currHeight {
 invalidMetaTxs = append(invalidMetaTxs, tx)
}

// ...

sponsorAmount, _ := types.CalculateSponsorPercentAmount(metaTxParams, txGasCost)
if pool.currentState.GetBalance(metaTxParams.GasFeeSponsor).Cmp(sponsorAmount) >= 0 {
 sponsorCostSum = new(big.Int).Add(sponsorCostSum, sponsorAmount)
} else {
 invalidMetaTxs = append(invalidMetaTxs, tx)
}
}
return invalidMetaTxs, sponsorCostSum

```

Even though the transaction is invalid and therefore added to the `invalidMetaTxs`, it does not `continue` to the next transaction. It will end up adding sponsor amount of the expired meta transaction to the `sponsorCostSum`. This will result in overestimation of sponsor payment (by including expired sponsor's payment), therefore validating invalid transactions as valid.

## Recommendation

**lemonmon:** ```go // validateMetaTxList if metaTxParams.ExpireHeight < currHeight { invalidMetaTxs = append(invalidMetaTxs, tx)

- `continue`
- `}`

## Client Response

**lemonmon:** Fixed. <https://github.com/mantlenetworkio/op-geth/pull/43/files>

## MNT-16: Some of the invalid transactions will not be deleted from the future queue

Category	Severity	Client Response	Contributor
Logical	Medium	Acknowledged	biakia

### Code Reference

- code/op-geth/core/txpool/txpool.go#L1477-L1507

```

....
1477: func (pool *TxPool) validateMetaTxList(list *list) ([]*types.Transaction, *big.Int) {
1478: currHeight := pool.chain.CurrentBlock().Number.Uint64()
1479:
1480: var invalidMetaTxs []*types.Transaction
1481: sponsorCostSum := big.NewInt(0)
1482: for _, tx := range list.txs.Flatten() {
1483: metaTxParams, err := types.DecodeAndVerifyMetaTxParams(tx)
1484: if err != nil {
1485: invalidMetaTxs = append(invalidMetaTxs, tx)
1486: continue
1487: }
1488: if metaTxParams == nil {
1489: continue
1490: }
1491: if metaTxParams.ExpireHeight < currHeight {
1492: invalidMetaTxs = append(invalidMetaTxs, tx)
1493: }
1494: txGasCost := new(big.Int).Mul(tx.GasPrice(), new(big.Int).SetUint64(tx.Gas()))
1495: l1Cost := pool.l1CostFn(tx.RollupDataGas(), tx.IsDepositTx(), tx.To())
1496: if l1Cost != nil {
1497: txGasCost = new(big.Int).Add(txGasCost, l1Cost) // gas fee sponsor must sponsor additional l1Cost fee
1498: }
1499: sponsorAmount, _ := types.CalculateSponsorPercentAmount(metaTxParams, txGasCost)
1500: if pool.currentState.GetBalance(metaTxParams.GasFeeSponsor).Cmp(sponsorAmount) >= 0 {
1501: sponsorCostSum = new(big.Int).Add(sponsorCostSum, sponsorAmount)
1502: } else {
1503: invalidMetaTxs = append(invalidMetaTxs, tx)
1504: }
1505: }
1506: return invalidMetaTxs, sponsorCostSum
1507: }

```

### Description

**biakia:** In `txpool.go`, the function `promoteExecutables` is used to move transactions that have become processable from the future queue to the set of pending transactions. During this process, all invalidated transactions (low nonce, low balance) are deleted. It will call `validateMetaTxList` to get `invalidMetaTxs` and remove them:

```
for _, tx := range invalidMetaTxs {
 list.Remove(tx)
 hash := tx.Hash()
 pool.all.Remove(hash)
}
```

In function `validateMetaTxList`, It iterates through each transaction, calculates the `sponsorAmount`, and finally checks if the current sponsor vault is sufficient to cover it, and if the vault is insufficient, then the transaction is added to `invalidMetaTxs`:

```
func (pool *TxPool) validateMetaTxList(list *list) ([]*types.Transaction, *big.Int) {
 currHeight := pool.chain.CurrentBlock().Number.Uint64()

 var invalidMetaTxs []*types.Transaction
 sponsorCostSum := big.NewInt(0)
 for _, tx := range list.txs.Flatten() {
 metaTxParams, err := types.DecodeAndVerifyMetaTxParams(tx)
 if err != nil {
 invalidMetaTxs = append(invalidMetaTxs, tx)
 continue
 }
 if metaTxParams == nil {
 continue
 }
 if metaTxParams.ExpireHeight < currHeight {
 invalidMetaTxs = append(invalidMetaTxs, tx)
 }
 txGasCost := new(big.Int).Mul(tx.GasPrice(), new(big.Int).SetUint64(tx.Gas()))
 l1Cost := pool.l1CostFn(tx.RollupDataGas(), tx.IsDepositTx(), tx.To())
 if l1Cost != nil {
 txGasCost = new(big.Int).Add(txGasCost, l1Cost) // gas fee sponsor must sponsor additional l1Cost fee
 }
 sponsorAmount, _ := types.CalculateSponsorPercentAmount(metaTxParams, txGasCost)
 if pool.currentState.GetBalance(metaTxParams.GasFeeSponsor).Cmp(sponsorAmount) >= 0 {
 sponsorCostSum = new(big.Int).Add(sponsorCostSum, sponsorAmount)
 } else {
 invalidMetaTxs = append(invalidMetaTxs, tx)
 }
 }
 return invalidMetaTxs, sponsorCostSum
}
```

The issue here is that it simply compares the `sponsorAmount` needed for each transaction to the sponsor vault amount, rather than using the accumulated value `sponsorCostSum`. Consider the following case:

1. A user has submitted 10 transactions, each transaction has the same `sponsorAmount`, let's say the `sponsorAmount` is 10000.

2. The amount of the the sponsor vault is now only 11000.
3. For each iteration, the compare ``pool.currentState.GetBalance(metaTxParams.GasFeeSponsor).Cmp(sponsorAmount)>=0`` will be true because ``11000>10000``
4. At last, the ``sponsorCostSum`` will be 100000 and the ``invalidMetaTxs`` is empty.
5. All the transactions will be moved to the pending queue
6. When the first transaction is executed, the function ``TransitionDb()`` in ``state_transition.go`` will be called and it will call ``buyGas()``
7. In ``buyGas()``, It will check whether the balance of ``st.msg.MetaTxParams.GasFeeSponsor`` is enough:

```
if st.msg.RunMode != GasEstimationWithSkipCheckBalanceMode && st.msg.RunMode != EthcallMode {
 if st.msg.MetaTxParams != nil {
 pureGasFeeValue := new(big.Int).Sub(balanceCheck, st.msg.Value)
 sponsorAmount, selfPayAmount := types.CalculateSponsorPercentAmount(st.msg.MetaTxParams, pureGasFeeValue)
 if have, want := st.state.GetBalance(st.msg.MetaTxParams.GasFeeSponsor), sponsorAmount; have.Cmp(want) < 0 {
 return nil, fmt.Errorf("%w: gas fee sponsor %v have %v want %v", ErrInsufficientFunds, st.msg.MetaTxParams.GasFeeSponsor.Hex(), have, want)
 }
 selfPayAmount = new(big.Int).Add(selfPayAmount, st.msg.Value)
 if have, want := st.state.GetBalance(st.msg.From), selfPayAmount; have.Cmp(want) < 0 {
 return nil, fmt.Errorf("%w: address %v have %v want %v", ErrInsufficientFunds, st.msg.From.Hex(), have, want)
 }
 } else {
 if have, want := st.state.GetBalance(st.msg.From), balanceCheck; have.Cmp(want) < 0 {
 return nil, fmt.Errorf("%w: address %v have %v want %v", ErrInsufficientFunds, st.msg.From.Hex(), have, want)
 }
 }
}
```

For the first transaction, the ``sponsorAmount`` is 10000 and the balance of ``st.msg.MetaTxParams.GasFeeSponsor`` is 11000. After this execution succeeds, the balance of ``st.msg.MetaTxParams.GasFeeSponsor`` will be 1000.

8. Now the second transaction is executed, when calling ``buyGas()``, the check will fail because the balance of ``st.msg.MetaTxParams.GasFeeSponsor`` is not enough for the second transaction's ``sponsorAmount``.
9. All the 9 transactions will fail due to Insufficient balance of ``st.msg.MetaTxParams.GasFeeSponsor``.

## Recommendation

biakia: Consider comparing with ``sponsorCostSum`` instead in function ``validateMetaTxList``:

```
if
```

```
pool.currentState.GetBalance(metaTxParams.GasFeeSponsor).Cmp(new(big.Int).Add(sponsorCostSum, sponsorAmount)) >= 0 {
 sponsorCostSum = new(big.Int).Add(sponsorCostSum, sponsorAmount)
} else {
 invalidMetaTxs = append(invalidMetaTxs, tx)
}
```

## Client Response

biakia: Acknowledged. action TBD

## MNT-17:SELFDESTRUCT will be deprecated

Category	Severity	Client Response	Contributor
Logical	Medium	Acknowledged	rajatbeladiya

### Code Reference

- code/mantle-v2/packages/contracts-bedrock/contracts/L2/L2ToL1MessagePasser.sol#L91-L95

```
....
91: function burn() external {
92: uint256 balance = address(this).balance;
93: Burn.mnt(balance);
94: emit WithdrawerBalanceBurnt(balance);
95: }
```

### Description

rajatbeladiya: ``solidity function burn() external { uint256 balance = address(this).balance; Burn.mnt(balance); emit WithdrawerBalanceBurnt(balance); }

here, Mantle uses the `selfdestruct` to burn the MNT. However, after the EIP-4758 fork, the `SELFDESTRUCT` opcode will be deactivated in the future. The amount of MNT on L2 will be inflated when ETH is withdrawn from L2 as the MNT burning mechanism will not work as expected after the fork.

Consider updating the `L2ToL1MessagePasser.burn()` function in a way that sends MNT to `address(0)` to burn.

### Recommendation

rajatbeladiya: update `L2ToL1MessagePasser.burn()` to send MNT to `address(0)` to burn.

### Client Response

rajatbeladiya: Acknowledged. will fix later

## MNT-18:Resource params will be reset when reinitializing Resource Metering contract

Category	Severity	Client Response	Contributor
Logical	Medium	Fixed	Hacker007

### Code Reference

- code/mantle-v2/packages/contracts-bedrock/contracts/L1/ResourceMetering.sol#L179-L185

```
....
179: function __ResourceMetering_init() internal onlyInitializing {
180: params = ResourceParams({
181: prevBaseFee: 1 gwei,
182: prevBoughtGas: 0,
183: prevBlockNum: uint64(block.number)
184: });
185: }
```

### Description

**Hacker007:** The state param from contract `ResourceMetering` contains three fields, `prevBaseFee`, `prevBoughtGas` and `prevBlockNum`, which will be initialized in `__ResourceMetering_init()`.

```
function __ResourceMetering_init() internal onlyInitializing {
 params = ResourceParams({
 prevBaseFee: 1 gwei,
 prevBoughtGas: 0,
 prevBlockNum: uint64(block.number)
 });
}
```

After running a long time, these parameters will be used to calculate `gasCost` and updated in function `_metered()`.

```
function _metered(uint64 _amount, uint256 _initialGas) internal {
//....
 // Update new base fee, reset bought gas, and update block number.
 params.prevBaseFee = uint128(uint256(newBaseFee));
 params.prevBoughtGas = 0;
 params.prevBlockNum = uint64(block.number);
//...
}
```

However, if a new version of `OptimismPortal` which inherits from `ResourceMetering` is deployed, the three params will be reset and calculate a wrong gas cost, see <https://github.com/ethereum-optimism/optimism/pull/8639>

Only set resource parameters on the first initialization.

```
if (params.prevBlockNum == 0) {
 params = ResourceParams({ prevBaseFee: 1 gwei, prevBoughtGas: 0, prevBlockNum: uint64
(block.number) });
}
```

## Client Response

Hacker007: Fixed. <https://github.com/mantlenetworkio/mantle-v2/pull/111/files>

## MNT-19: Relay message on L1 will fail due to incorrect gas estimation

Category	Severity	Client Response	Contributor
Logical	Medium	Acknowledged	0xffchain, lemonmon

### Code Reference

- code/mantle-v2/packages/contracts-bedrock/contracts/L1/L1CrossDomainMessenger.sol#L212-L221
- code/mantle-v2/packages/contracts-bedrock/contracts/L1/L1CrossDomainMessenger.sol#L213-L221

```

....
212: bool mntSuccess = true;
213: if (_mntValue!=0){
214: mntSuccess = IERC20(L1_MNT_ADDRESS).approve(_target, _mntValue);
215: }
216: xDomainMsgSender = _sender;
217: bool success = SafeCall.call(_target, gasleft() - RELAY_RESERVED_GAS, _ethValue, _message);
218: xDomainMsgSender = Constants.DEFAULT_L2_SENDER;
219: if (_mntValue!=0){
220: mntSuccess = IERC20(L1_MNT_ADDRESS).approve(_target, 0);
221: }
....

213: if (_mntValue!=0){
214: mntSuccess = IERC20(L1_MNT_ADDRESS).approve(_target, _mntValue);
215: }
216: xDomainMsgSender = _sender;
217: bool success = SafeCall.call(_target, gasleft() - RELAY_RESERVED_GAS, _ethValue, _message);
218: xDomainMsgSender = Constants.DEFAULT_L2_SENDER;
219: if (_mntValue!=0){
220: mntSuccess = IERC20(L1_MNT_ADDRESS).approve(_target, 0);
221: }

```

- code/mantle-v2/packages/contracts-bedrock/contracts/L2/L2CrossDomainMessenger.sol#L90
- code/mantle-v2/packages/contracts-bedrock/contracts/L2/L2CrossDomainMessenger.sol#L215-L224

```

....
90: baseGas(_message, _minGasLimit),
....

215: bool ethSuccess = true;
216: if (_ethValue != 0) {
217: ethSuccess = IERC20(Predeploys.BVM_ETH).approve(_target, _ethValue);
218: }
219: xDomainMsgSender = _sender;

```

## MNT-20:Reinitialization causes metering parameter to be reset

Category	Severity	Client Response	Contributor
Logical	Medium	Fixed	lemonmon

### Code Reference

- code/mantle-v2/packages/contracts-bedrock/contracts/L1/ResourceMetering.sol#L179-L185

```
....
179: function __ResourceMetering_init() internal onlyInitializing {
180: params = ResourceParams({
181: prevBaseFee: 1 gwei,
182: prevBoughtGas: 0,
183: prevBlockNum: uint64(block.number)
184: });
185: }
```

### Description

**lemonmon:** Reinitialization with an updated Constants.INITIALIZER version will reset of Metering ResourceParams. This means all parameters will reset to defaults, affecting how gas prices are calculated on the metered modifier.

```
function __ResourceMetering_init() internal onlyInitializing {
 params = ResourceParams({
 prevBaseFee: 1 gwei,
 prevBoughtGas: 0,
 prevBlockNum: uint64(block.number)
 });
}
```

### Recommendation

**lemonmon:** The gas parameters should only be updated in the case of a fresh initialization:

```
function __ResourceMetering_init() internal onlyInitializing {
+ if (params.prevBlockNum == 0) {
 params = ResourceParams({
 prevBaseFee: 1 gwei,
 prevBoughtGas: 0,
 prevBlockNum: uint64(block.number)
 });
+ }
}
```

See the pull request for more context: <https://github.com/ethereum-optimism/optimism/pull/8639/files>

Fixed. <https://github.com/mantlenetworkio/mantle-v2/pull/111/files>

# MNT-21: Overestimation could lead to permanent locking of funds on L1

Category	Severity	Client Response	Contributor
Logical	Medium	Acknowledged	0xffchain

## Code Reference

- code/mantle-v2/packages/contracts-bedrock/contracts/L1/OptimismPortal.sol#L468-L471

```
....
468: require(
469: _gasLimit >= minimumGasLimit(uint64(_data.length)),
470: "OptimismPortal: gas limit too small"
471:);
```

## Description

**0xffchain:** When supplying gas limit for transaction L1->L2, the bytes for message sent is called as such

```
require(
 _gasLimit >= minimumGasLimit(uint64(_data.length)),
 "OptimismPortal: gas limit too small"
);
```

With minimum gas limit function being:

```
function minimumGasLimit(uint64 _byteCount) public pure returns (uint64) {
 return _byteCount * 16 + 21000; // @audit-issue over estimates gas cause 0 should be 4 and
 not 16 and if data is big will make it impossible to send as will always reach gas limit of 30M
}
```

The challenge with this is that the zero bytes do not cost 16 gas, but rather cost 4 gas, meaning for each zero byte in the message, the user is charged 14 gas more...

## Recommendation

**0xffchain:** Take cognisance of the zero bytes in the message and estimate accordingly.

## Client Response

**0xffchain:** Acknowledged. currently the implementation is the same as the op stack and will keep it that way for now.

## MNT-22:Missing paranthesis causes fund loss

Category	Severity	Client Response	Contributor
Logical	Medium	Fixed	csanuragjain

### Code Reference

- code/mantle-v2/packages/contracts-bedrock/contracts/L1/OptimismPortal.sol#L431

```
....
431: if (success && l1mntSuccess == false && tx.origin == Constants.ESTIMATION_ADDRESS) {
```

### Description

**csanuragjain:** Messages can be created directly using `initiateWithdrawal` function in `l2tol1MessagePasser` which will then be finalized and executed by Optimism Portal. If the execution is running via `ESTIMATION_ADDRESS` then transaction revert if target calling fails, allowing user to retry execution later. But this logic is not working correctly meaning User can lose eth funds

Marking this as Medium since approval normally should never fail

### Steps

1. In case of message passed via CrossDomainMessenger, the messenger itself reverts the transaction of failed transaction send by `ESTIMATION_ADDRESS`
2. But if message was created directly using `initiateWithdrawal` function in `l2tol1MessagePasser`, then it relies on below check at OptimismPortal

```
if (success && l1mntSuccess == false && tx.origin == Constants.ESTIMATION_ADDRESS) {
 revert("OptimismPortal: withdrawal failed");
}
```

3. This was meant to trigger if success or l1mntSuccess is false and tx.origin == Constants.ESTIMATION\_ADDRESS
4. But due to missing parenthesis this is interpreted as success==true, l1mntSuccess == false, tx.origin == Constants.ESTIMATION\_ADDRESS which is incorrect
5. This means if target calling was unsuccessful then revert wont happen and transaction will be marked completed (considering l1mntSuccess failed and tx.origin is Constants.ESTIMATION\_ADDRESS)
6. This means User lose all eth linked to that target call

### Recommendation

**csanuragjain:** Change the check like below:

Fixed. <https://github.com/mantlenetworkio/mantle-v2/pull/95/files>

## MNT-23:Lack of distribution of BVM\_ETH for the call of BVM\_ETH.approve()

Category	Severity	Client Response	Contributor
Logical	Medium	Acknowledged	Yaodao

### Code Reference

- code/mantle-v2/packages/contracts-bedrock/contracts/L2/BVM\_ETH.sol#L24-30

```
....

24: function mint(address _to, uint256 _amount)
25: public
26: virtual
27: override
28: {
29: revert("BVM_ETH: mint is disabled by normal contract calling. BVM_ETH mint can only be t
triggered in deposit transaction execution, similar to MNT mint on L2.");
30: }
```

- code/mantle-v2/packages/contracts-bedrock/contracts/universal/OptimismMintableERC20.sol#L60-68

```
....

60: constructor(
61: address _bridge,
62: address _remoteToken,
63: string memory _name,
64: string memory _symbol
65:) ERC20(_name, _symbol) Semver(1, 0, 0) {
66: REMOTE_TOKEN = _remoteToken;
67: BRIDGE = _bridge;
68: }
```

- code/op-geth/core/state\_transition.go#L667-673
- code/op-geth/core/state\_transition.go#L675-681

```
....

667
```

```

: func (st *StateTransition) addBVMETHBalance(ethValue *big.Int) {
668: key := getBVMETHBalanceKey(*st.msg.To)
669: value := st.state.GetState(BVM_ETH_ADDR, key)
670: bal := value.Big()
671: bal = bal.Add(bal, ethValue)
672: st.state.SetState(BVM_ETH_ADDR, key, common.BigToHash(bal))
673: }

....

675: func (st *StateTransition) addBVMETHTotalSupply(ethValue *big.Int) {
676: key := getBVMETHTotalSupplyKey()
677: value := st.state.GetState(BVM_ETH_ADDR, key)
678: bal := value.Big()
679: bal = bal.Add(bal, ethValue)
680: st.state.SetState(BVM_ETH_ADDR, key, common.BigToHash(bal))
681: }

```

## Description

**Yaodao:** The contract ``BVM_ETH`` is the ERC20 token contract of the ETH on L2 network(Mantle). First, there is no initial ``BVM_ETH`` mint and distribution in the following ``constructor()`` function.

```

constructor(
 address _bridge,
 address _remoteToken,
 string memory _name,
 string memory _symbol
) ERC20(_name, _symbol) Semver(1, 0, 0) {
 REMOTE_TOKEN = _remoteToken;
 BRIDGE = _bridge;
}

```

Furthermore, the ``mint()`` function is overridden and the ``mint()`` function will always revert, and no new tokens can be minted in the solidity contract.

```

function mint(address _to, uint256 _amount)
 public
 virtual
 override
{
 revert("BVM_ETH: mint is disabled by normal contract calling. BVM_ETH mint can only be triggered in deposit transaction execution, similar to MNT mint on L2.");
}

```

As a result, there is no ``BVM_ETH`` in circulation and it cannot be minted in solidity.

In the design of the protocol, the following functions ``addBVMETHBalance()`` and ``addBVMETHTotalSupply()`` in the op-geth are used to mint the ``BVM_ETH`` directly at the Golang level, which updates the database directly. However, solidity contracts store their state variables on the blockchain to make the data persistent and difficult to tamper with. The design of the protocol for the ``mint()`` of the ``BVM_ETH`` against the design of the solidity contracts.

Recommend adding privilege control and mint at the solidity contract level instead of at the Golang level.

## Client Response

**Yaodao:** Acknowledged. will not fix as too much complexity, this does not impose security risk

## MNT-24:Lack of check `st.msg.To` in function `TransitionDb()`

Category	Severity	Client Response	Contributor
Logical	Medium	Fixed	biakia

### Code Reference

- code/op-geth/core/state\_transition.go#L436-L446

```
....
436: func (st *StateTransition) TransitionDb() (*ExecutionResult, error) {
437: if mint := st.msg.Mint; mint != nil {
438: st.state.AddBalance(st.msg.From, mint)
439: }
440: //add eth value
441: if ethValue := st.msg.ETHValue; ethValue != nil && ethValue.Cmp(big.NewInt(0)) != 0 {
442: st.addBVMETHBalance(ethValue)
443: st.addBVMETHTotalSupply(ethValue)
444: st.generateBVMETHMintEvent(*st.msg.To, ethValue)
445: }
446: snap := st.state.Snapshot()
```

### Description

**biakia:** When a transaction is a contract creation transaction, the value of the `st.msg.To` will be `nil`. In function `TransitionDb()`, there is no check on whether the `st.msg.To` is `nil`:

```
if ethValue := st.msg.ETHValue; ethValue != nil && ethValue.Cmp(big.NewInt(0)) != 0 {
 st.addBVMETHBalance(ethValue)
 st.addBVMETHTotalSupply(ethValue)
 st.generateBVMETHMintEvent(*st.msg.To, ethValue)
}
```

In function `addBVMETHBalance`, the function `getBVMETHBalanceKey` will be called with `*st.msg.To` as the input param:

```
func (st *StateTransition) addBVMETHBalance(ethValue *big.Int) {
 key := getBVMETHBalanceKey(*st.msg.To)
 value := st.state.GetState(BVM_ETH_ADDR, key)
 bal := value.Big()
 bal = bal.Add(bal, ethValue)
 st.state.SetState(BVM_ETH_ADDR, key, common.BigToHash(bal))
}
```

In function `getBVMETHBalanceKey`, it will call `addr.Bytes()`:

```
func
```

```
getBVMETHBalanceKey(addr common.Address) common.Hash {
 position := common.Big0
 hasher := sha3.NewLegacyKeccak256()
 hasher.Write(common.LeftPadBytes(addr.Bytes(), 32))
 hasher.Write(common.LeftPadBytes(position.Bytes(), 32))
 digest := hasher.Sum(nil)
 return common.BytesToHash(digest)
}
```

if `st.msg.To` is `nil`, the call of `addr.Bytes()` will panic.

## Recommendation

biakia: Consider adding a check in `TransitionDb()`:

```
if ethValue := st.msg.ETHValue; ethValue != nil && ethValue.Cmp(big.NewInt(0)) != 0 && st.msg.To != nil {
 st.addBVMETHBalance(ethValue)
 st.addBVMETHTotalSupply(ethValue)
 st.generateBVMETHMintEvent(*st.msg.To, ethValue)
}
```

## Client Response

biakia: Fixed. Already fixed in <https://github.com/mantlenetworkio/op-geth/pull/32/files>

# MNT-25:Insufficient Validation of Sponsor Balance in Meta Transactions

Category	Severity	Client Response	Contributor
Logical	Medium	Fixed	BradMoonUESTC

## Code Reference

- code/op-geth/core/txpool/txpool.go#L1477-L1507

```
....
1477: func (pool *TxPool) validateMetaTxList(list *list) ([]*types.Transaction, *big.Int) {
1478: currHeight := pool.chain.CurrentBlock().Number.Uint64()
1479:
1480: var invalidMetaTxs []*types.Transaction
1481: sponsorCostSum := big.NewInt(0)
1482: for _, tx := range list.txs.Flatten() {
1483: metaTxParams, err := types.DecodeAndVerifyMetaTxParams(tx)
1484: if err != nil {
1485: invalidMetaTxs = append(invalidMetaTxs, tx)
1486: continue
1487: }
1488: if metaTxParams == nil {
1489: continue
1490: }
1491: if metaTxParams.ExpireHeight < currHeight {
1492: invalidMetaTxs = append(invalidMetaTxs, tx)
1493: }
1494: txGasCost := new(big.Int).Mul(tx.GasPrice(), new(big.Int).SetUint64(tx.Gas()))
1495: l1Cost := pool.l1CostFn(tx.RollupDataGas(), tx.IsDepositTx(), tx.To())
1496: if l1Cost != nil {
1497: txGasCost = new(big.Int).Add(txGasCost, l1Cost) // gas fee sponsor must sponsor additional l1Cost fee
1498: }
1499: sponsorAmount, _ := types.CalculateSponsorPercentAmount(metaTxParams, txGasCost)
1500: if pool.currentState.GetBalance(metaTxParams.GasFeeSponsor).Cmp(sponsorAmount) >= 0 {
1501: sponsorCostSum = new(big.Int).Add(sponsorCostSum, sponsorAmount)
1502: } else {
1503: invalidMetaTxs = append(invalidMetaTxs, tx)
1504: }
1505: }
1506: return invalidMetaTxs, sponsorCostSum
1507: }
```

## Description

**BradMoonUESTC:** In the `validateMetaTxList` function of the provided Go code, which is part of the transaction pool management for processing meta transactions. This function is responsible for validating a list of meta transactions, specifically checking if the gas fee sponsors (identified by `GasFeeSponsor` in meta transaction parameters) have sufficient balances to sponsor the gas fees of these transactions.

The vulnerability arises due to the function's logic that accumulates the required sponsor amount (`sponsorCostSum`) for all transactions sponsored by the same address without deducting the sponsored amount from the sponsor's

```
pool.currentState.GetBalance(metaTxParams.GasFeeSponsor).Cmp(sponsorAmount) >= 0 {
 sponsorCostSum = new(big.Int).Add(sponsorCostSum, sponsorAmount)
} else {
 invalidMetaTxs = append(invalidMetaTxs, tx)
}
```

If different transactions (`tx`) share the same `GasFeeSponsor`, it means that the check `pool.currentState.GetBalance(metaTxParams.GasFeeSponsor).Cmp(sponsorAmount) >= 0` can only ensure that the `GasFeeSponsor`'s balance is greater than `sponsorAmount` for the current transaction. It cannot guarantee that the `GasFeeSponsor`'s balance is greater than the sum of `sponsorAmount` that may accumulate in the loop.

## Recommendation

**BradMoonUESTC:** To mitigate this vulnerability and prevent exploitation, it is recommended to adjust the `validateMetaTxList` function to accurately track and deduct the sponsor amounts for each transaction from the sponsor's balance before proceeding with the next transaction validation. This ensures that the sponsor's balance is genuinely sufficient to cover all the transactions they intend to sponsor.

A proposed code adjustment is as follows:

```
func
```

```

(pool *TxPool) validateMetaTxList(list *list) ([]*types.Transaction, *big.Int) {
 currHeight := pool.chain.CurrentBlock().Number.Uint64()
 var invalidMetaTxs []*types.Transaction
 sponsorBalances := make(map[common.Address]*big.Int)

 for _, tx := range list.txs.Flatten() {
 metaTxParams, err := types.DecodeAndVerifyMetaTxParams(tx)
 if err != nil || metaTxParams == nil {
 invalidMetaTxs = append(invalidMetaTxs, tx)
 continue
 }

 // Calculate required gas cost and L1 cost for the transaction
 txGasCost := new(big.Int).Mul(tx.GasPrice(), new(big.Int).SetUint64(tx.Gas()))
 l1Cost := pool.l1CostFn(tx.RollupDataGas(), tx.IsDepositTx(), tx.To())
 if l1Cost != nil {
 txGasCost.Add(txGasCost, l1Cost)
 }
 sponsorAmount, _ := types.CalculateSponsorPercentAmount(metaTxParams, txGasCost)

 // Check and update the sponsor's balance
 if _, ok := sponsorBalances[metaTxParams.GasFeeSponsor]; !ok {
 sponsorBalances[metaTxParams.GasFeeSponsor] = pool.currentState.GetBalance(metaTxParams.GasFeeSponsor)
 }

 if sponsorBalances[metaTxParams.GasFeeSponsor].Cmp(sponsorAmount) < 0 {
 invalidMetaTxs = append(invalidMetaTxs, tx)
 } else {
 // Deduct the sponsor amount from the sponsor's tracked balance
 sponsorBalances[metaTxParams.GasFeeSponsor].Sub(sponsorBalances[metaTxParams.GasFeeSponsor], sponsorAmount)
 }
 }

 // Convert map values to a single sum of sponsor costs
 sponsorCostSum := big.NewInt(0)
 for _, balance := range sponsorBalances {
 sponsorCostSum.Add(sponsorCostSum, balance)
 }

 return invalidMetaTxs, sponsorCostSum
}

```

This revised approach ensures that each sponsor's balance is checked and deducted appropriately for each transaction they sponsor, thereby preventing the exploitation of the system through insufficient balance validation for meta transactions.

Fixed. <https://github.com/mantlenetworkio/op-geth/pull/43>

## MNT-26: Incorrect return value in function `getTokenPricesFromUniswap()`

Category	Severity	Client Response	Contributor
Logical	Medium	Acknowledged	Yaodao

### Code Reference

- code/mantle-v2/gas-oracle/tokenratio/tokenratio\_dex.go#L57-73

```
....
57: func (c *Client) getTokenPricesFromUniswap() (float64, float64) {
58:
59: eth2mntPrice, err := c.getTokenPriceFromUniswap(c.uniswapQuoterClient.ethAddress,
60: c.uniswapQuoterClient.mntAddress, c.uniswapQuoterClient.mntDecimals)
61: if err != nil {
62: log.Warn("get token prices from dex", "query eth/mnt error", err)
63: return 0, 0
64: }
65: eth2usdtPrice, err := c.getTokenPriceFromUniswap(c.uniswapQuoterClient.ethAddress,
66: c.uniswapQuoterClient.usdtAddress, c.uniswapQuoterClient.usdtDecimals)
67: if err != nil {
68: log.Warn("get token prices from dex", "query eth/usdt error", err)
69: return 0, eth2mntPrice
70: }
71:
72: return eth2usdtPrice / eth2mntPrice, eth2usdtPrice
73: }
```

- code/mantle-v2/gas-oracle/tokenratio/tokenratio.go#L125

```
....
125: mntPrice1, ethPrice1 := c.getTokenPricesFromUniswap()
```

### Description

**Yaodao:** According to the following codes, the function `getTokenPricesFromUniswap()` is used to return the price of `mnt` and `ETH`.

```
mntPrice1, ethPrice1 := c.getTokenPricesFromUniswap()
```

```
func (c
```

```
*Client) getTokenPricesFromUniswap() (float64, float64) {

 eth2mntPrice, err := c.getTokenPriceFromUniswap(c.uniswapQuoterClient.ethAddress,
 c.uniswapQuoterClient.mntAddress, c.uniswapQuoterClient.mntDecimals)
 if err != nil {
 log.Warn("get token prices from dex", "query eth/mnt error", err)
 return 0, 0
 }
 eth2usdtPrice, err := c.getTokenPriceFromUniswap(c.uniswapQuoterClient.ethAddress,
 c.uniswapQuoterClient.usdtAddress, c.uniswapQuoterClient.usdtDecimals)
 if err != nil {
 log.Warn("get token prices from dex", "query eth/usdt error", err)
 return 0, eth2mntPrice
 }

 return eth2usdtPrice / eth2mntPrice, eth2usdtPrice
}
```

However, for the logic of L65-70, the `0` and `eth2mntPrice` is return. The `eth2mntPrice` is returned for the price of `ETH`, which is incorrect.

## Recommendation

**Yaodao:** Recommend fixing the logic and returning the correct value.

## Client Response

**Yaodao:** Acknowledged. Will fix later

## MNT-27: Incorrect calculation for the cost of the replaced transaction in `validateTx`

Category	Severity	Client Response	Contributor
Logical	Medium	Acknowledged	biakia

### Code Reference

- code/op-geth/core/txpool/txpool.go#L723-L741

```
....

723: // Verify that replacing transactions will not result in overdraft
724: list := pool.pending[from]
725: if list != nil { // Sender already has pending txs
726: _, sponsorCostSum := pool.validateMetaTxList(list)
727: userBalance = new(big.Int).Add(userBalance, sponsorCostSum)
728: sum := new(big.Int).Add(cost, list.totalcost)
729: if repl := list.txs.Get(tx.Nonce()); repl != nil {
730: // Deduct the cost of a transaction replaced by this
731: replL1Cost := repl.L1Cost()
732: if l1Cost := pool.l1CostFn(tx.RollupDataGas(), tx.IsDepositTx(), tx.To()); l1Cost !=
nil { // add rollup cost
733: replL1Cost = replL1Cost.Add(cost, l1Cost)
734: }
735: sum.Sub(sum, replL1Cost)
736: }
737: if userBalance.Cmp(sum) < 0 {
738: log.Trace("Replacing transactions would overdraft", "sender", from, "balance", userB
alance, "required", sum)
739: return ErrOverdraft
740: }
741: }
```

### Description

**biakia:** In `txpool.go`, the function `validateTx` is used to check whether a transaction is valid according to the consensus rules and adheres to some heuristic limits of the local node (price and size). When the tx to be validated already exists, it means a replaced transaction occurs. It should verify that replacing transactions will not result in overdraft:

```
list := pool.pending[from]
```

```

if list != nil { // Sender already has pending txs
 _, sponsorCostSum := pool.validateMetaTxList(list)
 userBalance = new(big.Int).Add(userBalance, sponsorCostSum)
 sum := new(big.Int).Add(cost, list.totalcost)
 if repl := list.txs.Get(tx.Nonce()); repl != nil {
 // Deduct the cost of a transaction replaced by this
 replL1Cost := repl.Cost()
 if l1Cost := pool.l1CostFn(tx.RollupDataGas(), tx.IsDepositTx(), tx.To()); l1Cost != nil {
 replL1Cost = replL1Cost.Add(cost, l1Cost)
 }
 sum.Sub(sum, replL1Cost)
 }
 if userBalance.Cmp(sum) < 0 {
 log.Trace("Replacing transactions would overdraft", "sender", from, "balance", userBalance, "required", sum)
 return ErrOverdraft
 }
}

```

It will get the tx to be replaced, which is the variable `repl`, and then deduct the cost from the `sum`:

```

replL1Cost := repl.Cost()
if l1Cost := pool.l1CostFn(tx.RollupDataGas(), tx.IsDepositTx(), tx.To()); l1Cost != nil {
 replL1Cost = replL1Cost.Add(cost, l1Cost)
}
sum.Sub(sum, replL1Cost)

```

The cost of a tx includes transaction cost and the L1 cost. The issue here is that when calculating the L1 cost, it will use the `tx` instead of `repl`:

```
pool.l1CostFn(tx.RollupDataGas(), tx.IsDepositTx(), tx.To())
```

The `tx` is the new transaction and the `repl` is the old transaction. It is incorrect to use the data from `tx` to calculate the L1 cost of `repl`.

As per the document from mantle, the code is forked from optimism's op-geth v1.101105.3. However, in the latest version of op-geth, the calculation for `l1Cost` is based on the existed `tx` now (<https://github.com/ethereum-optimism/op-geth/blob/v1.101308.1-rc.1/core/txpool/legacypool/legacypool.go#L658-L671>):

```
if
```

```
list := pool.pending[addr]; list != nil {
 if tx := list.txs.Get(nonce); tx != nil {
 cost := tx.Cost()
 if pool.l1CostFn != nil {
 if l1Cost := pool.l1CostFn(tx.RollupCostData()); l1Cost != nil { // add ro
llup cost
 cost = cost.Add(cost, l1Cost)
 }
 }
 return cost
 }
}
```

## Recommendation

biakia: Consider using `repl` to calculate the L1 cost:

```
if repl := list.txs.Get(tx.Nonce()); repl != nil {
 // Deduct the cost of a transaction replaced by this
 replL1Cost := repl.Cost()
 if l1Cost := pool.l1CostFn(repl.RollupDataGas(), repl.IsDepositTx(), repl.To()); l1Cos
t != nil { // add rollup cost
 replL1Cost = replL1Cost.Add(cost, l1Cost)
 }
 sum.Sub(sum, replL1Cost)
}
```

## Client Response

biakia: Acknowledged. Will fix later

# MNT-28:Incorrect Calculation of Median in Non-Zero Element Filtering Logic

Category	Severity	Client Response	Contributor
Logical	Medium	Fixed	BradMoonUESTC

## Code Reference

- code/mantle-v2/gas-oracle/tokenratio/tokenratio.go#L209-L221

```
....
209: func getMedian(nums []float64) float64 {
210: nonZeros := make([]float64, 0)
211: for _, num := range nums {
212: if num != 0 {
213: nonZeros = append(nonZeros, num)
214: }
215: }
216: sort.Float64s(nonZeros)
217: if len(nonZeros) == 0 {
218: return 0
219: }
220: return nonZeros[len(nonZeros)/2]
221: }
```

## Description

**BradMoonUESTC:** The vulnerability exists in the `getMedian` function, which is designed to calculate the median of a slice of `float64` numbers, excluding zero values. The function first filters out zero values and then calculates the median of the remaining non-zero values. However, the logic used to calculate the median after filtering is flawed, specifically for even-numbered lists. The function returns the middle element of the sorted slice without considering that for an even number of elements, the median should be the average of the two middle numbers. This oversight can lead to incorrect median calculations, impacting functionalities relying on this operation for decision-making or processing.

For example, consider a slice `[]float64{1, 2, 3, 4}`. The correct median should be  $(2 + 3) / 2 = 2.5$ , but due to the error in logic, the function would incorrectly return `3`, which is merely the middle element of the array after sorting.

This incorrect calculation can be exploited in scenarios where precise median values are crucial for further computations or decisions, potentially leading to erroneous outcomes that could benefit an adversary, especially in financial or data analytics applications where median values influence transaction logic or data interpretation.

## Recommendation

**BradMoonUESTC:** To mitigate this vulnerability, the `getMedian` function should be modified to correctly handle the calculation of the median for both even and odd numbers of elements in the non-zero filtered slice. Below is the corrected code with an updated logic for calculating the median:

```
package
```

```
main

import (
 "sort"
)

// Corrected implementation of getMedian function
func getMedian(nums []float64)float64 {
 nonZeros := make([]float64, 0)
 for _, num := range nums {
 if num != 0 {
 nonZeros = append(nonZeros, num)
 }
 }
 sort.Float64s(nonZeros)
 n := len(nonZeros)
 if n == 0 {
 return 0
 }
 if n%2 == 0 {
 return (nonZeros[n/2-1] + nonZeros[n/2]) / 2
 }
 return nonZeros[n/2]
}
```

## Client Response

BradMoonUESTC: Fixed. <https://github.com/mantlenetworkio/mantle-v2/pull/113/files>

## MNT-29: If the token's decimal is not 18, the function `getTokenPriceFromUniswap` will return the wrong price

Category	Severity	Client Response	Contributor
Logical	Medium	Acknowledged	biakia

### Code Reference

- code/mantle-v2/gas-oracle/tokenratio/tokenratio\_dex.go#L76-L92

```
....

76: func (c *Client) getTokenPriceFromUniswap(fromToken, toToken common.Address, decimals *big.Float)
 t) (float64, error) {
77: fee := big.NewInt(3000)
78: fromAmount := floatStringToBigInt("1.00", 18)
79: sqrtPriceLimitX96 := big.NewInt(0)
80:
81: var out []interface{}
82: rawCaller := &bindings.Uniswapv3QuoterRaw{Contract: c.uniswapQuoterClient.uniswapV3Quoter}
83: err := rawCaller.Call(nil, &out, "quoteExactInputSingle", fromToken, toToken,
84: fee, fromAmount, sqrtPriceLimitX96)
85: if err != nil {
86: return 0, err
87: }
88:
89: resultBigFloat := new(big.Float).SetInt(out[0].(*big.Int))
90: result, _ := new(big.Float).Quo(resultBigFloat, decimals).Float64()
91: return result, nil
92: }
```

### Description

biakia: The function `getTokenPriceFromUniswap` is used to calculate token price by executing swapping `from_token` to `to_token`. It will use 18 as the decimal of the `from_token`:

```
fee := big.NewInt(3000)
fromAmount := floatStringToBigInt("1.00", 18)
sqrtPriceLimitX96 := big.NewInt(0)
```

The `from_token` now is the `eth` token:

```
eth2mntPrice, err := c.getTokenPriceFromUniswap(c.uniswapQuoterClient.ethAddress,
 c.uniswapQuoterClient.mntAddress, c.uniswapQuoterClient.mntDecimals)
if err != nil {
 log.Warn("get token prices from dex", "query eth/mnt error", err)
 return 0, 0
}
eth2usdtPrice, err := c.getTokenPriceFromUniswap(c.uniswapQuoterClient.ethAddress,
 c.uniswapQuoterClient.usdtAddress, c.uniswapQuoterClient.usdtDecimals)
```

Since eth's decimal is 18, there is no problem with the calculation here. However, if the `from_token` is not eth, then its decimal may not be 18, and the calculated price will be wrong.

## Recommendation

**biakia:** Consider passing the token's decimal as a parameter instead of using the default 18:

```
func (c *Client) getTokenPriceFromUniswap(fromToken, fromDecimals int, toToken common.Address, decimals *big.Float) (float64, error) {
 fee := big.NewInt(3000)
 fromAmount := floatStringToBigInt("1.00", fromDecimals)
 sqrtPriceLimitX96 := big.NewInt(0)
```

## Client Response

**biakia:** Acknowledged. will fix later

# MNT-30: Exploitation of Fixed `fromAmount` in Price Queries on Low Liquidity Pools

Category	Severity	Client Response	Contributor
Logical	Medium	Acknowledged	BradMoonUESTC

## Code Reference

- code/mantle-v2/gas-oracle/tokenratio/tokenratio\_dex.go#L78

```
....

78: fromAmount := floatStringToBigInt("1.00", 18)
```

## Description

**BradMoonUESTC:** The identified vulnerability lies within the mechanism of querying token prices from Uniswap V3 through the `quoteExactInputSingle` function. The code snippet uses a fixed `fromAmount` of 1 token (considering 18 decimal places) for price queries between any given pair of tokens (`fromToken` to `toToken`). This fixed amount disregards the liquidity depth of the pool, which can significantly affect the accuracy of the quoted price, especially in pools with low liquidity.

In Uniswap V3, the price of a token can be highly sensitive to the size of a trade in relation to the pool's liquidity. A fixed `fromAmount` does not account for this sensitivity, leading to potential price manipulation in low liquidity pools. An attacker could exploit this by either adding or removing a small amount of liquidity, or by executing trades that significantly alter the price due to the pool's low depth, and subsequently use the manipulated price for profitable trades or to exploit other contracts relying on these price quotes.

## Recommendation

**BradMoonUESTC:** To mitigate this vulnerability, it is recommended to dynamically adjust the `fromAmount` based on the liquidity of the pool. This approach requires fetching the liquidity information of the involved pool and adjusting the `fromAmount` to a value that minimizes the impact on the quoted price. Additionally, considering prices from multiple sources or using a price aggregation service can enhance the accuracy and reliability of the price information.

## Client Response

**BradMoonUESTC:** Acknowledged. will fix later

## MNT-31:EigenDA/MantleDA is not checked validated for liveness

Category	Severity	Client Response	Contributor
Logical	Medium	Acknowledged	Oxffchain

### Code Reference

- code/mantle-v2/op-batcher/batcher/driver.go#L63-L82

```
....
63: l2Client, err := opclient.DialEthClientWithTimeout(ctx, cfg.L2EthRpc, opclient.DefaultDialTimeou
t)
64: if err != nil {
65: return nil, err
66: }
67:
68: rollupClient, err := opclient.DialRollupClientWithTimeout(ctx, cfg.RollupRpc, opclient.Defau
ltDialTimeout)
69: if err != nil {
70: return nil, err
71: }
72:
73: rcfg, err := rollupClient.RollupConfig(ctx)
74: if err != nil {
75: return nil, fmt.Errorf("querying rollup config: %w", err)
76: }
77:
78: txManager, err := txmgr.NewSimpleTxManager("batcher", l, m, cfg.TxMgrConfig)
79: if err != nil {
80: return nil, err
81: }
```

### Description

**Oxffchain:** The EigenDA/MantleDA is not validated for liveness like it is done for other services/clients/components of the system.

```
l1Client, err :
```

```

= opclient.DialEthClientWithTimeout(ctx, cfg.L1EthRpc, opclient.DefaultDialTimeout)
 if err != nil {
 return nil, err
 }

 l2Client, err := opclient.DialEthClientWithTimeout(ctx, cfg.L2EthRpc, opclient.DefaultDialTime
out)
 if err != nil {
 return nil, err
 }

 rollupClient, err := opclient.DialRollupClientWithTimeout(ctx, cfg.RollupRpc, opclient.Default
DialTimeout)
 if err != nil {
 return nil, err
 }

```

As it is seen above when `NewBatchSubmitterFromCLIConfig` is called in `batch_submitter`, it takes account of checking the liveness of the three components that constitute the mantle V1 chain: EthDA, Sequencer and RollupRPC. This should be all fine if the batcher is interested in Just submitting transactions straight to ETHDA, but that is not the case as required in MantleV2, as V2 introduces a another external component, which is EigenDA. It should also check for the liveness of these forth service as the service as stated by the EigenDA team could possibly have outages: **EigenDA is new software, and we don't expect protocols to trust it blindly. If, for whatever reason, EigenDA goes down.....**

This is also important as the service to which the sequencer connects to directly, the disperse service, is a centralized service that is run by EigenDA team. All request to EigenDA has to pass through this single service before the request is dispersed to nodes in the quorum set. This introduces a single point of failure that is critical to the working of MantleDA.

## Recommendation

**Oxiffchain:** The MantleDA service should be checked for liveness when starting the `batch_submitter` and if it is not live, then it should exit just like it does for the other components, as this component is as crucial as the others when `rcfg.MantleDaSwitch` is true.

```

if rcfg.MantleDaSwitch{
 // Code to check for liveness
}

```

## Client Response

**Oxiffchain:** Acknowledged. only support MantleDA for now. every time interacte with EigenDA a new connection will be established, and will raise error and retry if unsuccessful. will not fix or add liveness check for now

## MNT-32:ERC 20 approve is assigned twice and validated once.

Category	Severity	Client Response	Contributor
Logical	Medium	Fixed	0xffchain

### Code Reference

- code/mantle-v2/packages/contracts-bedrock/contracts/L1/L1CrossDomainMessenger.sol#L213-L221

```
....
213: if (_mntValue!=0){
214: mntSuccess = IERC20(L1_MNT_ADDRESS).approve(_target, _mntValue);
215: }
216: xDomainMsgSender = _sender;
217: bool success = SafeCall.call(_target, gasleft() - RELAY_RESERVED_GAS, _ethValue, _message);
218: xDomainMsgSender = Constants.DEFAULT_L2_SENDER;
219: if (_mntValue!=0){
220: mntSuccess = IERC20(L1_MNT_ADDRESS).approve(_target, 0);
221: }
```

### Description

**0xffchain:** The approve function of the ERC20 token is called twice, first time the approve is called with a amount > 0 and second time to approve an exactly zero amount, at each time a bool response returned. But it is only validated once, meaning the first response is overridden by the second response, without a Validation if the first was a success response.

### Recommendation

**0xffchain:** Check both times that the response is successful (true)...

### Client Response

**0xffchain:** Fixed. <https://github.com/mantlenetworkio/mantle-v2/pull/98/files>

## MNT-33:EOAs depositing ETH through L1CrossDomainMessenger will lose their funds.

Category	Severity	Client Response	Contributor
Logical	Medium	Fixed	SerSomeone

### Code Reference

- code/mantle-v2/packages/contracts-bedrock/contracts/L1/L1CrossDomainMessenger.sol#L94

```
....
94: msg.value,
```

### Description

**SerSomeone:** When sending a message from L1->L2 through the `L1CrossDomainMessenger`, the funds attached through `msg.value` will be minted into `BVM_ETH` on L2 to the `L2CrossDomainMessenger`. Then `relayMessage` approves the exact amount so the targets can pull the funds through `transferFrom`.

```
function relayMessage(
 uint256 _nonce,
 address _sender,
 address _target,
 uint256 _mntValue,
 uint256 _ethValue,
 uint256 _minGasLimit,
 bytes calldata _message
) external payable override {

 bool ethSuccess = true;
 if (_ethValue != 0) {
 ethSuccess = IERC20(Predeploys.BVM_ETH).approve(_target, _ethValue);
 }
 xDomainMsgSender = _sender;
 bool success = SafeCall.call(_target, gasleft() - RELAY_RESERVED_GAS, _mntValue, _message);
 xDomainMsgSender = Constants.DEFAULT_L2_SENDER;
 if (_ethValue != 0) {
 ethSuccess = IERC20(Predeploys.BVM_ETH).approve(_target, 0);
 }

}
```

The mechanism is in pull and not push because `relayMessage` does not revert on failed transactions, it stored them for replayability - but ERC funds sent cannot be retrieved. Since `targets` can only pull the funds when called

s therefore **CANNOT** be `E0A`s`.

Any EOA sending ETH funds through `L1CrossDomainMessenger`s`sendMessage`` function to itself will lose the funds. There is no protection for this in `sendMessage``.

```
function sendMessage(
 uint256 _mntAmount,
 address _target,
 bytes calldata _message,
 uint32 _minGasLimit
) external payable override {

 _sendMessage(
 _mntAmount,
 OTHER_MESSENGER,
 baseGas(_message, _minGasLimit),
 abi.encodeWithSelector(
 L2CrossDomainMessenger.relayMessage.selector,
 messageNonce(),
 msg.sender,
 _target,
 _mntAmount,
 msg.value,
 _minGasLimit,
 _message
)
);
}
```

## Recommendation

SerSomeone: Consider adding a check in `L1CrossDomainMessenger`` to see revert if `target==tx.origin && msg.value > 0``

## Client Response

SerSomeone: Fixed. <https://github.com/mantlenetworkio/mantle-v2/pull/110>

## MNT-34: Due to incorrectly calculation of the user balance, the `txpool.go` does not properly remove invalid transactions

Category	Severity	Client Response	Contributor
Logical	Medium	Fixed	lemonmon

### Code Reference

- code/op-geth/core/txpool/txpool.go#L713

```
....
```

```
713: userBalance = new(big.Int).Add(selfBalance, sponsorBalance)
```

### Description

**lemonmon:** As stated in the other report "The `txpool.go` does not properly remove invalid transactions by including expired sponsor payment to the sponsor cost sum", validating transactions and removing invalid transaction is important.

However, in the `core/txpool/txpool.go::validatTx``, the function calculates the user balance incorrectly, resulting in validating otherwise invalid transaction.

[https://github.com/Secure3Audit/code\\_Mantle\\_V2/blob/97bb2aad421c306241ca1681dd3c68b6dfc3355b/code/op-geth/core/txpool/txpool.go#L713](https://github.com/Secure3Audit/code_Mantle_V2/blob/97bb2aad421c306241ca1681dd3c68b6dfc3355b/code/op-geth/core/txpool/txpool.go#L713)

In the line 713, the `userBalance`` is calculated by adding up the balance of the `from`` and the balance of the `sponsor``. But the sponsor should only pay the portion of gas and should not be included as `userBalance``:

```
// core/txpool/txpool.go::validatTx
 sponsorAmount, selfPayAmount := types.CalculateSponsorPercentAmount(metaTxParams, txGasCost)

 selfPayAmount = new(big.Int).Add(selfPayAmount, tx.Value())

 sponsorBalance := pool.currentState.GetBalance(metaTxParams.GasFeeSponsor)
 if sponsorBalance.Cmp(sponsorAmount) < 0 {
 return types.ErrSponsorBalanceNotEnough
 }
 selfBalance := pool.currentState.GetBalance(from)
 if selfBalance.Cmp(selfPayAmount) < 0 {
 return core.ErrInsufficientFunds
 }
 userBalance = new(big.Int).Add(selfBalance, sponsorBalance)
```

### Recommendation

**lemonmon:** recommendation

It should add the `sponsorAmount`` instead of `sponsorBalance``

```
sponsorAmount, selfPayAmount := types.CalculateSponsorPercentAmount(metaTxParams, txGasCost)
selfPayAmount = new(big.Int).Add(selfPayAmount, tx.Value())

sponsorBalance := pool.currentState.GetBalance(metaTxParams.GasFeeSponsor)
if sponsorBalance.Cmp(sponsorAmount) < 0 {
 return types.ErrSponsorBalanceNotEnough
}
selfBalance := pool.currentState.GetBalance(from)
if selfBalance.Cmp(selfPayAmount) < 0 {
 return core.ErrInsufficientFunds
}
- userBalance = new(big.Int).Add(selfBalance, sponsorBalance)
+ userBalance = new(big.Int).Add(selfBalance, sponsorAmount)
```

## Client Response

lemonmon: Fixed. <https://github.com/mantlenetworkio/op-geth/pull/45/files>

## MNT-35: Could not estimate the exact gas cost in `finalizeWithdrawalTransaction` when `mntValue` is 0

Category	Severity	Client Response	Contributor
DOS	Medium	Fixed	lemonmon, Hacker007

### Code Reference

- code/mantle-v2/packages/contracts-bedrock/contracts/L1/OptimismPortal.sol#L416-L419
- code/mantle-v2/packages/contracts-bedrock/contracts/L1/OptimismPortal.sol#L431-L434

```
....
416: bool l1mntSuccess = false;
417: if (_tx.mntValue>0){
418: l1mntSuccess = IERC20(L1_MNT_ADDRESS).transfer(_tx.target, _tx.mntValue);
419: }

....

431: if (success && l1mntSuccess == false && tx.origin == Constants.ESTIMATION_ADDRESS) {
432: revert("OptimismPortal: withdrawal failed");
433: }
434: }
```

### Description

**lemonmon:** To bridge between mainnet and mantle without using `CrossDomainMessenger``, it is very important to estimate the gas needed for the call. If the call to the target fails due to out of gas error, the user would not be able to replay the transaction again. In the case of L2 to L1 transaction, if the user specifies too less gas as minimum gas limit (than the real call), a malicious attacker can finalize the transaction via `OptimismPortal::finalizeWithdrawalTransaction`` with the user specified minimum gas limit. If the user underestimated the gas limit, the call by the attacker will result in out of gas error, and the user's fund will be locked.

There is a mechanism to estimate the gas usage by calling on the OptimismPortal using `Constants.ESTIMATION_ADDRESS`` as the from address. Using this mechanism the user can estimate how much gas is needed for a call to the OptimismPortal.

<https://docs.optimism.io/stack/protocol/deposit-flow>

If the `OptimismPortal::finalizeWithdrawalTransaction`` is called by the `Constants.ESTIMATION_ADDRESS``, it will revert unless the call to the target succeed. It is to ensure to estimate the gas, enough for the call to the target and succeed.

To estimate gas, the user would call `OptimismPortal::finalizeWithdrawalTransaction`` as the `ESTIMATION_ADDRESS`` and try to call with enough gas to make the call succeed. The call would succeed only when all the out going call from portal should succeed (`success`` and `l1mntSuccess``)

However, in the `OptimismPortal::finalizeWithdrawalTransaction``, the `l1mntSuccess`` is defaulting to be false. It will be set to true, only when `_tx.mntValue`` is non-zero and the transfer succeed.

Therefore, for the calls with zero `_tx.mntValue`` would not be able to estimate the gas using this mechanism.

```
bool l1mntSuccess = false;
 if (_tx.mntValue > 0) {
 l1mntSuccess = IERC20(L1_MNT_ADDRESS).transfer(_tx.target, _tx.mntValue);
 }
 bool success = SafeCall.callWithMinGas(_tx.target, _tx.gasLimit, _tx.ethValue, _tx.data);
```

**Hacker007:** When `Constants.ESTIMATION_ADDRESS` calls the function `finalizeWithdrawalTransaction()`, a user can estimate the exact gas cost for the sub-call.

```
bool success = SafeCall.callWithMinGas(_tx.target, _tx.gasLimit, _tx.ethValue, _tx.data);
```

However, if the `mntValue` of the meta transaction is zero, `l1mntSuccess` is false.

```
bool l1mntSuccess = false;
if (_tx.mntValue > 0) {
 l1mntSuccess = IERC20(L1_MNT_ADDRESS).transfer(_tx.target, _tx.mntValue);
}
```

If `l1mntSuccess` is false, any `finalizeWithdrawalTransaction` function calls performed by `ESTIMATION_ADDRESS` will be reverted whether the sub-call is successful or not, thus the user could not estimate the exact gas cost.

```
if ((success == false || l1mntSuccess == false) && tx.origin == Constants.ESTIMATION_ADDRESS) {
 revert("OptimismPortal: withdrawal failed");
}
```

## Recommendation

**lemonmon:** `l1mntSuccess` should default to true.

```
// OptimismPortal::finalizeWithdrawalTransaction
- bool l1mntSuccess = false;
+ bool l1mntSuccess = true;
 if (_tx.mntValue > 0) {
 l1mntSuccess = IERC20(L1_MNT_ADDRESS).transfer(_tx.target, _tx.mntValue);
 }
 bool success = SafeCall.callWithMinGas(_tx.target, _tx.gasLimit, _tx.ethValue, _tx.data);
```

**Hacker007:** Set the initial value of `l1mntSuccess` to true.

```
bool l1mntSuccess = true;
```

## Client Response

**lemonmon:** Fixed. <https://github.com/mantlenetworkio/mantle-v2/pull/105>

**Hacker007:** Fixed. <https://github.com/mantlenetworkio/mantle-v2/pull/105>

## MNT-36: Calling the `bridgeMNT` function in contract `L2StandardBridge` may result in a loss of funds

Category	Severity	Client Response	Contributor
Logical	Medium	Fixed	biakia

### Code Reference

- code/mantle-v2/packages/contracts-bedrock/contracts/L2/L2StandardBridge.sol#L518-L523

```
....
518: function bridgeMNT(
519: uint32 _minGasLimit,
520: bytes calldata _extraData
521:) public payable {
522: _initiateBridgeMNT(msg.sender, msg.sender, msg.value, _minGasLimit, _extraData);
523: }
```

### Description

**biakia:** The function `bridgeMNT` in contract `L2StandardBridge` is used to send `MNT` to a receiver's address on the L1 chain. It will use `msg.sender` as the value of the parameters `_from` and `_to`:

```
function bridgeMNT(
 uint32 _minGasLimit,
 bytes calldata _extraData
) public payable {
 _initiateBridgeMNT(msg.sender, msg.sender, msg.value, _minGasLimit, _extraData);
}
```

Since this function does not have a modifier `onlyEOA`, the `msg.sender` can be a contract address. However, the address of the same contract in ethereum and mantle are different. You should use `AddressAliasHelper` to convert address between L1 and L2:

```
library
```



# MNT-37:Bridge Insolvency Due to Rebasing, Deflationary or Fee-on-Transfer Tokens

Category	Severity	Client Response	Contributor
Logical	Medium	Acknowledged	rajatbeladiya

## Code Reference

- code/mantle-v2/packages/contracts-bedrock/contracts/L1/L1StandardBridge.sol#L693-L716
- code/mantle-v2/packages/contracts-bedrock/contracts/L1/L1StandardBridge.sol#L798-L845

```
....
693
```

```

: function finalizeBridgeERC20(
694: address _localToken,
695: address _remoteToken,
696: address _from,
697: address _to,
698: uint256 _amount,
699: bytes calldata _extraData
700:) public onlyOtherBridge override {
701: if (_isOptimismMintableERC20(_localToken)) {
702: require(
703: _isCorrectTokenPair(_localToken, _remoteToken),
704: "StandardBridge: wrong remote token for Optimism Mintable ERC20 local token"70
5:);
706: OptimismMintableERC20(_localToken).mint(_to, _amount);
707: } else {
708: deposits[_localToken][_remoteToken] = deposits[_localToken][_remoteToken] - _amount;
709: IERC20(_localToken).safeTransfer(_to, _amount);
710: }
711: }
712:
713: // Emit the correct events. By default this will be ERC20BridgeFinalized, but child714:
// contracts may override this function in order to emit legacy events as well.715: _emitERC
20BridgeFinalized(_localToken, _remoteToken, _from, _to, _amount, _extraData);
716: }

....

798: function _initiateBridgeERC20(
799: address _localToken,
800: address _remoteToken,
801: address _from,
802: address _to,
803: uint256 _amount,
804: uint32 _minGasLimit,
805: bytes memory _extraData
806:) internal override {
807: require(_localToken !=address(0) && _remoteToken != Predeploys.BVM_ETH,
808: "L1StandardBridge: BridgeERC20 do not support ETH bridging.");
809: require(_localToken != L1_MNT_ADDRESS && _remoteToken !=address(0x0),
810: "L1StandardBridge: BridgeERC20 do not support MNT bridging.");
811:
812: if (_isOptimismMintableERC20(_localToken)) {
813: require(
814: _isCorrectTokenPair(_localToken, _remoteToken),
815: "StandardBridge: wrong remote token for Optimism Mintable ERC20 local token"81
6:);
817: OptimismMintableERC20(_localToken).burn(_from, _amount);
818: } else {
819: IERC20(_localToken).safeTransferFrom(_from, address(this), _amount);
820: deposits[_localToken][_remoteToken] = deposits[_localToken][_remoteToken] + _amount;
821: }
822: }
823:
824: // Emit the correct events. By default this will be ERC20BridgeInitiated, but child825:
// contracts may override this function in order to emit legacy events as well.826: _emitERC
20BridgeInitiated(_localToken, _remoteToken, _from, _to, _amount, _extraData);
827: uint256 zeroMNTValue =0;
828: MESSENGER.sendMessage(
829: zeroMNTValue,

```

```

832: L2StandardBridge.finalizeBridgeERC20.selector,
833: // Because this call will be executed on the remote chain, we reverse the order
of834: // the remote and local token addresses relative to their order in the835:
// finalizeBridgeERC20 function.836: _remoteToken,
837: _localToken,
838: _from,
839: _to,
840: _amount,
841: _extraData
842:),
843: _minGasLimit
844:);
845: }

```

## Description

**rajatbeladiya:** The bridge does not calculate ``amount`` for deflationary or fee-on-transfer tokens correctly can lead to the bridge becoming insolvent.  
ERC20s are bridged using,

```

IERC20(_localToken).safeTransferFrom(_from, address(this), _amount);
deposits[_localToken][_remoteToken] = deposits[_localToken][_remoteToken] + _amount;

```

and minted using ``OptimismMintableERC20()``

```

OptimismMintableERC20(_localToken).mint(_to, _amount);

```

here, ``_amount`` is calculated based on users are sending tokens to bridge, not what exact tokens are received by the bridge. Fee-on-transfer tokens can be received less than expected.  
So malicious user can deposit less erc20 and redeem more erc20 than he should receive will lead to draining the bridge.

## Recommendation

**rajatbeladiya:** calculate the exact amount of bridge received.

```

uint balanceBefore = IERC20(_localToken).balance(address(this));
IERC20(_localToken).safeTransferFrom(_from, address(this), _amount);
uint balanceAfter = IERC20(_localToken).balance(address(this));

uint receivedAmount = balanceAfter - balanceBefore;

deposits[_localToken][_remoteToken] = deposits[_localToken][_remoteToken] + receivedAmount;

```

## Client Response

**rajatbeladiya:** Acknowledged. will fix later

## MNT-38:deposited funds by the contract can be lost

Category	Severity	Client Response	Contributor
Logical	Low	Acknowledged	rajatbeladiya

### Code Reference

- code/mantle-v2/packages/contracts-bedrock/contracts/L1/L1StandardBridge.sol#L150-L152

```
....
150: receive() external payable override onlyEOA {
151: _initiateETHDeposit(msg.sender, msg.sender, RECEIVE_DEFAULT_GAS_LIMIT, bytes(""));
152: }
```

- code/mantle-v2/packages/contracts-bedrock/contracts/L1/OptimismPortal.sol#L218-L220

```
....
218: receive() external payable {
219: depositTransaction(0, msg.sender, 0, RECEIVE_DEFAULT_GAS_LIMIT, false, bytes(""));
220: }
```

- code/mantle-v2/packages/contracts-bedrock/contracts/universal/StandardBridge.sol#L176-L186

```
....
176: modifier onlyEOA() {
177: require(
178: !Address.isContract(msg.sender),
179: "StandardBridge: function can only be called from an EOA"
180:);
181: require(
182: msg.sender==tx.origin,
183: "StandardBridge: msg sender must equal to tx origin"
184:);
185: _;
186: }
```

### Description

rajatbeladiya: ``solidity receive() external payable { depositTransaction(0, msg.sender, 0, RECEIVE\_DEFAULT\_GAS\_LIMIT, false, bytes("")); }

`OptimismPortal.sol's

`receive()` function should only be called by EOA only just like `L1StandardBridge.sol`'s `receive()` function because `receive()` called by the contract, deposited MNT would be lost due to address aliasing.

```
``solidityreceive() external payable override onlyEOA{
 _initiateETHDeposit(msg.sender, msg.sender, RECEIVE_DEFAULT_GAS_LIMIT, bytes(""));
}

modifier onlyEOA() {
 require(
 !Address.isContract(msg.sender),
 "StandardBridge: function can only be called from an EOA"
);
 require(
 msg.sender == tx.origin,
 "StandardBridge: msg sender must equal to tx origin"
);
 _;
}
```

## Recommendation

rajatbeladiya: add `onlyEOA` modifier to `OptimismPortal.sol#receive()` function

## Client Response

rajatbeladiya: Acknowledged. will fix later

## MNT-39: BVM\_ETH can be minted to address(0)

Category	Severity	Client Response	Contributor
Logical	Low	Acknowledged	biakia

### Code Reference

- code/op-geth/core/state\_transition.go#L667-L673

```
....
667: func (st *StateTransition) addBVMETHBalance(ethValue *big.Int) {
668: key := getBVMETHBalanceKey(*st.msg.To)
669: value := st.state.GetState(BVM_ETH_ADDR, key)
670: bal := value.Big()
671: bal = bal.Add(bal, ethValue)
672: st.state.SetState(BVM_ETH_ADDR, key, common.BigToHash(bal))
673: }
```

### Description

biakia: In `state_transition.go`, the function `addBVMETHBalance` is used to mint mantle's ETH to the `st.msg.To`:

```
func (st *StateTransition) addBVMETHBalance(ethValue *big.Int) {
 key := getBVMETHBalanceKey(*st.msg.To)
 value := st.state.GetState(BVM_ETH_ADDR, key)
 bal := value.Big()
 bal = bal.Add(bal, ethValue)
 st.state.SetState(BVM_ETH_ADDR, key, common.BigToHash(bal))
}
```

There is no check whether `st.msg.To` is `address(0)`. In `ERC20`, tokens are not allowed to be minted to `address(0)`. You can see OpenZeppelin's `ERC20` (<https://github.com/OpenZeppelin/openzeppelin-contracts/blob/master/contracts/token/ERC20/ERC20.sol#L226-L231>):

```
function _mint(address account, uint256 value) internal {
 if (account == address(0)) {
 revert ERC20InvalidReceiver(address(0));
 }
 _update(address(0), account, value);
}
```

As a result, the current mantle's `ETH` token is not compatible with the `ERC20` standard.

### Recommendation

biakia: Consider adding a check on the `st.msg.To` to prevent minting tokens to `address(0)`.

Acknowledged.Will fix next time.

## MNT-40: Use Two-Step Transfer Pattern for Access Controls

Category	Severity	Client Response	Contributor
Logical	Low	Acknowledged	thereksfour

### Code Reference

- code/mantle-v2/packages/contracts-bedrock/contracts/L2/GasPriceOracle.sol#L62-L77

```
....
62: function setOperator(address _operator) public onlyOwner {
63: address previousOperator = operator;
64: operator = _operator;
65: emit OperatorUpdated(previousOperator, operator);
66: }
67:
68: /**
69: * @dev Transfers ownership of the contract to a new account (`_owner`).
70: * Can only be called by the current owner.
71: */
72: function transferOwnership(address _owner) public onlyOwner {
73: require(_owner != address(0), "new owner is the zero address");
74: address previousOwner = owner;
75: owner = _owner;
76: emit OwnershipTransferred(previousOwner, owner);
77: }
```

### Description

**thereksfour:** Contracts implementing access control's, e.g. ``owner``, ``operator``, should consider implementing a Two-Step Transfer pattern.

Otherwise it's possible that the role mistakenly transfers ownership to the wrong address, resulting in a loss of the role.

```
function setOperator(address _operator) public onlyOwner {
 address previousOperator = operator;
 operator = _operator;
 emit OperatorUpdated(previousOperator, operator);
}

/**
 * @dev Transfers ownership of the contract to a new account (`_owner`).
 * Can only be called by the current owner.
 */function transferOwnership(address _owner) public onlyOwner {
 require(_owner !=address(0), "new owner is the zero address");
 address previousOwner = owner;
 owner = _owner;
 emit OwnershipTransferred(previousOwner, owner);
}
```

## Recommendation

thereksfour: Should be:

```
function setPendingOwner(address _newPendingOwner) external mixedAuth {
 require(_newPendingOwner != address(0));
 pendingOwner = _newPendingOwner;
}

function acceptOwnership() external {
 require(msg.sender == pendingOwner);
 owner = msg.sender;
 pendingOwner = address(0);
}
```

## Client Response

thereksfour: Acknowledged. will not fix as current solution is fine, not have to be 2 step transfer

## MNT-41:Unsafe type conversion

Category	Severity	Client Response	Contributor
Code Style	Low	Acknowledged	biakia

### Code Reference

- code/mantle-v2/op-batcher/batcher/driver\_da.go#L273-L275

```
....
273: datastoreTxData, err := l.dataStoreTxData(
274: l.DataLayrServiceManagerABI, uploadHeader, uint8(l.state.params.Duration), l.state.param
s.ReferenceBlockNumber, l.state.params.TotalOperatorsIndex,
275:)
```

### Description

biakia: In `driver_da.go`, the function `sendInitDataStoreTransaction` will call `dataStoreTxData` to build `TxD` `ata`:

```
dataStoreTxData, err := l.dataStoreTxData(
 l.DataLayrServiceManagerABI, uploadHeader, uint8(l.state.params.Duration), l.state.params.
ReferenceBlockNumber, l.state.params.TotalOperatorsIndex,
)
if err != nil {
 return nil, err
}
```

There is an unsafe type conversion here, the `l.state.params.Duration` is defined as a `uint32` in `StoreParams`:

```
type
```

```

StoreParams struct {
 ReferenceBlockNumber uint32
 TotalOperatorsIndex uint32
 OrigDataSize uint32// unique nonce for each data store
 NumTotal uint32// total number data node active on chain
 Quorum uint32// minimal amount of signatures from data node
 NumSys uint32// number of data node which contains the systematic chunk
 NumPar uint32// number of data node which contains the parity chunk
 Duration uint32// duration which data is stored// Data and Encoding
 KzgCommit []byte// elliptic curve kzg commitment
 LowDegreeProof []byte
 Degree uint32// degree of the polynomial
 TotalSize uint64// total size of the data
 Order []uint32// mapping for deciding the storer of each coded data chunk// Chain
 Fee *big.Int
 HeaderHash []byte
 Disperser []byte
}

```

However, in function `sendInitDataStoreTransaction`, it will be forced to convert to `uint8`. If `l.state.params.Duration` is greater than 255, the conversion will be wrong.

## Recommendation

biakia: Consider returning an error when `l.state.params.Duration` is greater than 255:

```

if l.state.params.Duration > 255 {
 return nil, errors.New("op-batcher l.state.params.Duration is greater than 255, it can't be converted to uint8")
}

```

## Client Response

biakia: Acknowledged. Will fix next time.

## MNT-42: The amount of gas to reserve for the caller after a sub call in `relayMessage` is smaller than expected

Category	Severity	Client Response	Contributor
Logical	Low	Fixed	biakia

### Code Reference

- code/mantle-v2/packages/contracts-bedrock/contracts/L1/L1CrossDomainMessenger.sol#L194-L197

```
....
194: if (
195: !SafeCall.hasMinGas(_minGasLimit, RELAY_RESERVED_GAS + RELAY_GAS_CHECK_BUFFER) ||
196: xDomainMsgSender != Constants.DEFAULT_L2_SENDER
197:) {
```

- code/mantle-v2/packages/contracts-bedrock/contracts/L2/L2CrossDomainMessenger.sol#L197-L200

```
....
197: if (
198: !SafeCall.hasMinGas(_minGasLimit, RELAY_RESERVED_GAS + RELAY_GAS_CHECK_BUFFER) ||
199: xDomainMsgSender != Constants.DEFAULT_L2_SENDER
200:) {
```

### Description

**biakia:** In contract `L1CrossDomainMessenger`, the function `relayMessage` will call `SafeCall.hasMinGas()` to check if there is enough gas left to perform the external call and finish the execution. The second param in `hasMinGas()` is used as amount of gas to reserve for the caller after the execution:

```
/*
```

```

*
* @notice Helper function to determine if there is sufficient gas remaining within the context
*
* to guarantee that the minimum gas requirement for a call will be met as well as
* optionally reserving a specified amount of gas for after the call has concluded.
* @param _minGas The minimum amount of gas that may be passed to the target context.
* @param _reservedGas Optional amount of gas to reserve for the caller after the execution
* of the target context.
* @return `true` if there is enough gas remaining to safely supply `_minGas` to the target
* context as well as reserve `_reservedGas` for the caller after the execution of
* the target context.
* @dev !!!!! FOOTGUN ALERT !!!!!
* 1.) The 40_000 base buffer is to account for the worst case of the dynamic cost of the
* `CALL` opcode's `address_access_cost`, `positive_value_cost`, and
* `value_to_empty_account_cost` factors with an added buffer of 5,700 gas. It is
* still possible to self-rekt by initiating a withdrawal with a minimum gas limit
* that does not account for the `memory_expansion_cost` & `code_execution_cost`
* factors of the dynamic cost of the `CALL` opcode.
* 2.) This function should *directly* precede the external call if possible. There is an
* added buffer to account for gas consumed between this check and the call, but it
* is only 5,700 gas.
* 3.) Because EIP-150 ensures that a maximum of 63/64ths of the remaining gas in the call
* frame may be passed to a subcontext, we need to ensure that the gas will not be
* truncated.
* 4.) Use wisely. This function is not a silver bullet.
*/
function hasMinGas(uint256 _minGas, uint256 _reservedGas) internal view returns (bool) {
 bool _hasMinGas;
 assembly {
 // Equation: gas × 63 ≥ minGas × 64 + 63(40_000 + reservedGas)
 _hasMinGas :=iszero(
 lt(mul(gas(), 63), add(mul(_minGas, 64), mul(add(40000, _reservedGas), 63)))
)
 }
 return _hasMinGas;
}

```

In **optimism**, after the execution of the external call, only the local variables are updated here(<https://github.com/ethereum-optimism/optimism/blob/develop/packages/contracts-bedrock/src/universal/CrossDomainMessenger.sol#L282-L304>):

```
xDomainMsgSender
```

```

= _sender;

 bool success = SafeCall.call(_target, gasleft() - RELAY_RESERVED_GAS, _value, _message);
 xDomainMsgSender = Constants.DEFAULT_L2_SENDER;

 if (success) {
 // This check is identical to one above, but it ensures that the same message cannot be
 // relayed twice, and adds a layer of protection against reentrancy.
 assert(successfulMessages[versionedHash] == false);
 successfulMessages[versionedHash] = true;
 emit RelayedMessage(versionedHash);
 } else {
 failedMessages[versionedHash] = true;
 emit FailedRelayedMessage(versionedHash);

 // Revert in this case if the transaction was triggered by the estimation address. This
 // should only be possible during gas estimation or we have bigger problems. Reverting here will
 // make the behavior of gas estimation change such that the gas limit computed will be the amount
 // required to relay the message, even if that amount is greater than the minimum gas limit specified
 // by the user.
 if (tx.origin == Constants.ESTIMATION_ADDRESS) {
 revert("CrossDomainMessenger: failed to relay message");
 }
 }
}

```

In mantle, however, there are two extra MNT token approves:

```
bool
```

```

mntSuccess =true;
 if (_mntValue!=0){
 mntSuccess = IERC20(L1_MNT_ADDRESS).approve(_target, _mntValue);
 }
 xDomainMsgSender = _sender;
 bool success = SafeCall.call(_target, gasleft() - RELAY_RESERVED_GAS, _ethValue, _messag
e);

 xDomainMsgSender = Constants.DEFAULT_L2_SENDER;
 if (_mntValue!=0){
 mntSuccess = IERC20(L1_MNT_ADDRESS).approve(_target, 0);
 }
 if (success && mntSuccess) {
 successfulMessages[versionedHash] =true;
 emit RelayedMessage(versionedHash);
 } else {
 failedMessages[versionedHash] =true;
 emit FailedRelayedMessage(versionedHash);

 // Revert in this case if the transaction was triggered by the estimation address. Thi
s// should only be possible during gas estimation or we have bigger problems. Reverting// here wil
l make the behavior of gas estimation change such that the gas limit// computed will be the amount
required to relay the message, even if that amount is// greater than the minimum gas limit specifi
ed by the user.if (tx.origin== Constants.ESTIMATION_ADDRESS) {
 revert("CrossDomainMessenger: failed to relay message");
 }
 }
}

```

As a result, more gas should be reserved after the external call in mantle, that means when calling `SafeCall.hasMinGas()`, the second param `_reservedGas` should be a little bit larger than now. The same issue exists in `L2CrossDomainMessenger` too.

## Recommendation

biakia: Consider making the second parameter `_reservedGas` a little bit larger.

## Client Response

biakia: Fixed. <https://github.com/mantlenetworkio/mantle-v2/pull/114>

## MNT-43:Overdraft check is incorrect

Category	Severity	Client Response	Contributor
Logical	Low	Acknowledged	csanuragjain

### Code Reference

- code/op-geth/core/txpool/txpool.go#L727-L737

```
....
727: userBalance = new(big.Int).Add(userBalance, sponsorCostSum)
728: sum := new(big.Int).Add(cost, list.totalcost)
729: if repl := list.txs.Get(tx.Nonce()); repl != nil {
730: // Deduct the cost of a transaction replaced by this
731: replL1Cost := repl.Cost()
732: if l1Cost := pool.l1CostFn(tx.RollupDataGas(), tx.IsDepositTx(), tx.To()); l1Cost !=
nil { // add rollup cost
733: replL1Cost = replL1Cost.Add(cost, l1Cost)
734: }
735: sum.Sub(sum, replL1Cost)
736: }
737: if userBalance.Cmp(sum) < 0 {
```

### Description

**csanuragjain:** It seems if a transaction is replaced then its associated validation is incorrect, causing overdraft validation to fail Eventually transaction will fail on execution due to insufficient funds but it would waste gas.

Steps

1. Observe validateTx function

```
list := pool.pending[from]
if list != nil { // Sender already has pending txs
 _, sponsorCostSum := pool.validateMetaTxList(list)
 userBalance = new(big.Int).Add(userBalance, sponsorCostSum)
 sum := new(big.Int).Add(cost, list.totalcost)
 if repl := list.txs.Get(tx.Nonce()); repl != nil {
 // Deduct the cost of a transaction replaced by this
 replL1Cost := repl.Cost()
 if l1Cost := pool.l1CostFn(tx.RollupDataGas(), tx.IsDepositTx()); l1Cost != nil { // a
dd rollup cost
 replL1Cost = replL1Cost.Add(cost, l1Cost)
 }
 sum.Sub(sum, replL1Cost)
 }
}
```

2. This first adds up all sponsor fee from pending transaction and then add this to current user balance and current transaction sponsor contribution

```
_, sponsorCostSum := pool.validateMetaTxList(list)
userBalance = new(big.Int).Add(userBalance, sponsorCostSum)
```

3. Notice this userBalance contains sponsor amount for transaction to be replaced as well
4. Now pending transaction total cost is added with current transaction cost. Then cost of transaction to be replaced is removed from this total cost

```
sum := new(big.Int).Add(cost, list.totalcost)
if repl := list.txs.Get(tx.Nonce()); repl != nil {
 // Deduct the cost of a transaction replaced by this
 replL1Cost := repl.Cost()
 if l1Cost := pool.l1CostFn(tx.RollupDataGas(), tx.IsDepositTx()); l1Cost != nil { // add rollup cost
 replL1Cost = replL1Cost.Add(cost, l1Cost)
 }
 sum.Sub(sum, replL1Cost)
}
```

5. Now check is made if user balance is > sum but it is incorrect check since user balance contains sponsor fee for replaced transaction whereas sum does not

## Recommendation

**csanuragjain:** Fixing this simply require to deduct sponsor fee of the replaced transaction from sponsorCostSum

## Client Response

**csanuragjain:** Acknowledged.Will fix next time

# MNT-44:No storage gap for upgradeable contract might lead to storage slot collision

Category	Severity	Client Response	Contributor
Code Style	Low	Acknowledged	rajatbeladiya

## Code Reference

- code/mantle-v2/packages/contracts-bedrock/contracts/L1/SystemConfig.sol#L4-L6

```
....
4: import {
5: OwnableUpgradeable
6: } from "@openzeppelin/contracts-upgradeable/access/OwnableUpgradeable.sol";
```

## Description

**rajatbeladiya:** When creating upgradeable contracts, it is important to include a storage gap. This allows for new state variables to be added in the future without causing compatibility issues with existing deployments. Without a storage gap, it becomes difficult to write new implementation code. In such cases, when new variables are added to the base contract, the variable in the child contract may be overwritten by the upgraded base contract. This can have unintended and serious consequences for the child contracts.

Refer: <https://docs.openzeppelin.com/upgrades-plugins/1.x/writing-upgradeable>  
[https://docs.openzeppelin.com/contracts/4.x/upgradeable#storage\\_gaps](https://docs.openzeppelin.com/contracts/4.x/upgradeable#storage_gaps)

## Recommendation

**rajatbeladiya:** add storage gaps to `SystemConfig.sol`

## Client Response

**rajatbeladiya:** Acknowledged. will fix later

# MNT-45:Logic Vulnerability in CalculateSponsorPercentAmount Function Allowing Zero or Undefined Transaction Costs

Category	Severity	Client Response	Contributor
Logical	Low	Fixed	BradMoonUESTC

## Code Reference

- code/op-geth/core/types/meta\_transaction.go#L60-L69
- code/op-geth/core/types/meta\_transaction.go#L92-L148

```
....
60
```

```

: func CalculateSponsorPercentAmount(mxParams *MetaTxParams, amount *big.Int) (*big.Int, *big.Int) {
61: if mxParams == nil {
62: return nil, nil
63: }
64: sponsorAmount :=new(big.Int).Div(
65: new(big.Int).Mul(amount, big.NewInt(int64(mxParams.SponsorPercent))),
66: big.NewInt(OneHundredPercent))
67: selfAmount :=new(big.Int).Sub(amount, sponsorAmount)
68: return sponsorAmount, selfAmount
69: }

....

92: func DecodeAndVerifyMetaTxParams(tx*Transaction) (*MetaTxParams, error) {
93: if tx.Type() != DynamicFeeTxType {
94: return nil, nil
95: }
96:
97: if mtp :=tx.metaTxParams.Load(); mtp != nil {
98: mtpCache, ok := mtp.(*MetaTxParamsCache)
99: if ok {
100: return mtpCache.metaTxParams, nil
101: }
102: }
103:
104: metaTxParams, err := DecodeMetaTxParams(tx.Data())
105: if err != nil {
106: return nil, err
107: }
108: // Not metaTx
109: if metaTxParams == nil {
110: tx.metaTxParams.Store(&MetaTxParamsCache{
111: metaTxParams: nil,
112: })
113: return nil, nil
114: }
115:
116: if metaTxParams.SponsorPercent > OneHundredPercent {
117: return nil, ErrInvalidSponsorPercent
118: }
119:
120: metaTxSignData :=&MetaTxSignData{
121: ChainID: tx.ChainId(),
122: Nonce: tx.Nonce(),
123: GasTipCap: tx.GasTipCap(),
124: GasFeeCap: tx.GasFeeCap(),
125: Gas: tx.Gas(),
126: To: tx.To(),
127: Value: tx.Value(),
128: Data: metaTxParams.Payload,
129: AccessList: tx.AccessList(),
130: ExpireHeight: metaTxParams.ExpireHeight,
131: SponsorPercent: metaTxParams.SponsorPercent,
132: }
133:
134: gasFeeSponsorSigner, err := recoverPlain(metaTxSignData.Hash(), metaTxParams.R, metaTxParam
s.S, metaTxParams.V, true)
135: if err != nil {
136: return nil, ErrInvalidGasFeeSponsorSig
137: }
138:
139: if gasFeeSponsorSigner != metaTxParams.GasFeeSponsor {

```

```

142:
143: tx.metaTxParams.Store(&MetaTxParamsCache{
144: metaTxParams: metaTxParams,
145: })
146:
147: return metaTxParams, nil
148: }

```

## Description

**BradMoonUESTC:** in the `CalculateSponsorPercentAmount` function of the meta transaction handling code, as seen in the specified Go code segment. This function is responsible for calculating the division of transaction costs between the initiator of a meta transaction and a sponsor based on the `SponsorPercent` parameter. Two distinct but related issues have been identified:

1. **Zero Transaction Cost for 100% Sponsorship:** When `SponsorPercent` is set to 100, the function unintentionally allows a scenario where the transaction cost for the sponsor is calculated as zero. This arises from the calculation `selfAmount := new(big.Int).Sub(amount, sponsorAmount)` yielding zero when the sponsorship covers the entirety of the transaction fee. This vulnerability can be exploited to send transactions without incurring any cost by setting the sponsor percentage to 100%, undermining the intended economic model and potentially leading to network abuse.
2. **Insufficient Validation of `SponsorPercent` and `GasFeeSponsor`:** The code lacks validation for the `GasFeeSponsor` address when `SponsorPercent` is 0, allowing transactions to proceed without a valid sponsor. This opens the door for submitting meta transactions with invalid or random `GasFeeSponsor` addresses, leading to undefined behavior or misuse of the transaction system without direct financial loss but potentially causing network congestion or other forms of denial-of-service attacks.

## Recommendation

**BradMoonUESTC:** ##### Code Fix for Zero Transaction Cost:

Limit `SponsorPercent` to a maximum of 99%, ensuring the initiator always bears some transaction cost.

```

func CalculateSponsorPercentAmount(mxParams *MetaTxParams, amount *big.Int) (*big.Int, *big.Int) {
 if mxParams == nil {
 return nil, nil
 }
 // Limit SponsorPercent to a maximum of 99%
 effectiveSponsorPercent := mxParams.SponsorPercent
 if effectiveSponsorPercent > 99 {
 effectiveSponsorPercent = 99
 }

 sponsorAmount := new(big.Int).Div(
 new(big.Int).Mul(amount, big.NewInt(int64(effectiveSponsorPercent))),
 big.NewInt(OneHundredPercent))
 selfAmount := new(big.Int).Sub(amount, sponsorAmount)
 return sponsorAmount, selfAmount
}

```

`GasFeeSponsor` correctly based on `SponsorPercent`.

```
func DecodeAndVerifyMetaTxParams(tx *Transaction) (*MetaTxParams, error) {
 // Existing logic remains unchanged

 // Additional validation for GasFeeSponsor
 if metaTxParams.SponsorPercent == 0 {
 if metaTxParams.GasFeeSponsor != (common.Address{}) {
 return nil, errors.New("GasFeeSponsor should not be set when SponsorPercent is 0")
 }
 } else {
 // Validate GasFeeSponsor signature for SponsorPercent > 0
 gasFeeSponsorSigner, err := recoverPlain(metaTxSignData.Hash(), metaTxParams.R, metaTxParams.S, metaTxParams.V, true)
 if err != nil {
 return nil, ErrInvalidGasFeeSponsorSig
 }
 if gasFeeSponsorSigner != metaTxParams.GasFeeSponsor {
 return nil, ErrGasFeeSponsorMismatch
 }
 }
 return metaTxParams, nil
}
```

## Client Response

BradMoonUESTC: Fixed. <https://github.com/mantlenetworkio/op-geth/pull/46/files>

## MNT-46:Lack of nil check for `StateTransition.msg.To`

Category	Severity	Client Response	Contributor
Logical	Low	Fixed	Yaodao

### Code Reference

- code/op-geth/core/state\_transition.go#L441-445

```
....
441: if ethValue := st.msg.ETHValue; ethValue != nil && ethValue.Cmp(big.NewInt(0)) != 0 {
442: st.addBVMETHBalance(ethValue)
443: st.addBVMETHTotalSupply(ethValue)
444: st.generateBVMETHMintEvent(*st.msg.To, ethValue)
445: }
```

### Description

Yaodao: According to the following codes, the `st.msg.ETHValue` will be added to the `st.msg.To` when the `st.msg.ETHValue` is not zero. However, whether the `st.msg.To` is nil is not checked.

```
if ethValue := st.msg.ETHValue; ethValue != nil && ethValue.Cmp(big.NewInt(0)) != 0 {
 st.addBVMETHBalance(ethValue)
 st.addBVMETHTotalSupply(ethValue)
 st.generateBVMETHMintEvent(*st.msg.To, ethValue)
}
```

As a result, the function will fail when the `st.msg.To` is nil.

Besides, the functions `addBVMETHBalance()` and `addBVMETHTotalSupply()` are used to mint the `BVM_ETH` to the address. The ERC20 token is not allowed to mint to the `address(0)`.

Reference: <https://github.com/OpenZeppelin/openzeppelin-contracts/blob/f8b1ddf5915dbe14fbeb21a42f5b8d654e358c58/contracts/token/ERC20/ERC20.sol#L226>

### Recommendation

Yaodao: Recommend checking whether `st.msg.To` is nil and adding the logic to deal with the nil.

### Client Response

Yaodao: Fixed. <https://github.com/mantlenetworkio/op-geth/pull/32>

## MNT-47:L2StandardBridge.bridgeMNT should add onlyEOA modifier

Category	Severity	Client Response	Contributor
Logical	Low	Fixed	thereksfour

### Code Reference

- code/mantle-v2/packages/contracts-bedrock/contracts/L2/L2StandardBridge.sol#L518-L523

```
....
518: function bridgeMNT(
519: uint32 _minGasLimit,
520: bytes calldata _extraData
521:) public payable {
522: _initiateBridgeMNT(msg.sender, msg.sender, msg.value, _minGasLimit, _extraData);
523: }
```

### Description

thereksfour: The `L2StandardBridge.bridgeETH()` function has the `onlyEOA` modifier because `bridgeETH()` specifies `msg.sender` as `to` address. Since the smart contract addresses in L1 and L2 are different, the address of EOA is the same. So when EOA calls `L2StandardBridge.bridgeETH()`, ETH will be sent to `msg.sender` in L1 correctly. When the smart contract calls `L2StandardBridge.bridgeETH()`, ETH will be incorrectly sent to `msg.sender - offset` in L1 (this is restricted by the `onlyEOA` modifier).

```
function bridgeETH(uint256 _value, uint32 _minGasLimit, bytes calldata _extraData) public only
EOA {
 _initiateBridgeETH(msg.sender, msg.sender, _value, _minGasLimit, _extraData);
}
```

However, `L2StandardBridge.bridgeMNT()` does not do this. If the smart contract accidentally calls `L2StandardBridge.bridgeMNT()` to bridge MNT, the bridged MNT will be sent to the incorrect address.

```
function bridgeMNT(
 uint32 _minGasLimit,
 bytes calldata _extraData
) public payable {
 _initiateBridgeMNT(msg.sender, msg.sender, msg.value, _minGasLimit, _extraData);
}
```

### Recommendation

thereksfour: Change `L2StandardBridge.bridgeMNT()` as follows

```
function bridgeMNT(
 uint32 _minGasLimit,
 bytes calldata _extraData
-) public payable {
+) public payable onlyEOA{
 _initiateBridgeMNT(msg.sender, msg.sender, msg.value, _minGasLimit, _extraData);
}
```

## Client Response

thereksfour: Fixed. <https://github.com/mantlenetworkio/mantle-v2/pull/106>

# MNT-48:Known issues with compiler versions used to compile the contracts (sol 0.8.15 & 0.6.0)

Category	Severity	Client Response	Contributor
Logical	Low	Acknowledged	rajatbeladiya

## Code Reference

- code/mantle-v2/packages/contracts-bedrock/contracts/L1/L1ERC721Bridge.sol#L2

```
....

2: pragma solidity 0.8.15;
```

- code/mantle-v2/packages/contracts-bedrock/contracts/L1/L2OutputOracle.sol#L2

```
....

2: pragma solidity 0.8.15;
```

- code/op-geth/contracts/checkpointoracle/contract/oracle.sol#L1

```
....

1: pragma solidity ^0.6.0;
```

## Description

**rajatbeladiya:** The contracts in scope have been compiled using older versions of the Solidity compiler (specifically, up to version 0.8.15). However, there are known issues that have been addressed in the latest compiler versions (> 0.8.15). Unfortunately, these addressed issues are still present in the 0.8.15 version used by the contracts. One known issue relates to the abi.encode family of functions, which can lead to vulnerabilities when certain conditions are met. The contracts in question fulfill the criteria for being vulnerable to this issue. For more technical details about this issue, you can refer to the following link: [Solidity Blog Post](#). Additionally, another known issue is specific to the 0.6.0 compiler version. This version is used in the oracle.sol contract from op-geth. The issue arises when attempting to push an empty admin address (zero) in a function call. It occurs due to how the admin list is loaded into memory and stored in storage. More information about this issue can be found here: [Solidity Blog Post](#).

## Recommendation

**rajatbeladiya:** To mitigate these known issues and benefit from the improvements and bug fixes in the Solidity compiler, it is strongly recommended to upgrade the compiler version to either 0.8.16 or 0.8.17 for all the contracts. By using the latest compiler version, the contracts can avoid potential vulnerabilities and ensure better code quality. Updating the compiler version is an essential step in maintaining the security and reliability of the contracts.

---

Acknowledged. will check again, for now remain the same compiler version as the op stack.

## MNT-49:Incorrect sponsor balance check

Category	Severity	Client Response	Contributor
Logical	Low	Fixed	biakia

### Code Reference

- code/op-geth/internal/ethapi/api.go#L1234-L1241
- code/op-geth/internal/ethapi/api.go#L1337-L1345

```
....

1234: if metaTxParams != nil {
1235: sponsorAmount, _ := types.CalculateSponsorPercentAmount(metaTxParams, feeCap)
1236: sponsorBalance := state.GetBalance(metaTxParams.GasFeeSponsor)
1237: if sponsorBalance.Cmp(sponsorAmount) <= 0 {
1238: return 0, types.ErrSponsorBalanceNotEnough
1239: }
1240: balance = new(big.Int).Add(balance, sponsorAmount)
1241: }

....

1337: if metaTxParams != nil {
1338: feeCap := new(big.Int).Mul(gasPriceForEstimate, big.NewInt(0).SetUint64(gasCap))
1339: sponsorAmount, _ := types.CalculateSponsorPercentAmount(metaTxParams, feeCap)
1340: sponsorBalance := state.GetBalance(metaTxParams.GasFeeSponsor)
1341: if sponsorBalance.Cmp(sponsorAmount) <= 0 {
1342: return 0, types.ErrSponsorBalanceNotEnough
1343: }
1344: balance = new(big.Int).Add(balance, sponsorAmount)
1345: }
```

### Description

biakia: In function `DoEstimateGas`, if the `MetaTxParams` exists, it will try to sponsor the user:

```
if metaTxParams != nil {
 sponsorAmount, _ := types.CalculateSponsorPercentAmount(metaTxParams, feeCap)
 sponsorBalance := state.GetBalance(metaTxParams.GasFeeSponsor)
 if sponsorBalance.Cmp(sponsorAmount) <= 0 {
 return 0, types.ErrSponsorBalanceNotEnough
 }
 balance = new(big.Int).Add(balance, sponsorAmount)
}
```

It will calculate the `sponsorAmount` and then compare it with the `sponsorBalance`. When the `sponsorBalance` is not enough, an error `ErrSponsorBalanceNotEnough` will be returned. The issue here is that when the `sponsorAmount == sponsorBalance` (which means `sponsorBalance.Cmp(sponsorAmount) == 0`), it will return an error too. The same issue exists in function `calculateGasWithAllowance`.

Consider following fix:

```
if metaTxParams != nil {
 sponsorAmount, _ := types.CalculateSponsorPercentAmount(metaTxParams, feeCap)
 sponsorBalance := state.GetBalance(metaTxParams.GasFeeSponsor)
 if sponsorBalance.Cmp(sponsorAmount) < 0 {
 return 0, types.ErrSponsorBalanceNotEnough
 }
 balance = new(big.Int).Add(balance, sponsorAmount)
}
```

```
if metaTxParams != nil {
 feeCap := new(big.Int).Mul(gasPriceForEstimate, big.NewInt(0).SetUint64(gasCap))
 sponsorAmount, _ := types.CalculateSponsorPercentAmount(metaTxParams, feeCap)
 sponsorBalance := state.GetBalance(metaTxParams.GasFeeSponsor)
 if sponsorBalance.Cmp(sponsorAmount) < 0 {
 return 0, types.ErrSponsorBalanceNotEnough
 }
 balance = new(big.Int).Add(balance, sponsorAmount)
}
```

## Client Response

biakia: Fixed. <https://github.com/mantlenetworkio/op-geth/pull/43>

## MNT-50: Incorrect return check for the available balance

Category	Severity	Client Response	Contributor
Logical	Low	Fixed	Yaodao

### Code Reference

- code/op-geth/internal/ethapi/api.go#L1242-1248
- code/op-geth/internal/ethapi/api.go#L1347-1352

```

....

1242: available := new(big.Int).Set(balance)
1243: if args.Value != nil {
1244: if args.Value.ToInt().Cmp(available) >= 0 {
1245: return 0, core.ErrInsufficientFundsForTransfer
1246: }
1247: available.Sub(available, args.Value.ToInt())
1248: }

....

1347: if args.Value != nil {
1348: if args.Value.ToInt().Cmp(available) >= 0 {
1349: return 0, core.ErrInsufficientFundsForTransfer
1350: }
1351: available.Sub(available, args.Value.ToInt())
1352: }
```

### Description

**Yaodao:** The following codes are used to compare the balance with the `args.Value` and then revert the function when the balance is insufficient.

```

available := new(big.Int).Set(balance)
if args.Value != nil {
 if args.Value.ToInt().Cmp(available) >= 0 {
 return 0, core.ErrInsufficientFundsForTransfer
 }
 available.Sub(available, args.Value.ToInt())
}
```

When the return value of function `Cmp()` is 0, the result is the two `big.Int` are equivalent and the balance is sufficient, which should not revert.

### Recommendation

**Yaodao:** Recommend updating the `if` condition as follows:.

```
if args.Value.ToInt().Cmp(available) > 0 {
 return 0, core.ErrInsufficientFundsForTransfer
}
```

## Client Response

Yaodao: Fixed. <https://github.com/mantlenetworkio/op-geth/pull/44>

# MNT-51:Import Contract not exist

Category	Severity	Client Response	Contributor
Logical	Low	Declined	BradMoonUESTC

## Code Reference

- code/mantle-v2/packages/contracts-bedrock/contracts/L2/CrossDomainOwnable.sol#L5

```
....
5: import { AddressAliasHelper } from "../vendor/AddressAliasHelper.sol";
```

## Description

**BradMoonUESTC:** The import contract: import { AddressAliasHelper } from "../vendor/AddressAliasHelper.sol"; not exist in the contracts-bedrock

## Recommendation

**BradMoonUESTC:** use `AddressAliasHelper`` in the `standard`` folder

## Client Response

**BradMoonUESTC:** Declined. exist in <https://github.com/mantlenetworkio/mantle-v2/blob/release%2Fv0.5.0/packages/contracts-bedrock/contracts/vendor/AddressAliasHelper.sol> valid issue in auditing version

## MNT-52:Gaps should be reserved for FeeVault

Category	Severity	Client Response	Contributor
Logical	Low	Acknowledged	thereksfour

### Code Reference

- code/mantle-v2/packages/contracts-bedrock/contracts/L2/BaseFeeVault.sol#L13

```
....
13: contract BaseFeeVault is FeeVault, Semver {
```

- code/mantle-v2/packages/contracts-bedrock/contracts/universal/FeeVault.sol#L12

```
....
12: abstract contract FeeVault {
```

### Description

**thereksfour:** There is no gaps reserved in ``FeeVault`` abstract contract.

The ``BaseFeeVault``, ``L1FeeVault``, and ``SequencerFeeVault`` that inherit from ``FeeVault`` don't have their own state variables yet, so it's possible to reserve gaps in ``FeeVault`` now.

If any contracts that inherit from ``FeeVault`` have their own state variables, then it will not be possible to add any new state variables to ``FeeVault``. This will break ``FeeVault`` contract's upgradability.

### Recommendation

**thereksfour:** It is recommended to reserve gaps in ``FeeVault``.

```
abstract contract FeeVault {
 /// @notice Enum representing where the FeeVault withdraws funds to.
 /// @custom:value L1 FeeVault withdraws funds to L1.
 /// @custom:value L2 FeeVault withdraws funds to L2.
 enum WithdrawalNetwork {
 L1,
 L2
 }

 /// @notice Minimum balance before a withdrawal can be triggered.
 uint256 public immutable MIN_WITHDRAWAL_AMOUNT;

 /// @notice Wallet that will receive the fees.
 address public immutable RECIPIENT;

 /// @notice Network which the RECIPIENT will receive fees on.
 WithdrawalNetwork public immutable WITHDRAWAL_NETWORK;

 /// @notice The minimum gas limit for the FeeVault withdrawal transaction.
 uint32 internal constant WITHDRAWAL_MIN_GAS = 35_000;

 /// @notice Total amount of wei processed by the contract.
 uint256 public totalProcessed;

 + uint256[49] private __gap;
```

## Client Response

thereksfour: Acknowledged. will fix later

# MNT-53:Flawed algorithm in tokenRatio function

Category	Severity	Client Response	Contributor
Logical	Low	Acknowledged	zircon

## Code Reference

- code/mantle-v2/gas-oracle/tokenratio/tokenratio.go#L121-L160
- code/mantle-v2/gas-oracle/tokenratio/tokenratio.go#L209-L220

```
....

121
```

```

: func (c *Client) tokenRatio() (float64, error) {
122: // Todo query token prices concurrent123: var mntPrices, ethPrices []float64
124: // get token price from oracle1(dex)125: mntPrice1, ethPrice1 := c.getTokenPricesFromUniswap()
126: mntPrices = append(mntPrices, mntPrice1)
127: ethPrices = append(ethPrices, ethPrice1)
128: log.Info("query prices from oracle1", "mntPrice", mntPrice1, "ethPrice", ethPrice1)
129:
130: // get token price from oracle2(cex)131: mntPrice2, ethPrice2 := c.getTokenPricesFromCex()
132: mntPrices = append(mntPrices, mntPrice2)
133: ethPrices = append(ethPrices, ethPrice2)
134: log.Info("query prices from oracle2", "mntPrice", mntPrice2, "ethPrice", ethPrice2)
135:
136: // get token price from oracle3(cex)137: // Todo add a third oracle to query prices138: mntPrice3, ethPrice3 := c.getTokenPricesFromCex()
139: mntPrices = append(mntPrices, mntPrice3)
140: ethPrices = append(ethPrices, ethPrice3)
141: log.Info("query prices from oracle3", "mntPrice", mntPrice3, "ethPrice", ethPrice3)
142:
143: // median price for eth & mnt144: medianMNTPrice := getMedian(mntPrices)
145: medianETHPrice := getMedian(ethPrices)
146:
147: // determine mnt_price, eth_price148: mntPrice := c.determineMNTPrice(medianMNTPrice)
149: ethPrice := c.determineETHPrice(medianETHPrice)
150: log.Info("prices after determine", "mntPrice", mntPrice, "ethPrice", ethPrice)
151:
152: // calculate ratio153: ratio := c.determineTokenRatio(mntPrice, ethPrice)
154:
155: c.lastRatio = ratio
156: c.lastEthPrice = ethPrice
157: c.lastMntPrice = mntPrice
158:
159: return ratio, nil
160: }

....

209: func getMedian(nums []float64) float64 {
210: nonZeros := make([]float64, 0)
211: for_, num := range nums {
212: if num != 0 {
213: nonZeros = append(nonZeros, num)
214: }
215: }
216: sort.Float64s(nonZeros)
217: if len(nonZeros) == 0 {
218: return 0
219: }
220: return nonZeros[len(nonZeros)/2]

```

## Description

**zircon:** The `tokenRatio` function utilizes a flawed algorithm to fetch token prices from an oracle three times and ultimately selects the median price using the `getMediant` function. However, a critical flaw exists in this algorithm. By making two calls to `getTokenPricesFromCex`, if the price from the Cex oracle is manipulated to an excessively high value, both retrieved prices will significantly exceed those obtained from Uniswap. Consequently, the `getMediant` function will inevitably select the lower of the two manipulated Cex prices as the median, since it lies in the middle. This manipulated price poses a serious risk as it does not accurately reflect the market value. Suppose the

`getMediant` would be 60, reflecting the manipulated Cex price.

## Recommendation

**zircon:** It is advised to refrain from making multiple requests to the Cex oracle. Instead, in cases where significant discrepancies are observed among prices obtained from multiple oracles, the function should either raise an error or revert to using the last known accurate price.

## Client Response

**zircon:** Acknowledged. Will fix next time.

## MNT-54:Errors are not returned in the function `wrapUpdateTokenRatio`

Category	Severity	Client Response	Contributor
Code Style	Low	Fixed	biakia

### Code Reference

- [code/mantle-v2/gas-oracle/oracle/token\\_ratio.go#L22-L60](#)

```
....

22: func wrapUpdateTokenRatio(l1Backend bind.ContractTransactor, l2Backend DeployContractBackend, to
kenRatio *tokenratio.Client, cfg *Config) (func() error, error) {
23: if cfg.l2ChainID == nil {
24: return nil, errNoChainID
25: }
26:
27: var opts *bind.TransactOpts
28: var err error
29: if !cfg.EnableHsm {
30: if cfg.privateKey == nil {
31: return nil, errNoPrivateKey
32: }
33: if cfg.l2ChainID == nil {
34: return nil, errNoChainID
35: }
36:
37: opts, err = bind.NewKeyedTransactorWithChainID(cfg.privateKey, cfg.l2ChainID)
38: if err != nil {
39: return nil, err
40: }
41: } else {
42: seqBytes, err := hex.DecodeString(cfg.HsmCreden)
43: if err != nil {
44: log.Crit("gasoracle", "decode hsm creden fail", err.Error())
45: }
46: apikey := option.WithCredentialsJSON(seqBytes)
47: client, err := kms.NewKeyManagementClient(context.Background(), apikey)
48: if err != nil {
49: log.Crit("gasoracle", "create signer error", err.Error())
50: }
51: mk := &bsscore.ManagedKey{
52: KeyName: cfg.HsmAPIName,
53: EthereumAddr: common.HexToAddress(cfg.HsmAddress),
54: Gclient: client,
55: }
56: opts, err = mk.NewEthereumTransactorWithChainID(context.Background(), cfg.l2ChainID)
57: if err != nil {
58: log.Crit("gasoracle", "create signer error", err.Error())
59: }
60: }
```

### Description

In function `wrapUpdateTokenRatio`, if the `cfg.EnableHsm` is true, it will use google's kms client to establish a connection to ethereum:

```
else {
 seqBytes, err := hex.DecodeString(cfg.HsmCreden)
 if err != nil {
 log.Crit("gasoracle", "decode hsm creden fail", err.Error())
 }
 apikey := option.WithCredentialsJSON(seqBytes)
 client, err := kms.NewKeyManagementClient(context.Background(), apikey)
 if err != nil {
 log.Crit("gasoracle", "create signer error", err.Error())
 }
 mk := &bsscore.ManagedKey{
 KeyName: cfg.HsmAPIName,
 EthereumAddr: common.HexToAddress(cfg.HsmAddress),
 Gclient: client,
 }
 opts, err = mk.NewEthereumTransactorWithChainID(context.Background(), cfg.l2ChainID)
 if err != nil {
 log.Crit("gasoracle", "create signer error", err.Error())
 }
}
```

When errors happen, this function will call `log.Crit` and it will log errors and panic. According to the definition of the function `wrapUpdateTokenRatio`, any error needs to be returned as soon as it occurs:

```
wrapUpdateTokenRatio(l1Backend bind.ContractTransactor, l2Backend DeployContractBackend, tokenRatio *tokenratio.Client, cfg *Config) (func() error, error)
```

While panic inside a function is not a problem at the moment, we still recommend returning the error to an external caller for processing, as internal panic rather than returning the error may lead to unexpected side effects when future code upgrades occur.

## Recommendation

**biakia:** Consider returning errors directly:

```
else
```

```
{
 seqBytes, err := hex.DecodeString(cfg.HsmCreden)
 if err != nil {
 log.Warn("gasoracle", "decode hsm creden fail", err.Error())
 return nil, err
 }
 apikey := option.WithCredentialsJSON(seqBytes)
 client, err := kms.NewKeyManagementClient(context.Background(), apikey)
 if err != nil {
 log.Warn("gasoracle", "create signer error", err.Error())
 return nil, err
 }
 mk := &bsscore.ManagedKey{
 KeyName: cfg.HsmAPIName,
 EthereumAddr: common.HexToAddress(cfg.HsmAddress),
 Gclient: client,
 }
 opts, err = mk.NewEthereumTransactorWithChainID(context.Background(), cfg.l2ChainID)
 if err != nil {
 log.Warn("gasoracle", "create signer error", err.Error())
 return nil, err
 }
}
```

## Client Response

biakia: Fixed. <https://github.com/mantlenetworkio/mantle-v2/pull/113/>

## Disclaimer

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Invoices, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Invoice. This report provided in connection with the services set forth in the Invoices shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Invoice. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without Secure3's prior written consent in each instance.

This report is not an "endorsement" or "disapproval" of any particular project or team. This report is not an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Secure3 to perform a security assessment. This report does not provide any warranty or guarantee of free of bug of codes analyzed, nor do they provide any indication of the technologies, business model or legal compliancy.

This report should not be used in any way to make decisions around investment or involvement with any particular project. Instead, it represents an extensive assessing process intending to help our customers increase the quality of their code and high-level consistency of implementation and business model, while reducing the risk presented by cryptographic tokens and blockchain technology.

Secure3's position on the final decisions over blockchain technologies and corresponding associated transactions is that each company and individual are responsible for their own due diligence and continuous security.

The assessment services provided by Secure3 is subject to dependencies and under continuing development. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

